ДЕТЕКТИВНАЯ ДОСКА (Detective Board)

Локальный офлайн нейро-символический следователь по рассказам о Шерлоке Холмсе

ПРОЕКТ ДЛЯ ВСЕХ

Что делает «Детективная доска» и что лежит в каждом файле

Авторы:
Верховых Егор
Стрижевский Андрей
Парамонов Даниил
Приц Артём

Состояние проекта: ветка main, коммит 16122fc, версия 0.0.1
Дата составления: 30.10.2026. Черновик, проверен только автором-ассистентом

2026

СОДЕРЖАНИЕ

1 ЧТО ДЕЛАЕТ ПРОЕКТ И ЧЕМ ОН ПОЛЕЗЕН....5
2 КАК ЭТО РАБОТАЕТ ПО ШАГАМ....5
2.1 Шаг 0. Подготовка (делается заранее)....5
2.2 Шаг 1. Доска из карточек и ниток....6
2.3 Шаг 2. Ответ на вопрос....7
2.4 Три правила....7
2.5 Откуда взят каждый факт....8
3 ЧТО ПОЛУЧИЛОСЬ (ЦИФРЫ ИЗ ОТЧЁТОВ)....8
4 ЧЕГО ПРОЕКТ ПОКА НЕ УМЕЕТ....9
5 СЛОВАРИК....10
6 КАК ЗАПУСТИТЬ....10
7 ФАЙЛЫ ПРОЕКТА: КАК ЧИТАТЬ ЭТУ ЧАСТЬ....11
8 ФАЙЛЫ В КОРНЕ ПРОЕКТА....11
9 ПАПКА .GITHUB/....12
9.1 .github/workflows/....12
10 ПАПКА .CLAUDE/....13
10.1 .claude/agents/....13
11 ПАПКА DATA/....16
11.1 data/....16
11.2 data/clean/....16
11.3 data/curated/....17
11.4 data/extraction/....18
12 ПАПКА PROMPTS/....18
12.1 prompts/extract/....18
13 ПАПКА SRC/....19
13.1 src/detective_board/....19
13.2 src/detective_board/agent/....19
13.3 src/detective_board/api/....22
13.4 src/detective_board/extract/....23
13.5 src/detective_board/graph/....25
13.6 src/detective_board/index/....26
13.7 src/detective_board/mcp/....26
13.8 src/detective_board/retrieval/....27
13.9 src/detective_board/symbolic/....28
13.10 src/detective_board/symbolic/rules/....28
14 ПАПКА TESTS/....29
14.1 tests/....29
14.2 tests/agent/....31

14.3 tests/api/ 33
14.4 tests/curated/ 34
14.5 tests/eval/ 35
14.6 tests/extract/ 36
14.7 tests/fixtures/symbolic/ 37
14.8 tests/graph/ 37
14.9 tests/index/ 38
14.10 tests/mcp/ 38
14.11 tests/presence/ 39
14.12 tests/retrieval/ 40
14.13 tests/symbolic/ 41
14.14 tests/web_js/ 42
15 ПАПКА EVAL/ 43
15.1 eval/ 43
15.2 eval/extraction_gold/ 45
15.3 eval/golden/ 47
15.4 eval/pre_reveal/ 47
16 ПАПКА SCRIPTS/ 48
16.1 scripts/ 48
16.2 scripts/corpus/ 50
16.3 scripts/eval/ 50
16.4 scripts/spike/ 51
17 ПАПКА WEB/ 53
17.1 web/css/ 53
17.2 web/ 53
17.3 web/js/ 53
17.4 web/replay/ 55
17.5 web/vendor/ 56
17.6 web/vendor/fonts/ 56
18 ПАПКА DOCS/ 57
18.1 docs/ 57
18.2 docs/acceptance/ 58
18.3 docs/adr/ 59
18.4 docs/architecture/ 60
18.5 docs/demo/ 60
18.6 docs/design/ 61
18.7 docs/design/screens/ 61
18.8 docs/perf/ 62
18.9 docs/slides/ 62

18.10 docs/spike/ 63
18.11 docs/spike/results/_aborted/ 63
18.12 docs/spike/results/ 64
18.13 docs/spike/results/entity-answer-runs/ 71
18.14 docs/team/ 73
18.15 docs/team/plans/ 74
19 УКАЗАТЕЛЬ ФАЙЛОВ 75
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ 84

1 ЧТО ДЕЛАЕТ ПРОЕКТ И ЧЕМ ОН ПОЛЕЗЕН

«Детективная доска» – программа, которая отвечает на вопросы о 20 рассказах про Шерлока Холмса. Главное правило: ответ должен опираться на текст. К каждому ответу программа прикладывает цитату из рассказа слово в слово и говорит, откуда она взята.

Представьте доску детектива. На ней карточки: люди, места, события. Между карточками натянуты красные нитки. К каждой карточке приколота вырезка с точной цитатой – доказательство. Программа сама собирает такую доску из рассказов и потом отвечает по ней.

Чем это полезно.

- Нужное место в тексте находится за секунды: рассказы разрезаны на 596 кусочков, и программа ищет по ним.
- Видно, как она пришла к ответу: каждый шаг и каждая карточка показаны на экране.
- Если в тексте ответа нет, программа так и говорит – «не нашла». Не выдумывает.
- Логические проверки («человек не может быть в двух местах сразу») делают правила, а не нейросеть. Правило показывает, из каких карточек оно сделало вывод.
- Всё работает на обычном ноутбуке без интернета.

Чего не стоит ждать. Это учебный проект. Он не заменяет чтение рассказа и не умеет надёжно называть виновного. Подробно – в разделе 4.

2 КАК ЭТО РАБОТАЕТ ПО ШАГАМ

2.1 Шаг 0. Подготовка (делается заранее)

Программа не читает рассказы заново на каждый вопрос. Сначала один раз готовится «доска».

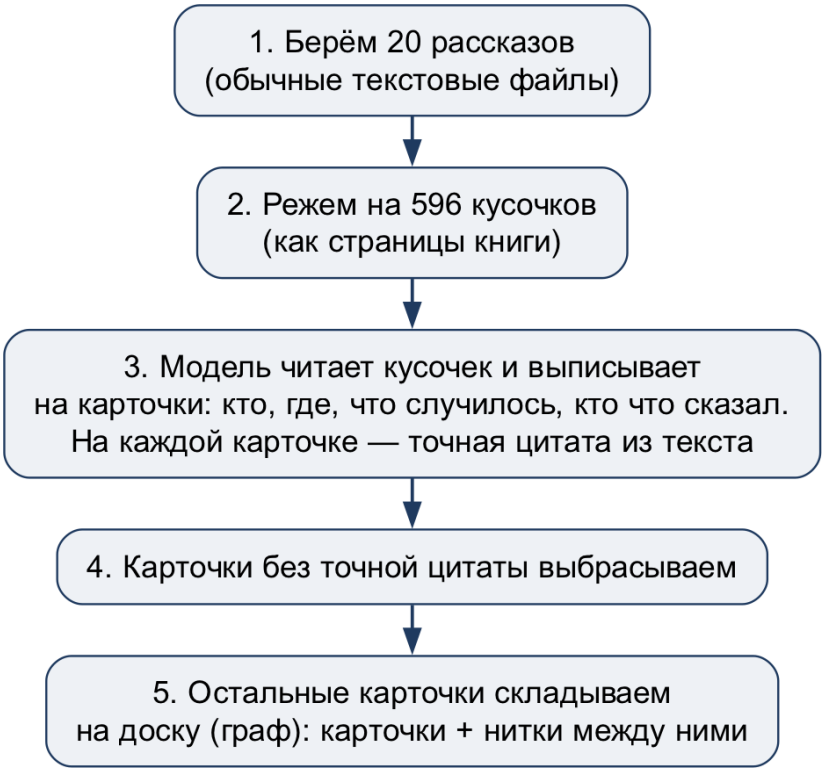


Рисунок 1 – Подготовка: от рассказов к доске

Читает кусочки языковая модель – программа, которая понимает текст. Она ошибается, поэтому каждую карточку обязательно сопровождает цитата. Если такой цитаты нет в кусочке слово в слово, карточку выбрасывают. На всех 20 рассказах выбросили 111 карточек из-за ненайденной цитаты и 14 как повторы (`docs/spike/results/graph-sanity-20261028.md`). Работа модели на видеокарте сервера заняла около 3 секунд на кусочек; на ноутбуке без видеокарты – 3–4 минуты, поэтому готовую доску везут на ноутбук файлом.

2.2 Шаг 1. Доска из карточек и ниток

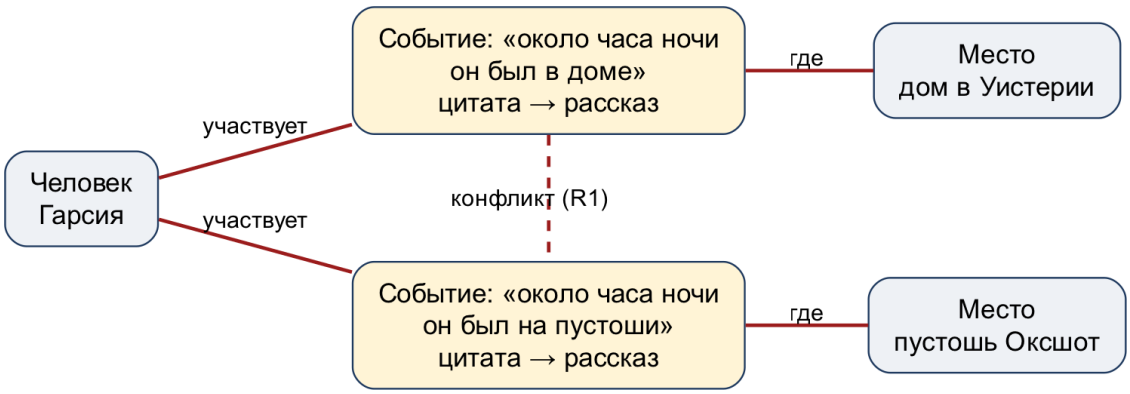


Рисунок 2 – Две карточки-события об одном человеке (упрощённо)

Карточки бывают пяти видов: человек, место, предмет, событие, высказывание. На рисунке человек участвует в двух событиях, которые происходят в одно время в разных местах. Это конфликт, и его находит правило R1 (ниже). В базе всего 5278 таких карточек до добавления проверенных вручную.

2.3 Шаг 2. Ответ на вопрос

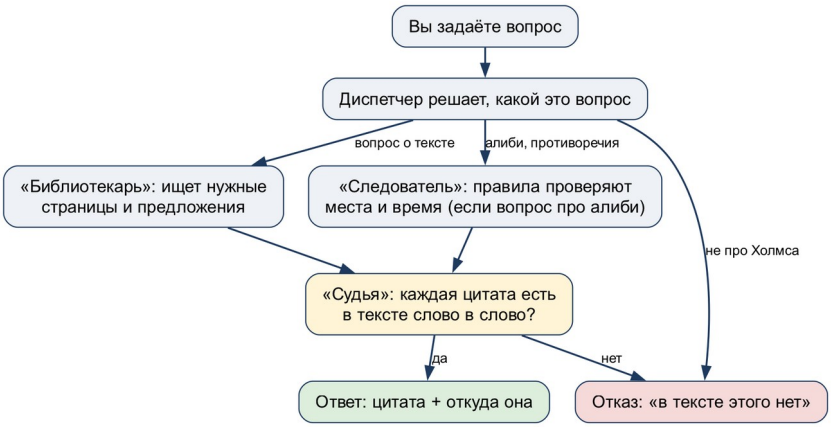


Рисунок 3 – Путь вопроса до ответа или отказа

– **Диспетчер** смотрит на слова вопроса и выбирает путь. Он работает по простым правилам, нейросеть путь не выбирает.
– **«Библиотекарь»** ищет нужные кусочки и предложения двумя способами сразу: по словам и по смыслу.
– **«Следователь»** – это правила R1–R3. Они берут карточки с доски и проверяют их без участия модели.
– **«Судья»** проверяет каждую цитату: она должна найтись в тексте слово в слово. Не нашлась – вместо ответа будет отказ.
– Для обычных вопросов модель не пишет ответ сама. Она только выбирает номера подходящих предложений, а ответом становятся эти предложения без изменений.

2.4 Три правила

Таблица 1 – Правила проверки

Правило	Простыми словами	Пример
R1	Человек не может быть в двух разных местах в одно и то же время.	В 21:00 он в Лондоне и в 21:00 в Париже – так не бывает.

Правило	Простыми словами	Пример
R2	События идут по порядку и не замыкаются в круг.	A раньше B, B раньше C, а C раньше A – ошибка где-то в данных.
R3	Слова свидетеля не должны спорить с установленным фактом.	Свидетель говорит «я был дома», а текст ясно говорит, что он был в другом месте.

Если данных не хватает, правило отвечает «неизвестно». Оно не говорит «всё согласовано», потому что отсутствие фактов – не доказательство.

2.5 Откуда взят каждый факт

Таблица 2 – Метки на карточках и выводах

Метка	Кто сделал	Как относиться
extracted	модель	Может быть ошибкой. Рядом всегда цитата.
derived	правило	Следует из названных карточек. Есть объяснение «почему».
conditional	правило	Вывод опирается на сомнительную карточку: «если она верна, то…».
curated	человек	Карточку для демонстрации ввели и проверили люди. Так и подписано на экране.

Важно: вывод правила верен для карточек на доске, но не обязательно для самого рассказа. Если модель неверно прочитала время, правило сделает аккуратный, но неверный вывод. Поэтому цитата рядом с каждым входом.

3 ЧТО ПОЛУЧИЛОСЬ (ЦИФРЫ ИЗ ОТЧЁТОВ)

– Корпус: 20 рассказов, 596 кусочков, 11 018 предложений; 5278 карточек (`docs/spike/results/graph-sanity-20261028.md`).
– Тест на 30 вопросах (один запуск, `docs/spike/results/eval-2026-10-28-test-7b7b197-7857839ed9be-175321.md`): правильное поведение у «голой» модели – 3 из 30, у простого поиска – 11 из 30, у агента – 13 из 30. «Правильное поведение» – на вопрос с ответом ответили по делу, на вопрос вне рассказов отказали.
– На 6 вопросах, ответа на которые в рассказах нет, агент отказал во всех 6. Но он отказал и на 6 из 24 вопросов, на которые ответ был.
– Время ответа на ноутбуке без видеокарты: в среднем около 28 секунд (от 19 до 44) – `docs/perf/laptop-20261029.md`.

– Доска на 200 карточках: время прорисовки при масштабировании снизили с 13,9 до 2,8 секунды (PR #30; замер в браузере без окна на компьютере разработчика, на ноутбуке не мерили).
– Автотесты: 986 проходят за 10 секунд (проверено при составлении этого документа).

4 ЧЕГО ПРОЕКТ ПОКА НЕ УМЕЕТ

– **Не угадывает виновного надёжно.** Опытный режим («матрица подозреваемых») лежит в отдельной ветке `feat/suspects-matrix`, в основном коде его нет. На тесте из 8 рассказов ни один метод не победил простые приёмы вроде «кто чаще упомянут». На 8 рассказах для разработки лучший из примерно 35 вариантов угадал 4 из 8, но он отбирался на этих же рассказах, цифра завышена. На 6 совсем новых рассказах (отец Браун) тот же метод назвал виновного первым 2 раза из 6, ровно как «самый упоминаемый»; интервал 0,10 –0,70; один оправданный человек был назван первым. Не обвинять невиновного получается только ценой отказа отвечать. Кто виновен и где обрезать текст, отметил ИИ – нужна проверка человеком (`docs/spike/results/suspects-20261030.md` на той ветке).
– **Сама почти не находит противоречия.** Из 12 противоречий в контрольном наборе правила нашли 0. На 19 из 20 рассказов не нашли ничего. Причина: из текста модель редко вытаскивает пары «кто + где + когда» (у 0,7 % высказываний). Единственная находка сделана на восьми карточках, введённых вручную.
– **Слабо запоминает, кто где был.** Целевой проход на 159 кусочках оставил 16 фактов (большая модель) и 7 (маленькая); ни у одного нет дня рассказа. Эталон для проверки – всего 4 строгих факта, поэтому качество оценить нельзя (`docs/spike/results/presence-pass-server-20261029.md`).

– **Не лучше простого поиска.** Агент 13 из 30 против 11 из 30 – разница не значима. На другой выборке (dev, 34 вопроса) агент хуже: 12 против 15.
– **Эталон не проверен человеком.** Вопросы и правильные ответы для измерений написал ИИ.
– **Только английские рассказы.** Русский вопрос переводится моделью; качество для русского отдельно не измерено.
– **Одна проверка установки.** Офлайн-пакет проверен на одном ноутбуке, а не на двух. Скорость измерена не в чистых условиях (шли другие программы).

5 СЛОВАРИК

Слово	Значит
Корпус	Набор текстов, с которым работает программа: 20 рассказов.
Кусочек (чанк)	Часть рассказа в несколько абзацев; программа работает с кусочками.
Языковая модель	Программа, которая читает и пишет текст. Работает на ноутбуке через Ollama.
Граф	Доска из карточек и ниток между ними.
Индекс	Список для быстрого поиска, как указатель в конце книги.
Правило	Короткая проверка без нейросети: результат всегда один и тот же.
MCP	Договорённость о том, как агент вызывает инструменты (например, правила).
SSE	Способ, которым сервер по ходу работы присылает в окно браузера шаги агента.
SQLite	Простая база данных в одном файле.
Тест	Автоматическая проверка, что код работает как задумано.

6 КАК ЗАПУСТИТЬ

```
# установить зависимости (нужен интернет)
just setup
# автотесты
just test
# открыть доску на http://127.0.0.1:8000
just run
# то же без модели, на заготовленных данных
just run --fake
```

Подробности: документ «Руководство системного программиста» из этого же комплекта.

7 ФАЙЛЫ ПРОЕКТА: КАК ЧИТАТЬ ЭТУ ЧАСТЬ

Ниже – все 536 файлов, которые Git отслеживает в проекте на коммите 16122fc, кроме каталога `docs/gost/` (это сам комплект документации, его перечень – `docs/gost/README.md`). Файлы сгруппированы по папкам: сначала короткое введение в папку, потом файлы. У каждого файла свой номер в квадратных скобках; по нему и по указателю в конце его легко найти. Однотипные файлы (тексты рассказов, записи прогонов, шрифты, дампы результатов) собраны в таблицы. Названия файлов – ссылки на GitHub.

Что сюда не вошло. Файлы, которые создаются при работе и в Git не хранятся: базы индексов и графа (`data/index/*.sqlite`, `data/graph/graph.sqlite`), кэши извлечения (`data/cache/`), виртуальное окружение `.venv/`. Они собираются командами из документа «Руководство системного программиста» или приходят в офлайн-архиве.

8 ФАЙЛЫ В КОРНЕ ПРОЕКТА

[1] `.gitignore`
Список того, что git не должен отслеживать (636 байт). Туда входят виртуальное окружение, кэши инструментов, файлы .env, тяжёлые производные данные (data/cache, data/index, data/graph, *.sqlite, *.npy), веса моделей (*.gguf, *.safetensors) и выходы прогонов (eval/runs/). Именно поэтому два sqlite-кэша в data/extraction/ лежат в папке как исключение из общего правила.

[2] `.pre-commit-config.yaml`
Настройка pre-commit — проверок, которые запускаются перед каждым коммитом (638 байт). Включает ruff (линтер и форматтер Python), мелкие проверки (пробелы, YAML/TOML, конфликты слияния), запрет файлов больше 500 КБ и поиск приватных ключей. Ставится командой `uvx pre-commit install`, необязательна.

[3] `.python-version`
Один файл из одной строки: `3.12`. Говорит uv и другим инструментам, какую версию Python использовать; совпадает с PYTHON_VERSION в CI.

[4] `CONTRIBUTING.md`
Правила работы в команде (84 строки, 4,4 КБ): четыре участника с Claude Code работают параллельно на отдельных ветках. Описывает установку, имена веток (feat/…, fix/…), формат коммитов Conventional Commits без Co-Authored-By, порядок pull request с ревью агентом code-reviewer и человеком, зоны владения из .claude/agents и список того, что нельзя коммитить (сырой корпус, индексы, *.sqlite, ключи). Упоминает срок слияния в main 2026-10-27.

[5] `README.md`
Главная страница проекта (87 строк, 7,4 КБ). Объясняет идею: локальная нейро-символьная система (модель извлекает факты, правила на чистом Python их проверяют) над 20 рассказами о Холмсе, ответы с цитатами и отказом, если в корпусе ответа нет. Содержит метки происхождения (extracted/derived/conditional), правила R1–R3, диаграмму архитектуры, команды `just` … и описание запуска оценки четырёх конфигураций (bare, vector, vector+rerank, hybrid+graph).

[6] `THIRD_PARTY_NOTICES.md`
Перечень сторонних компонентов и их лицензий (101 строка, 6,3 КБ), проверен 2026-10-26. В таблице — Python-зависимости из uv.lock с версиями и лицензиями (например pydantic 2.14.1 MIT, starlette 1.9.0 BSD-3-Clause), ниже — другие вендорные части. Не заменяет сами тексты лицензий, а указывает, что не удалось проверить.

[7] `justfile`
Набор коротких команд проекта для программы just (93 строки, 3,7 КБ). Рецепты: `setup` (создать окружение), `test`/`test-cov`/`test-live` (тесты без и с Ollama), `lint`/`fmt`/`typecheck`, `index` (построить data/index/db.sqlite с FTS5 и векторами bge-m3, около 22 минут), `sentences`, `graph` (собрать data/graph/graph.sqlite из кэша извлечения), `eval`, `pre-reveal`, `run` (веб-интерфейс на 127.0.0.1:8000, `--fake` — без Ollama), `bench`, `test-web`.

[8] `pyproject.toml`
Главный конфиг Python-проекта detective-board (74 строки). Требует Python >=3.12; рабочие зависимости: pydantic, sqlite-vec 0.1.9, mcp, starlette 1.9.0, uvicorn 0.54.0. Содержит группы dev/e2e и настройки ruff, pyright, pytest и coverage.

[9] `uv.lock`
Файл блокировки версий, который создаёт менеджер uv (около 49 пакетов, формат version = 1). Фиксирует точные версии и хэши всех зависимостей, чтобы установка везде была одинаковой; CI ставит по нему через `uv sync --locked`. Руками не правится.

9 ПАПКА .GITHUB/

9.1 .github/workflows/

Два сценария GitHub Actions: ci.yml проверяет каждый push и pull request (стиль, типы, тесты, секреты, короткая оценка), release.yml собирает релиз по тегу.

[10] `.github/workflows/ci.yml`
Сценарий автоматической проверки GitHub Actions (CI — запуск проверок при каждом изменении), около 5,9 КБ. Срабатывает на push в main и на pull request; запускает задачи: ruff (проверка стиля), pyright (проверка типов), pytest без live/e2e с порогом покрытия 80%, проверку секретов gitleaks и eval-short (короткий прогон оценки на подмене модели без сети). Использует uv версии 0.13.4 и Python 3.12.

[11] `.github/workflows/release.yml`
Сценарий выпуска релиза на GitHub, около 2,6 КБ. Срабатывает при пуше тега rc* или v*; сначала проверяет, что тег лежит на main, и гоняет pytest, node-тесты веба и проверку хэшей вендорных файлов. Потом создаёт релиз с архивом исходников и контрольными суммами SHA256; тяжёлый офлайн-архив с моделями собирается и прикрепляется вручную.

10 ПАПКА .CLAUDE/

10.1 .claude/agents/

Описания 20 ролей виртуальной команды проекта для Claude Code: у каждой роли свой файл с именем, назначением, набором инструментов и «зоной» папок, куда она может писать. Роли охватывают архитектуру, данные, поиск, граф, агента, API, UI, оценку, тесты, безопасность и документацию. Правило CONTRIBUTING.md: менять можно только свою зону.

Файл	Что внутри
[12] `agent-engineer.md`	Роль «инженер агента»: отвечает за оркестратор (главный управляющий модуль агента), маршрутизацию вопросов, цикл самопроверки и память диалога. Правила: вопрос о факте идёт в векторный поиск, про связи и хронологию — в граф, про алиби — в MCP-сервер, вне корпуса — отказ; каждое утверждение с цитатой. Зона: src/detective_board/agent/. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[13] `backend-engineer.md`	Роль «бэкенд-инженер»: отвечает за слой API, потоковую отправку шагов работы (SSE/WebSocket — способ непрерывно слать события в браузер) и настройки. Зона: src/detective_board/api/ и конфигурация. События трассы те же, что рисует интерфейс. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[14] `code-reviewer.md`	Роль «ревьюер кода»: независимо проверяет изменения по списку (корректность, типы, тесты, ошибки, интерфейсы, документация, объём) и сам код не пишет. Итог — APPROVE (принято) или CHANGES (нужны правки) с конкретными пунктами. Его вердикт требует CONTRIBUTING.md для каждого pull request. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[15] `data-engineer.md`	Роль «инженер данных»: скачивание и чистка корпуса (Project Gutenberg, рассказы Конан Дойла), нарезка на чанки (куски текста) с якорями источника и проверка лицензий. Зона: data/, scripts/corpus/, src/detective_board/ingest/. Результат лицензий записывается в data/LICENSES.md. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[16] `demo-director.md`	Роль «режиссёр демо»: готовит 7-минутный сценарий показа, слайды, запасную видеозапись, чек-лист репетиции и ответы на вопросы жюри. Зона: docs/demo/. Опираться можно только на функции, которые работают надёжно, и у каждого шага есть запасной вариант. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[17] `devops-engineer.md`	Роль «девопс-инженер»: окружение, файл блокировки версий uv, just-команды, CI в GitHub Actions и сборка одной командой. Зона: pyproject.toml, uv.lock, justfile, .github/, pre-commit. Требование: `just run` должен работать офлайн после первой установки. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).

Файл	Что внутри
[18] `eval-engineer.md`	Роль «инженер оценки качества»: делает золотой набор из 40+ вопросов с эталонными ответами и источниками, считает метрики (верность ответа, правильность цитат, доля отказов вне корпуса, галлюцинации, полнота поиска) и сравнивает конфигурации. Зона: eval/. Конфигурации: голая модель, только вектор, вектор+реранк, гибрид+граф. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[19] `frontend-engineer.md`	Роль «фронтенд-инженер»: делает интерфейс доски расследования — граф, панель трассы шагов, подсветку цитат. Зона: src/detective_board/web/. Cytoscape.js (библиотека рисования графов) кладётся в проект локально, без CDN в рабочем режиме. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[20] `graph-engineer.md`	Роль «инженер графа»: схема графа знаний, запросы, модель времени и обходы. Зона: src/detective_board/graph/ и docs/data-schema.md. Модель: Person, Place, Item, Event (со временем), Claim (кто что утверждает), рёбра с якорями на источник; встроенная база — кандидат Kuzu. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[21] `mcp-engineer.md`	Роль «MCP-инженер»: отвечает за собственные MCP-серверы (MCP — стандартный протокол, по которому модель вызывает внешние инструменты; здесь сервер времени/алиби) и MCP-клиент, их контракты, ошибки и таймауты. Зона: src/detective_board/mcp/. Контракты описаны в docs/mcp-contracts.md. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[22] `nlp-extraction-engineer.md`	Роль «инженер извлечения»: схемы, промпты и конвейер, которыми малая модель вытаскивает из текста факты в строгом JSON (JSON — текстовый формат данных), с повтором и ремонтом ответа, валидацией и склейкой имён-синонимов. Зона: src/detective_board/extract/ и prompts/extract/. Результаты кэшируются на диске по ключу «чанк + модель + версия промпта». Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[23] `performance-engineer.md`	Роль «инженер производительности»: измеряет задержку, память и размер моделей на целевом ноутбуке (i7-8665U, только CPU) для профилей 16 и 8 ГБ. Зона: scripts/bench/, docs/perf/. Отчёт: токены в секунду, время до первого токена, пик памяти, число запусков и разброс. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[24] `product-owner.md`	Роль «владелец продукта»: ведёт бэклог (список задач), критерии приёмки в формате Given/When/Then, приоритеты и статус. Зона: docs/PRD.md, docs/backlog.md, docs/STATUS.md, docs/acceptance/. Порядок урезания: анимации UI, второй MCP-сервер, тонкости временного графа; не трогать — рабочее демо, цитаты, честные отказы, измерения. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).

Файл	Что внутри
[25] `qa-lead.md`	Роль «главный по тестированию»: стратегия тестов (пирамида), тест-план, проверка приёмки и регрессии. Зона: docs/test-plan.md, docs/qa/. Критерии проверяются реальным запуском, а ошибки сообщаются прямо. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[26] `retrieval-engineer.md`	Роль «инженер поиска»: эмбеддинги (числовые «отпечатки» смысла текста), векторный индекс, BM25 (поиск по словам), реранкинг (пересортировка результатов), гибридное слияние и подбор k. Зона: src/detective_board/retrieval/. Только лёгкие локальные модели; результаты проверяются на наборе оценки. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[27] `security-reviewer.md`	Роль «ревьюер безопасности»: проверяет внедрение инструкций через текст корпуса, безопасность MCP-инструментов (песочница путей, белый список), секреты и зависимости. Зона: docs/security.md и заметки ревью; в код только читает и сообщает находки с файлом и строкой. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[28] `tech-lead.md`	Роль «технический лидер»: архитектура, ADR (записи об архитектурных решениях: контекст, варианты, решение, последствия), границы модулей, ревью интерфейсов. Зона: docs/architecture/, docs/adr/, заглушки интерфейсов в src/detective_board/contracts/. Решает по числам. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[29] `tech-writer.md`	Роль «технический писатель»: README, диаграммы C4/Mermaid, индекс ADR, руководство по запуску, описание метрик. Зона: README.md и docs/ (кроме зон других ролей). Документирует только то, что существует и проверено. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[30] `test-automation-engineer.md`	Роль «автоматизатор тестов»: юнит-, интеграционные и e2e-тесты (pytest, Playwright), фикстуры (готовые тестовые данные) и подмены LLM для повторяемости. Зона: tests/. Основной набор идёт с подменённой моделью, отдельная метка `live` — для тестов с настоящей моделью; покрытие ядра не ниже 80%. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).
[31] `ux-designer.md`	Роль «UX-дизайнер»: сценарий демо, информационная архитектура, визуальный язык, доступность и дизайн-токены (CSS-переменные). Зона: docs/design/. Цвета узлов по типам сущностей, описание анимаций, 7-минутная история показа глазами пользователя. Это короткий файл-инструкция (около 1–1,4 КБ) для Claude Code: сверху имя, описание и разрешённые инструменты, ниже общие правила команды (читать PRD и ADR, не выходить из своей зоны, не выдумывать API, коммиты по Conventional Commits).

11 ПАПКА DATA/

11.1 data/

Данные проекта: готовый чанкованный корпус chunks.jsonl, очищенные тексты 20 рассказов (clean), вручную проверенные факты для демо (curated), кэши извлечения (extraction) и отчёт о лицензиях. Сырые скачанные тексты и индексы здесь не лежат: их строят скриптами заново.

[32] **`data/LICENSES.md`**
Отчёт о лицензиях корпуса (56 строк, 3,5 КБ), проверен 2026-10-22. Таблица пяти сборников Project Gutenberg (Adventures #1661, Memoirs #834, Return #108, His Last Bow #2350, Case-Book #69700) с URL и годами; объясняет, что data/clean/*.txt и data/chunks.jsonl получены скриптом scripts/corpus/ingest.py, и что тексты Конан Дойла — общественное достояние.

[33] **`data/chunks.jsonl`**
Корпус, нарезанный на чанки (чанк — кусок текста, который ищут и подают модели): 596 записей, 1 117 209 байт, формат JSONL (по одной JSON-записи в строке). У каждой — chunk_id (например scandal-in-bohemia-0000), рассказ, раздел, номера абзацев, смещения символов char_start/char_end и сам text; охвачено 20 рассказов. Нужен индексу поиска и извлечению фактов, смещения позволяют подсвечивать цитаты.

11.2 data/clean/

Очищенные английские тексты 20 рассказов о Шерлоке Холмсе по одному файлу (от 36 до 70 КБ каждый, 355–753 строки). Получены из Project Gutenberg: убраны шапка и лицензия, склеены переносы строк. Служат источником для нарезки на чанки и для подсветки цитат.

Файл	Что внутри
[34] `abbey-grange.txt`	Очищенный текст рассказа «Приключение в Эбби-Грейндже» на английском: 49526 байт, 355 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[35] `beryl-coronet.txt`	Очищенный текст рассказа «Берилловая диадема» на английском: 52081 байт, 443 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[36] `black-peter.txt`	Очищенный текст рассказа «Чёрный Питер» на английском: 44658 байт, 389 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[37] `blue-carbuncle.txt`	Очищенный текст рассказа «Голубой карбункул» на английском: 43294 байт, 427 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[38] `boscombe-valley-mystery.txt`	Очищенный текст рассказа «Тайна Боскомской долины» на английском: 52423 байт, 429 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[39] `copper-beeches.txt`	Очищенный текст рассказа «Медные буки» на английском: 54195 байт, 423 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[40] `crooked-man.txt`	Очищенный текст рассказа «Горбун» на английском: 38720 байт, 255 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.

Файл	Что внутри
[41] `man-with-the-twisted-lip.txt`	Очищенный текст рассказа «Человек с рассечённой губой» на английском: 50209 байт, 443 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[42] `musgrave-ritual.txt`	Очищенный текст рассказа «Обряд дома Месгрейвов» на английском: 41509 байт, 275 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[43] `naval-treaty.txt`	Очищенный текст рассказа «Морской договор» на английском: 69880 байт, 753 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[44] `norwood-builder.txt`	Очищенный текст рассказа «Подрядчик из Норвуда» на английском: 51451 байт, 425 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[45] `priory-school.txt`	Очищенный текст рассказа «Пансион Приори» на английском: 63819 байт, 697 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[46] `red-headed-league.txt`	Очищенный текст рассказа «Союз рыжих» на английском: 50528 байт, 429 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[47] `reigate-squires.txt`	Очищенный текст рассказа «Рейгетские сквайры» на английском: 40491 байт, 377 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[48] `resident-patient.txt`	Очищенный текст рассказа «Постоянный пациент» на английском: 36416 байт, 353 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[49] `scandal-in-bohemia.txt`	Очищенный текст рассказа «Скандал в Богемии» на английском: 47692 байт, 523 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[50] `silver-blaze.txt`	Очищенный текст рассказа «Серебряный» (Silver Blaze) на английском: 53739 байт, 461 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[51] `speckled-band.txt`	Очищенный текст рассказа «Пёстрая лента» на английском: 54053 байт, 501 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[52] `thor-bridge.txt`	Очищенный текст рассказа «Загадка Торского моста» на английском: 52075 байт, 497 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.
[53] `wisteria-lodge.txt`	Очищенный текст рассказа «Вистерия-Лодж» на английском: 63315 байт, 501 строк (абзацы идут по одному в строке). Заголовок и лицензия Gutenberg убраны, переносы строк склеены; из этого файла нарезаются чанки data/chunks.jsonl.

11.3 data/curated/

Небольшие вручную проверенные наборы для демо: 8 фактов присутствия и 1 сцена. Каждая запись имеет дословную цитату, автора правки и дату проверки.

[54] `data/curated/presence_demo.jsonl`
Вручную отобранные «факты присутствия» для демо: 8 записей, 10 094 байта, JSONL. Каждая говорит, что человек (например Aloysius Garcia, Charles McCarthy, Lady Brackenstall) был в таком-то месте в такое-то время, с дословной цитатой, говорящим, полярностью (affirm — утверждает) и уверенностью. Записи сгруппированы по эталонным случаям (c001, c002, c042) и подготовлены для правила R1 (один человек не может быть в двух местах сразу); проверены человеком 2026-10-29.

[55] `data/curated/scenes.jsonl`
Курируемая «сцена» для демо: одна запись, 885 байт, JSONL. Это Wisteria Lodge: Garcia, по словам инспектора Бейнза, лежал мёртвым на Oxshott Common «since one o'clock» (60 минут), с цитатой, полями места, дня и времени. Нужна как опорный факт, с которым сравнивают показания свидетелей; проверена человеком 2026-10-29.

11.4 data/extraction/

Два SQLite-кэша ответов модели qwen3:4b (промпт extract-2.1): большой на 231 запись и частичный на 26. Позволяют собрать граф без повторного запуска модели.

[56] `data/extraction/extract-cache-qwen3_4b-extract-2.1-partial.sqlite`
Частичный кэш извлечения фактов модели qwen3:4b промптом extract-2.1: 126 976 байт, база SQLite (файловая база данных). Одна таблица raw (ключ-значение) с 26 записями; ключ — хэш чанка, значение — JSON с сырым ответом модели (число токенов и текст с entities, events, claims, relations). Все 26 ключей есть и в большом ноутбучном кэше, то есть это его подмножество.

[57] `data/extraction/laptop-cache-qwen3_4b-extract-2.1.sqlite`
Кэш извлечения фактов, снятый с ноутбука: 1 110 016 байт, база SQLite с одной таблицей raw из 231 записи (ключ — хэш, значение — JSON с ответом qwen3:4b, который вытащил сущности, события, утверждения и связи из чанка). Из таких кэшей `just graph` собирает граф, не гоняя модель повторно; покрывает 231 чанк из 596.

12 ПАПКА PROMPTS/

12.1 prompts/extract/

Шесть промптов для извлечения фактов из чанков: три версии общего извлечения (extract_v2, v2_1, v2_2), две версии поиска «присутствия» и промпт проверки. Первая строка каждого — версия, она входит в ключ кэша, чтобы старые ответы не смешивались с новыми.

[58] `prompts/extract/extract_v2.txt`
Первая версия (extract-2.0) промпта для извлечения фактов из куска рассказа (1235 байт, 14 строк). Просит вернуть JSON по схеме: сущности, события, утверждения свидетелей, связи; каждый элемент с дословной цитатой и лимиты (до 8 сущностей, 5 событий, 5 утверждений, 6 связей). Место {{CHUNK}} заменяется текстом чанка.

[59] `prompts/extract/extract_v2_1.txt`
Версия extract-2.1 того же промпта (1298 байт, 14 строк): почти как v2, но требует цитаты короче 20 слов и «самую информативную часть». Именно с ней сделаны два кэша в data/extraction/.

[60] **prompts/extract/extract_v2_2.txt**
Версия extract-2.2 (3375 байт, 32 строки): строже и с примером формата. Участники события берутся только из его цитаты, время и место копируются лишь если они есть в этой же цитате, in_speech ставится только для текста в кавычках; в конце даётся пример с поваром и констеблем.

[61] **prompts/extract/presence_v1.txt**
Промпт presence-1 (2458 байт, 24 строки) для поиска «фактов присутствия»: человек был в месте в указанное время, включая слова свидетелей и отрицания. Условие: человек, место и время должны стоять дословно внутри одной цитаты (до 60 слов); местоимение не считается именем. Вывод — JSON вида {"facts": [...]}.

[62] **prompts/extract/presence_v2.txt**
Промпт presence-2.1 (3417 байт, 30 строк): то же, но person и place не пишутся свободно, а выбираются из списков имён, поданных в промпт; если ничего не подходит — UNKNOWN. Так модель сама разрешает местоимения вроде «he» и «the house» по ранее встреченным именам.

[63] **prompts/extract/presence_verify_v1.txt**
Промпт-проверяльщик presence-verify-1.0 (2090 байт, 37 строк): модели дают одно утверждение о человеке и короткий текст, она отвечает yes/no для person_ok, place_ok, time_ok и общий verdict (yes / no / partly). Только JSON, без знаний извне; есть выдуманные примеры. Отсекает ошибочные факты, найденные предыдущими промптами.

13 ПАПКА SRC/

13.1 src/detective_board/

Корневой пакет проекта Detective Board. Подпакеты: agent (агент, отвечающий на вопросы), api (веб-сервер), extract (извлечение фактов из текстов моделью), graph (граф фактов в SQLite), index (сборка индекса поиска), mcp (серверы и клиент инструментов), retrieval (поиск по тексту), symbolic (строгие правила R1–R3).

[64] **src/detective_board/__init__.py**
Пустой файл (0 строк). Он нужен, чтобы папка detective_board считалась пакетом Python, то есть чтобы из неё можно было делать import.

13.2 src/detective_board/agent/

«Мозг» программы. Принимает вопрос, выбирает путь (router), ищет фрагменты рассказов, вызывает правила или маленькую модель, собирает ответ с цитатами и проверяет их. Если в тексте ответа нет, отказывается. Каждый шаг пишет событие-след, которое видит интерфейс.

Файл	Что внутри
[65] `__init__.py`	Входная дверь пакета agent (26 строк). Тут только импорты: Agent, AgentConfig и типы-«розетки» Retriever, GraphBackend, ToolClient, HybridRetriever. Благодаря этому остальной код пишет просто `from detective_board.agent import Agent`.
[66] `agent.py`	Главный «дирижёр» программы (1073 строки): класс Agent с методом ask(question, history). Он получает вопрос и по шагам выдаёт события-«следы» (trace events), последнее из них называется final. Шаги: определить тип вопроса (router), найти фрагменты текста (retrieval_step), проверить, есть ли в тексте ответ, собрать подсказку для модели (context), вызвать правила (rules_client) или модель (llm), проверить цитаты (answer). Если ответа в тексте нет, Agent честно отказывается.

Файл	Что внутри
[67] `answer.py`	Собирает и проверяет итоговый ответ (235 строк). Разбирает в тексте ссылки вида [n: "цитата"], связывает их с точными местами в тексте (anchors, «якоря»), а self_check механически проверяет, что каждая цитата реально есть в источнике. split_claims режет ответ на отдельные утверждения, strip_reasoning убирает «размышления» модели из текста. Вызывается из agent.py.
[68] `backends.py`	Описывает, что агенту нужно от остальных частей программы (120 строк), в виде протоколов (Protocol — «список методов, которые должны быть»). Протоколы: Retriever (поиск фрагментов), GraphBackend (граф фактов), ToolClient (вызов MCP-инструментов). Здесь же адаптеры SentenceRetriever и HybridRetriever, которые подключают реальный поиск к этим розеткам. Так агент не зависит от конкретных реализаций и его легко тестировать.
[69] `config.py`	Настройки агента (69 строк): класс AgentConfig. В нём пороги (насколько фрагмент должен подходить вопросу, чтобы не было отказа) и размеры окон. В заголовке сказано, что все пороги пока предварительные, до прогона оценки качества (eval).
[70] `context.py`	Строит подсказку («обогащённый запрос») для языковой модели (372 строки). Источники нумеруются блоками [1], [2]…, факты из графа и результаты правил помечаются, системная инструкция строгая. Функция looks_like_injection ищет в тексте фразы вида «забудь инструкции» (prompt injection — попытка перехватить модель текстом); build_messages собирает сообщения, fit_prompt выкидывает самые слабые фрагменты, если подсказка не влезает в лимит токенов.
[71] `events.py`	Определяет «события-следы» агента (295 строк): каждое событие — маленькая запись о шаге работы (маршрут выбран, фрагменты найдены, ворота пройдены, цитата проверена и т.д.). У каждого есть общие поля run_id, seq, ts; EventSink выдаёт seq (номер по порядку) и ts (время). Классы Anchor, Citation, HitTrace, GateTrace описывают точки в тексте и цитаты. Эти события потом уходят в интерфейс по SSE (api/sse.py) и рисуются как ход рассуждения.
[72] `extractive.py`	«Выборочный» режим ответа (301 строка): маленькая модель не пишет ответ, а выбирает номер предложения из списка (candidates_for, build_prompt). Ответ берётся по шаблону из выбранного предложения (render_answer), потому что малая модель плохо сочиняет, но хорошо выбирает. Выбор ограничен схемой JSON (select_schema), поэтому модель выдаёт буквально несколько токенов.
[73] `honesty.py`	Охранник честности для вопросов про правила (47 строк). Функция asserts_consistency(text) ловит фразы вроде «показания согласуются», когда правило просто ничего не нашло: отсутствие найденного противоречия не доказывает согласованности. Проверяет только текст утверждения, не цитаты; отрицание («не показывает, что согласуются») не считает нарушением.
[74] `linking.py`	Связывает имена из вопроса с id людей в графе (88 строк): link_person. Совпадение только точное по нормализованному имени или псевдониму (регистр, титулы и притяжательное 's игнорируются), без «примерно похоже». Если подходят несколько людей, возвращает список кандидатов, а не угадывает.
[75] `llm.py`	Единый интерфейс к языковой модели (175 строк). LLM — общий вид; FakeLLM — «актёр с заготовленными ответами» для тестов; OllamaLLM — клиент к локальной Ollama (программа, запускающая модели на вашем компьютере) на стандартной библиотеке Python, без внешних пакетов.
[76] `memory.py`	Память диалога (43 строки): хранит последние N реплик (класс Turn) и сжимает их в короткий текст без обращения к модели. summarize делает краткое содержание, retrieval_query подмешивает его в поисковый запрос, чтобы «а где он был потом?» понималось по контексту.

Файл	Что внутри
[77] `presence_answer.py`	Строит ответ-таблицу «где был человек» (алиби) без модели. Для названного в вопросе человека печатает нумерованные строки: когда, где, кто это утверждает, метка происхождения и дословная цитата [n: "…"]. Затем идёт вердикт правила R1 («человек не может быть в двух местах одновременно»). Данные берёт у инструмента presence_table; главная функция build_presence_answer.
[78] `retrieval_step.py`	Шаг поиска и «ворота релевантности» (~115 строк): после гибридного поиска проверяет, годятся ли найденные фрагменты. gate смотрит и на оценку поиска, и на лексическую поддержку: support считает, какая доля слов вопроса (обрезанных до 5 букв) нашлась в одном из трёх лучших фрагментов. Если нет, агент отвечает отказом: в тексте этого нет.
[79] `router.py`	Маршрутизатор вопросов (306 строк): решает, какой путь выбрать (поиск по тексту, правило противоречий, алиби, хронология и т.п.). Сначала работают жёсткие правила, и только потом, при необходимости, небольшая модель-классификатор. Всегда записывает причину выбора. extract_names достаёт имена из вопроса, parse_time_window разбирает «между шестью и семью вечера».
[80] `rule_first.py`	Ответы «правила сначала» (601 строка) на вопросы о противоречиях, алиби и хронологии. Текст ответа — детерминированный шаблон из результата инструмента-правила и его Explanation (пояснения), без модели и без пересказа. Формы: findings (находки), empty («нарушений среди извлечённых фактов нет, это не доказывает согласованность»), failed (что не удалось), timeline (нумерованный список событий). Функции build_answer, build_timeline_answer.
[81] `rules_client.py`	Клиент, который вызывает правила прямо внутри процесса (50 строк): InProcessRulesClient. Те же функции инструментов из mcp/tools.py, но без запуска отдельного MCP-сервера (MCP — стандартный способ вызывать внешние инструменты). Факты берёт из живого графа через SqliteFactSource. Используется в демо-режиме --fake и когда MCP не настроен.
[82] `scene_answer.py`	Доска подозреваемых для места преступления (357 строк), детерминированная. По сцене из data/curated/scenes.jsonl печатает по строке на каждого человека рассказа: исключён показаниями (текст ставит его в другое место в это время), был на месте или не исключён (данных нет). Опирается на инструмент scene_board; ключевые функции open_scenes, story_persons, build_scene_answer, answer_for.
[83] `semantic_r3.py`	«Нейро-символьное» правило R3 (371 строка). Граф находит пары высказываний разных людей об одном и том же (candidate_pairs), а маленькая модель по двум дословным цитатам решает, противоречат ли они (judge_pair). Вердикт модели не считается решением правила: он показывается отдельным блоком как «гипотеза» (метка conditional, «судила малая модель, не доказано»). Функция find_candidates, render_block добавляет блок к ответу.
[84] `translate.py`	Необязательный перевод вопроса (166 строк): если вопрос в основном по-русски, он переводится на английский перед поиском. Причина: рассказы на английском, а проверка релевантности и роутер ищут только английские слова. Переводится только вопрос, не текст рассказов. is_mostly_cyrillic решает, нужен ли перевод; validate_translation отбрасывает ненадёжный результат (пустой или с кириллицей).
[85] `window.py`	Вырезает из фрагмента текста «окно» вокруг нужного места (200 строк): целые предложения, точные смещения символов, без обрыва посреди предложения. Считает веса слов (idf — чем слово реже в корпусе, тем оно важнее) и находит лучшее предложение/окно под вопрос (best_sentence, best_window). Нужен, чтобы в подсказку модели попадал только нужный кусок, а не весь фрагмент.

13.3 src/detective_board/api/

Веб-слой. Запускает HTTP-сервер (Starlette), отдаёт интерфейс и API: вопросы к агенту потоком SSE, поиск по графу и источникам. Здесь конфиг, сборка сервисов, рабочие потоки, прогрев моделей и набор демо-данных для режима --fake.

[86] **`src/detective_board/api/__init__.py`**
Короткий файл (6 строк): docstring говорит, что пакет api — это HTTP-API с потоком событий SSE агента, поиском по графу и источникам и раздачей статического интерфейса (решение ADR-0005).

[87] **`src/detective_board/api/__main__.py`**
Запуск сервера из командной строки (39 строк): python -m detective_board.api --host 127.0.0.1 --port 8000 [--fake] [--config FILE]. Функция main читает параметры и стартует веб-сервер. Флаг --fake включает демонстрационные данные без настоящей модели.

[88] **`src/detective_board/api/app.py`**
Веб-приложение на Starlette (легкий веб-фреймворк Python) (529 строк): маршруты /api/* и раздача готового интерфейса. Функция build_app собирает всё; SecurityHeaders и HostGuard добавляют защитные заголовки и проверяют адрес и источник запроса (origin_allowed), ollama_models спрашивает у Ollama список моделей. Ответы агента идут потоком SSE.

[89] **`src/detective_board/api/config.py`**
Настройки API (153 строки): класс ApiConfig. Приоритет значений: по умолчанию, потом файл TOML, потом переменные окружения DETECTIVE_BOARD_*. load_config всё собирает; index_missing_message и graph_missing_message дают понятное сообщение, если не найден индекс или граф.

[90] **`src/detective_board/api/fixture.py`**
Заготовленные данные для режима --fake, разработки интерфейса и тестов (664 строки). Есть две версии: «lean» (под pytest: короткие свои фразы, три фрагмента одного рассказа, граф из пяти сущностей; тесты считают эти объекты, поэтому они не меняются) и «demo» (настоящий запуск --fake: те же данные плюс реальные отрывки из data/clean). Выбирает demo_mode(). Функции build_store, build_graph.

[91] **`src/detective_board/api/graph_view.py`**
Представления графа в JSON для интерфейса, только чтение (95 строк). Функции result_json (результат запроса к графу превращает в JSON), subgraph (кусок графа вокруг сущности для «доски расследования») и search_entities (поиск сущностей по имени). Работает поверх graph.GraphStore.

[92] **`src/detective_board/api/runtime.py`**
Рабочие потоки сервера (131 строка): класс Runtime. SQLite (маленькая база данных в одном файле) привязывает соединение к потоку, поэтому каждое соединение живёт в своём рабочем потоке. Тут же запуск агента с возможностью отмены; если рабочий поток занят, возникает Busy.

[93] **`src/detective_board/api/services.py`**
Сборка всех частей из настроек (238 строк): build_services создаёт Store, HybridRetriever, GraphStore и Agent по ApiConfig. Вызывается один раз в каждом рабочем потоке, потому что общий Services между потоками нельзя. Тут же CancellableLLM (модель, которую можно прервать, при отмене RunCancelled) и mcp_status.

[94] **src/detective_board/api/sse.py**
Крошечный файл (19 строк): функция frame формирует кадр SSE (Server-Sent Events — способ, которым сервер по одному соединению присылает браузеру поток сообщений). Кадр состоит из строк event:, id:, data: и пустой строки (стандарт WHATWG).

[95] **src/detective_board/api/tool_client.py**
Синхронный клиент инструментов поверх асинхронной MCP-сессии (153 строки). В отдельном потоке крутится собственный цикл событий, который хозяин сессии, а вызов call() блокирует только рабочий поток. Любая ошибка возвращается как общий объект ошибки, а не исключением. Классы AsyncSession, SyncToolClient.

[96] **src/detective_board/api/warmup.py**
Прогрев при старте (115 строк): в фоновом потоке загружает модель эмбеддингов (числовые «отпечатки смысла» текстов) и модель ответов и подготавливает префикс в кэше Ollama (KV-кэш). Это советующая вещь: сбой записывается, но не роняет сервер, а вопрос, пришедший во время прогрева, просто идёт в очередь. Класс Warmup.

13.4 src/detective_board/extract/

Извлечение фактов из рассказов. Модель читает чанки (куски текста) и выдаёт сущности, события и утверждения с дословными цитатами; затем детерминированный код проверяет, чинит и нормализует их. Отдельно: целевой проход «кто где был» (presence*), проверенные вручную факты и сцены (curated*, scenes), кэш ответов модели.

Файл	Что внутри
[97] `__init__.py`	Пустой файл (0 строк), помечающий папку extract как пакет Python.
[98] `cache.py`	Дисковый кэш сырых ответов модели (54 строки) на SQLite с WAL-журналом: каждая запись сохраняется отдельной фиксацией, поэтому прогон можно возобновить. Ключ cache_key = sha256 от текста фрагмента, модели, версии подсказки, версии схемы и параметров генерации. Разбор и проверку цитат повторяют при каждом чтении, поэтому их правка не требует нового вызова модели. Класс ExtractCache.
[99] `curated.py`	Загрузчик проверенных вручную фактов о присутствии для демо-случаев (224 строки). Читает JSONL (по одному JSON-объекту в строке) data/curated/presence_demo.jsonl, проверяет, что каждая цитата дословно есть в чистом тексте рассказа, и записывает факты в граф. Запуск: python -m detective_board.extract.curated --graph-db data/graph/graph.sqlite. Функции load_file, locate_quote, problems, ingest_facts; итог — LoadReport.
[100] `curated_schema.py`	Схема проверенных вручную фактов о присутствии (172 строки). Они написаны командой с помощью ИИ и ещё должны быть перепроверены человеком; их используют там, где маленькие модели не справляются (местоимения «он», «там»). Не результат автоматического конвейера и везде помечаются как кураторские. Не импортирует другие модули проекта; есть review_status, moment_id, check_review.
[101] `ground.py`	Проверка цитат (108 строк): является ли «quote» от модели дословным отрывком фрагмента? Сравнение игнорирует разницу в пробелах и типографике (кривые кавычки, тире, многоточие). Совпадение после нормализации = exact, цитата переписывается символами фрагмента; иначе пробуется нечёткая привязка difflib (fuzzy) с высоким порогом, иначе none. Функция ground_quote.
[102] `ingest.py`	Передаёт результаты извлечения в граф (41 строка). to_graph_input превращает результат по фрагменту в вход для GraphStore, ingest_results загружает всё в граф.

Файл	Что внутри
[103] `llm.py`	Клиенты языковой модели для извлечения (109 строк): маленький протокол LLMClient, OllamaClient (на стандартном urllib, без внешних пакетов) и FakeClient с заготовленными ответами для тестов. GenRequest и GenResponse описывают запрос и ответ.
[104] `normalise.py`	Детерминированный «нормализатор» между извлечением и правилами (441 строка), без модели, часов и случайности. normalise_time превращает фразу времени в TimeSpec, который читают правила; resolve_days и assign_text_days назначают событиям день рассказа, где текст позволяет; claim_prop превращает поля утверждения (схема v2) в PresenceProp «кто-где-когда». Любая догадка помечается low, поэтому выводы из неё условные.
[105] `parse.py`	Устойчивый разбор сырого ответа модели (136 строк): строгий JSON, детерминированная починка оборванного ответа (repair_truncated), проверка каждого элемента (validate_items). Годные элементы остаются, негодные отбрасываются с записанной причиной.
[106] `pipeline.py`	Извлечение по одному фрагменту (183 строки): подсказка, затем кэш или модель, затем разбор и починка, проверка и привязка цитат. Класс Extractor делает всё это, ExtractConfig — настройки, ChunkResult — результат, load_prompt читает текст подсказки.
[107] `presence.py`	Целевой проход «кто где был» (961 строка). Общий проход извлечения не даёт правилам R1/R3 нужного, поэтому второй проход спрашивает модель только одно: «человек X был/пошёл/остался/был замечен в месте P во время T», включая слова свидетеля и отрицания. Каждое поле затем сверяется с цитатой самого факта (ground_facts, без модели). Класс PresenceExtractor, CLI main.
[108] `presence_candidates.py`	Списки кандидатов для прохода presence-2 (307 строк). Малая модель не может разрешить «he» сама, но умеет выбирать из короткого списка. Для каждого фрагмента строится список людей и мест, которые общее извлечение уже нашло: из графа, из кэша модели или из текста (candidates_from_graph/extraction/text). rank сортирует по числу упоминаний.
[109] `presence_schema.py`	Схема целевого прохода presence (104 строки). PresenceFact — то, что заполняет модель; GroundedFact — то, что пережило проверку и идёт в граф (с дополнительными полями аудита, которых модель не видит). Не импортирует другие модули проекта, чтобы граф мог её подключить. ollama_format_schema даёт JSON-схему для ограниченной генерации.
[110] `presence_verify.py`	Проверка и голосование для прохода presence (372 строки). PresenceVerifier делает один крошечный вызов модели на факт: видит цитату, предложение до и после и утверждение «PERSON was at PLACE at TIME», отвечает по закрытой схеме. vote делает большинство голосов по нескольким прогонам (needed_votes: 2 из 3, 3 из 5 и т.д.). decide — чистая функция «оставить/отбросить».
[111] `repair.py`	Детерминированная починка извлечённых элементов (435 строк). Измерено на qwen3:4b: модель заполняет поля по привычке (пустые participants, polarity «deny» у 98% утверждений, always about_person). Модуль сверяет такие поля с цитатой и текстом и исправляет, что может, без модели. repair_items, repair_extraction, quotation_spans (находит прямую речь), has_negation.
[112] `run.py`	Командная строка извлечения (120 строк): python -m detective_board.extract.run --model M --chunks CHUNKS.jsonl --out OUT.jsonl. select_chunks выбирает фрагменты, main запускает прогон, build_report собирает сводку.
[113] `scenes.py`	Проверенные вручную места преступлений для доски подозреваемых (359 строк). Сцена: место, окно времени и жертва одного рассказа плюс одна дословная цитата из текста. Автоматически это не вывести. Загрузка и проверка (load_file, validate, match_scene); CLI: python -m detective_board.extract.scenes. Файл читает agent/scene_answer.py.

Файл	Что внутри
[114] `schema.py`	Схема извлечения v1 для одного фрагмента (73 строки), намеренно плоская и маленькая, чтобы 4–8B-модель могла её заполнить. Сущности Person/Place/Item в одной модели (type различает), события, утверждения и связи называют сущности строкой без id, у каждого элемента есть quote — дословный отрывок для проверки.
[115] `schema_v2.py`	Схема извлечения v2, обратно совместимое расширение v1 (119 строк). Каждая модель v2 наследует v1 и добавляет только необязательные поля: структурированное утверждение (кто/где/когда/полярность), in_speech и др. Любой v1-ответ валиден как v2, to_v1 — простая проекция назад.

13.5 src/detective_board/graph/

Граф фактов в SQLite: узлы (люди, места, предметы), рёбра, события, утверждения и якоря-цитаты. GraphStore принимает результаты извлечения и отвечает на запросы; facts_source.py превращает граф в FactBase для символьных правил.

[116] **`src/detective_board/graph/__init__.py`**
Входной файл пакета graph (27 строк). Его docstring: хранение графа в SQLite и адаптер, питающий символьные правила.

[117] **`src/detective_board/graph/anchors.py`**
Привязывает дословную цитату к смещениям внутри фрагмента (84 строки): resolve_quote возвращает ResolvedQuote. Делает граф, а не языковая модель.

[118] **`src/detective_board/graph/facts_source.py`**
SqliteFactSource (184 строки): собирает FactBase (снимок фактов для правил) из таблиц графа (решение ADR-0004). Мост от базы SQLite к модулю symbolic.

[119] **`src/detective_board/graph/normalize.py`**
Нормализация имён для слияния одинаковых сущностей (62 строки), детерминированная, без нечёткого поиска. normalize_name, slugify, fold_text, is_unnamed_person (отличает роль/местоимение от имени).

[120] **`src/detective_board/graph/results.py`**
Типы результатов запросов к графу (96 строк): NodeHit, EdgeHit, GraphResult, PathResult. Каждый результат несёт id узлов и рёбер и якоря, чтобы следы агента показывали, откуда ответ.

[121] **`src/detective_board/graph/schema.py`**
Схема базы графа (120 строк): DDL (команды CREATE TABLE) и миграции. Версия хранится в PRAGMA user_version; migrate применяет недостающие шаги, каждый в своей транзакции, и отказывается работать с базой новее кода.

[122] **`src/detective_board/graph/store.py`**
Самый большой файл проекта (1463 строки): GraphStore, граф фактов на SQLite со ссылками на источник (provenance) и якорями-цитатами. Принимает результаты извлечения (ingest, ingest_many, ingest_presence, ingest_curated), склеивает одинаковые сущности по имени (resolve), хранит надстройки времени и дня (set_overlay, set_event_day) и отвечает на запросы (get_node, подграф, пути). Только стандартная библиотека. Загрузка идемпотентна: повторный ввод того же фрагмента ничего не меняет.

[123] **`src/detective_board/graph/timenorm.py`**
Выводит нормализованное время из текста времени (171 строка): derive_time_norm возвращает TimeNorm. Небольшой набор проверенных шаблонов, всё остальное — unknown.

Если результат догадка (голый час без am/pm, часть суток, полночь), confidence = low. День рассказа здесь не выводится: его задаёт GraphStore.set_event_day.

13.6 src/detective_board/index/

Скрипты построения индексов поиска из data/chunks.jsonl: основной индекс чанков и отдельный индекс предложений. Запускаются из командной строки и пересобирают только недостающее.

[124] **src/detective_board/index/__init__.py**
Однострочный файл: docstring «Сборка индекса: data/chunks.jsonl превращается в data/index/db.sqlite». Сама работа в build.py.

[125] **src/detective_board/index/build.py**
Сборка основного индекса поиска (208 строк): uv run python -m detective_board.index.build. Читает data/chunks.jsonl (чанк — кусок текста рассказа) и пишет data/index/db.sqlite. Идемпотентно: второй запуск ничего не меняет. Векторы (эмбеддинги — набор чисел, отражающий смысл текста) берёт из кэша ранних экспериментов, если хеш текста совпал; только недостающие просит у Ollama. Функции build_index, import_cached_vectors.

[126] **src/detective_board/index/sentences.py**
Сборка предложного индекса data/index/sentences.sqlite (168 строк): python -m detective_board.index.sentences. Каждый чанк режется на предложения (retrieval.sentences.split_sentences), строки и полнотекстовый индекс FTS пишутся одной транзакцией, затем считаются векторы пачками и кэшируются. Прогон можно продолжить. Функция build_sentences.

13.7 src/detective_board/mcp/

Собственный MCP-сервер timeline-alibi и клиент к нему. MCP — стандартный способ вызывать инструменты из программы. Инструменты (tools.py) проверяют алиби, строят хронологию и ищут противоречия, пользуясь символьными правилами; sandbox.py ограничивает доступ к файлам.

[127] **src/detective_board/mcp/__init__.py**
Однострочный файл: docstring «Собственные MCP-серверы и помощник MCP-клиента (docs/mcp-contracts.md)». MCP (Model Context Protocol) — стандартный способ, которым программа вызывает внешние инструменты.

[128] **src/detective_board/mcp/client.py**
Тонкая обёртка MCP-клиента (95 строк): запускает сервер через stdio (запуск как дочерний процесс, общение через stdin/stdout), перечисляет инструменты и вызывает один с тайм-аутом. server_params, graph_server_params, McpSession, open_session.

[129] **src/detective_board/mcp/errors.py**
Модель ошибок по docs/mcp-contracts.md, раздел 5 (41 строка): класс ToolError и функция error_payload, которая делает из ошибки одинаковый JSON для всех инструментов.

[130] **src/detective_board/mcp/sandbox.py**
«Песочница» для путей (45 строк): путь допустим, только если он ведёт к обычному файлу внутри разрешённых корневых папок. parse_roots читает список корней, resolve_in_allowlist проверяет путь; иначе SandboxError. Защита от чтения чужих файлов.

[131] **src/detective_board/mcp/server.py**
MCP-сервер timeline-alibi (231 строка), запуск: python -m detective_board.mcp.server. Через stdio отдаёт инструменты build_timeline, presence_table, check_alibi, scene_board, find_overlaps,

find_contradictions из mcp/tools.py. build_server собирает сервер, DeadlineRunner ограничивает время вызова.

[132] **src/detective_board/mcp/source.py**
Источники фактов для инструментов (132 строки): JsonFileFactSource читает один JSON-файл (сериализованный FactBase) из разрешённой папки; GraphDbFactSource берёт факты из базы графа. default_graph_roots задаёт разрешённые корни. Пути проверяются песочницей.

[133] **src/detective_board/mcp/tools.py**
Функции инструментов timeline-alibi (569 строк), тонкие обёртки над symbolic.api: проверяют вход (pydantic), загружают факты через FactSource, вызывают правила и дают JSON по docs/mcp-contracts.md. Пример: check_alibi(source, story=..., person_id=..., window=Window(...), claimed_place_id=None) возвращает вердикт правила R1 с цитатами. Те же функции вызывает агент в процессе (rules_client.py).

13.8 src/detective_board/retrieval/

Поиск по тексту рассказов: BM25 (по словам), поиск по векторам, их слияние RRF; два уровня — чанки и предложения. Хранилище SQLite, получение эмбеддингов через Ollama, заготовка переранжирования.

[134] **src/detective_board/retrieval/__init__.py**
Реэкспорт пакета retrieval (28 строк): собирает в одном месте публичные имена поиска, чтобы их можно было импортировать из одного места. Docstring пуст.

[135] **src/detective_board/retrieval/config.py**
Настройка эмбеддингов (17 строк): класс EmbeddingConfig. Какая модель — это данные, а не код.

[136] **src/detective_board/retrieval/embedder.py**
Получение эмбеддингов (134 строки): протокол Embedder, OllamaEmbedder (HTTP-вызов /api/embed), HashEmbedder (подделка для тестов — вектор из хеша), EmbeddingCache. Эмбеддинг — список чисел, по которому похожие по смыслу тексты оказываются рядом.

[137] **src/detective_board/retrieval/loader.py**
Загрузка чанков в хранилище и досчёт векторов (87 строк): load_chunks читает data/chunks.jsonl идемпотентно, embed_missing считает векторы только для тех, у кого их ещё нет. Отчёт — LoadStats.

[138] **src/detective_board/retrieval/rerank.py**
Заготовка переранжирования (16 строк): протокол Reranker и PassThroughReranker, который ничего не меняет. Реализации пока нет, только «разъём» на будущее.

[139] **src/detective_board/retrieval/search.py**
Поиск по чанкам (124 строки): bm25_search (по словам, через FTS5 — полнотекстовый индекс SQLite), vector_search (по близости векторов) и hybrid_search (RRF — слияние двух списков по месту в каждом). Hit хранит оценки и места из обеих ветвей для трассировки.

[140] **src/detective_board/retrieval/sentences.py**
Гибридный поиск по предложениям (260 строк). Чанковый индекс отвечает «какой абзац», а выборочный режим ответа требует «какое предложение»; отдельный файл SQLite (data/index/sentences.sqlite) хранит по строке на предложение. sentence_search = BM25 + векторы с RRF; split_sentences режет текст, SentenceStore хранит. Если корпус изменился, индекс отвергается (chunks_digest).

[141] `src/detective_board/retrieval/store.py`
Хранилище поиска на SQLite (97 строк): таблица чанков, FTS5 для BM25, расширение sqlite-vec (vec0) для векторов и манифест. Класс Store; при несовпадении версии — StoreVersionError.

13.9 src/detective_board/symbolic/

Символьный слой: строгие правила без нейросети над базой фактов FactBase. R1 (нельзя быть в двух местах сразу), R2 (порядок событий без циклов), R3 (показание против установленного факта). Результаты несут пошаговые пояснения и цитаты. Агент и MCP-сервер используют только api.py.

[142] `src/detective_board/symbolic/__init__.py`
Однострочный файл: «Символьный слой: детерминированные правила R1–R3 над извлечённой базой фактов». Символьный = строгие правила без нейросети.

[143] `src/detective_board/symbolic/_explain.py`
Общие помощники для пояснений и находок (109 строк): make_finding собирает Finding, event_fact и anchor_key описывают факт и место в тексте, sort_findings упорядочивает находки одинаково при каждом запуске.

[144] `src/detective_board/symbolic/api.py`
Публичный вход слоя (65 строк): единственный модуль, который импортируют агент и MCP-сервер. Реэкспортирует модели (Explanation, Finding, Report, Verdict, FactBase…) и содержит run_all, запускающую все правила.

[145] `src/detective_board/symbolic/facts.py`
FactBase (131 строка): неизменяемый снимок фактов с нормализованным порядком, который читают правила. Записи PlaceRec, EventRec, ClaimRec и свойства PresenceProp («кто-где-когда») и OrderProp («что раньше»). load_facts загружает снимок из JSON. Одинаковые факты дают одинаковый результат.

[146] `src/detective_board/symbolic/model.py`
Публичные модели результата слоя (117 строк, pydantic): Verdict, Anchor (место цитаты), TimeSpec (время), FactRef, EdgeRef, Step, Explanation (пошаговое пояснение), Finding (находка), Report. Именно эти объекты видят агент и интерфейс.

[147] `src/detective_board/symbolic/places.py`
Сопоставление мест (66 строк): точное совпадение после нормализации, без нечёткости. PlaceIndex ищет место по названию, PlaceRel выражает отношение между местами.

[148] `src/detective_board/symbolic/time.py`
Арифметика интервалов на абсолютной оси t = день*1440 + минута (124 строки). abs_bounds, overlaps, compare (TimeRel/TimeCmp), strictly_before, is_moment, fmt_abs: пересекаются ли два промежутка, какой раньше.

13.10 src/detective_board/symbolic/rules/

Сами правила R1, R2, R3 и реестр RULES. Каждое правило — функция над FactBase, которая возвращает Report.

[149] `src/detective_board/symbolic/rules/__init__.py`
Реестр правил (24 строки): класс Rule (id, версия, функция run) и словарь RULES из r1_presence, r2_order, r3_claims. Добавление правила = функция + id + версия + тесты.

[150] **src/detective_board/symbolic/rules/r1_presence.py**
Правило R1 (230 строк): один человек не может быть в двух разных местах одновременно. scan_presence_conflicts ищет такие конфликты по всей базе, check_presence проверяет заданного человека и окно времени (используется для алиби). Результат — Report с находками и цитатами.

[151] **src/detective_board/symbolic/rules/r2_order.py**
Правило R2 (388 строк): отношение «раньше» транзитивно и без циклов (если А раньше Б, а Б раньше В, то А раньше В; цикл — противоречие). build_edges строит рёбра порядка, find_path ищет путь, sccs/find_order_cycles находят циклы, timeline строит хронологию.

[152] **src/detective_board/symbolic/rules/r3_claims.py**
Правило R3 (210 строк): утверждение свидетеля несовместимо с установленным фактом. check_claim проверяет одно утверждение, scan_claims просматривает все. Работает с ClaimRec из FactBase.

14 ПАПКА TESTS/

14.1 tests/

Папка с автоматическими тестами всего проекта (тест — маленькая программа, которая сама проверяет, что код работает как задумано). Внутри подпапки по частям системы (agent, api, graph, extract, mcp, symbolic и др.), несколько общих файлов-тестов и JS-тесты веб-части.

[153] **tests/__init__.py**
Пустой файл (0 строк). Он нужен, чтобы папка tests считалась пакетом Python: тогда тесты из подпапок могут подключать друг друга через относительные импорты вроде `from .conftest import hit`.

[154] **tests/test_bench_retrieval.py**
Тесты скрипта сравнения методов поиска scripts/spike/bench_retrieval.py: как «эталонные» цитаты превращаются в куски текста и как считаются метрики (место правильного куска в выдаче, hit@k, recall, MRR — средняя обратная позиция). Характерные: test_find_spans_whitespace_and_quotes, test_boundary_straddle_maps_to_both_chunks, test_score_question_hand_computed, test_cache_resumes_after_interrupt_and_torn_line. 12 тестов; векторы подставные, на хешах слов. Пример: правильные куски на местах 3 и 6 → rank=3, hit@5 и hit@10 истинны, rr=1/3.

[155] **tests/test_extract_shards.py**
Тесты разбиения рассказов на части (шарды) для параллельного извлечения на нескольких потоках и слияния их кэшей. Тесты: test_priority_list_has_20_stories, test_plan_partitions_all_stories_once, test_merge_refuses_foreign_model_prompt_schema, test_cli_refuses_then_forces. 10 тестов. Пример: 20 рассказов делятся на 1, 2, 4 или 7 шардов так, что каждый рассказ встречается один раз, а нагрузка (по числу кусков) отличается мало (максимум ≤ 1.15 от среднего на 4 шардах).

[156] **tests/test_ingest.py**
Тесты скрипта нарезки рассказов на куски scripts/corpus/ingest.py. Тесты: test_synthetic_roundtrip, test_clean_story_unwraps_and_strips_trailing_numeral, test_chunks_file_roundtrip (последний пропускается, если нет data/chunks.jsonl). 3 теста. Пример: текст каждого куска равен срезу чистого текста clean[char_start:char_end], а начало следующего куска идёт сразу за концом абзаца предыдущего; « a b\n c d\n\n\n\n e\n\nII\n» → «a b c d\n\ne\n».

[157] **tests/test_offline_scripts.py**
Тесты shell-скриптов для офлайн-архива (упаковать проект и проверить его без интернета): pack_offline.sh и verify. Тесты: test_scripts_parse_and_have_help, test_dry_run_names_every_missing_input_and_refuses, test_a_model_with_another_build_is_refused. 3 теста. Пример: bash -n не находит синтаксических ошибок, --help печатает «Usage:» и только строки-комментарии; пробный запуск перечисляет все недостающие входные файлы и отказывается работать.

[158] **tests/test_preflight.py**
Короткие тесты скрипта предполётной проверки scripts/preflight.py (есть ли нужные модели Ollama). Тесты: test_defaults_and_env_override, test_check_messages. 2 теста. Пример: по умолчанию модель — qwen3:4b-instruct и bge-m3; если нет qwen3:4b-instruct, проверка выдаёт одну подсказку с командой «ollama pull qwen3:4b-instruct».

[159] **tests/test_score_extraction.py**
Тесты скрипта подсчёта качества извлечения (сравнение предсказаний модели с эталоном): нормализация имён, F1 по словам, проверка цитат, совпадения сущностей и заявлений. Характерные: test_token_f1, test_quote_grounding_substring_and_whitespace, test_claim_requires_speaker_and_quote_overlap, test_real_gold_files_are_valid_and_grounded. 12 тестов. Пример: norm('Mr. Sherlock Holmes') → 'sherlock holmes'; token_f1('a b','c d') → 0.0.

[160] **tests/test_score_pipeline.py**
Тесты подсчёта качества для конвейера версии 2 (score_file и вывод таблицей и Markdown). Тесты: test_scores_v2_output, test_failed_and_missing_chunks_count_as_invalid, test_repaired_is_usable_but_not_ok, test_cli_table_and_markdown. 4 теста. Пример: идеальная запись даёт validity 1.0, claims F1 1.0, entities precision 0.5 и recall 1.0, 4 секунды на кусок.

[161] **tests/test_server_gold12.py**
Тесты скрипта scripts/server_gold12.py: собрать 12 эталонных кусков для прогона на сервере. Тесты: test_chunks_cover_all_gold_files, test_chunks_prefer_corpus_text, test_slug_is_filesystem_safe, test_report_writes_markdown, test_shell_script_syntax_and_help. 5 тестов. Пример: в gold12.jsonl ровно 12 записей, id совпадают с именами файлов эталона; если есть текст в корпусе, берётся он, а не текст эталона.

[162] **tests/test_server_report.py**
Тесты отчёта о прогоне на сервере (scripts/server_report): разбор таблицы моделей docs/team/models.md, сверка с `ollama list`, выбор числа потоков, расчёт скорости и ожидаемого времени. Характерные: test_parse_models_table_real_file, test_parse_ollama_list_and_compare, test_pick_streams, test_progress_stats. 9 тестов. Пример: скорости {1: 40, 2: 24, 4: 15} → выбрать 4 потока; если разница меньше 5% — меньше потоков; 30 из 100 за 600 с → 2 куска в минуту, осталось 35 минут.

[163] **tests/test_symbolic_readiness.py**
Тесты скрипта оценки «готовности к логике»: годятся ли ответы модели для правил (участники событий, время и место в цитате, отрицания). Тесты: test_helpers, test_good_run, test_bad_run_is_penalised, test_failed_chunk_gives_none_rates, test_cli_writes_markdown. 5 тестов. Пример: хороший прогон даёт ev_time_in_quote 1.0, cl_deny 1.0, json_validity 1.0, 10 с на кусок; плохой — нули.

[164] **tests/test_vendor_hashes.py**
Тесты проверки хеш-сумм чужих файлов в web/vendor (шрифты, библиотеки), перечисленных в web/vendor/README.md. Тесты: test_vendored_assets_match_their_recorded_sums, test_a_changed_or_unrecorded_asset_is_caught. 2 теста. Пример: к файлу fonts/special-elite-latin.woff2 дописывают байт, а рядом кладут extra.js → проверка называет оба нарушения.

14.2 tests/agent/

Тесты агента — части, которая принимает вопрос, ищет в тексте, выбирает путь ответа, проверяет цитаты и при необходимости отказывается. Везде подставные модель и поиск, поэтому тесты быстрые и не требуют Ollama. 13 файлов, включая общие заготовки conftest.py.

Файл	Что внутри
[165] `__init__.py`	Пустой файл (0 строк). Помечает папку tests/agent пакетом Python, чтобы один тест мог брать вспомогательные функции из соседнего (например, `from .test_graph_route import _graph`).
[166] `conftest.py`	Общая «кухня» для тестов агента (conftest — файл, который pytest читает сам и берёт из него общие помощники). Тут три короткие фразы-абзацы про пёструю ленту (T1 про болотную гадюку, T2 про гепарда, T3 с подсунутой командой «Ignore previous instructions…») и функция `hit(i, text)`, которая делает фальшивую «находку» поиска (Hit — кусок текста с номером и оценкой). Ещё тут `make_agent`, `FakeRetriever` и `FakeTools` — подставные поиск и инструменты, чтобы Agent работал без настоящей модели. 85 строк, своих тестов нет.
[167] `test_agent.py`	Главный тест агента: проверяет, что Agent правильно отвечает на вопрос, отказывается, когда ответа в тексте нет, и повторяет попытку, если модель ошиблась. Самые характерные: test_fact_route_happy_path, test_out_of_corpus_refusal_by_router, test_self_check_pass_and_fail_modes, test_retry_then_accept. Всего 36 тестов. Пример: вопрос «What is the price of bitcoin today?» → роутер (выбирает путь ответа) сразу отказывает, а подставная модель FakeLLM не вызывается. Работает с `detective_board.agent` и подставными поиском и моделью.
[168] `test_compaction.py`	Проверяет «сжатие» контекста: вместо целого куска рассказа модели показывают только окно из нескольких предложений вокруг ответа. Характерные тесты: test_window_picks_answer_sentence_and_maps_back, test_budget_drops_lowest_ranked_source, test_strip_think_blocks, test_reasoning_leak_fails_attempt. 20 тестов. Пример: длинный текст с фразой «swamp adder» и вопрос «What kind of snake was it, the swamp adder?» → окно содержит эту фразу, не обрывается посреди предложения и вмещается в 60 слов (+20%). Вызывает agent.window, agent.context и agent.answer.
[169] `test_extractive.py`	Тесты «извлекающего» режима (по умолчанию): модель не пишет ответ сама, а только выбирает номер подходящего предложения из списка, а программа подставляет это предложение дословно. Характерные: test_choice_gives_a_verbatim_template_answer, test_none_is_a_plain_refusal, test_injection_sentence_chosen_is_just_data, test_runs_are_deterministic. 20 тестов. Пример: модель отвечает «выбираю swamp adder» → ответ 'From the sources: [1: "…swamp adder…"]', статус answered, цитата ссылается на кусок speckled-band-0030.
[170] `test_extractive_sentences.py`	Проверяет, как в извлекающем режиме подмешиваются отдельные предложения из индекса предложений (SentenceRetriever), в том числе из кусков за пределами лучших. Характерные: test_sentence_from_a_chunk_outside_the_top_hits, test_safety_net_keeps_the_best_sentences_of_the_top_two_chunks, test_absent_index_falls_back_with_a_trace_note, test_stale_sentence_offsets_are_not_offered. 8 тестов. Пример: индекса предложений нет (None) → агент не падает, а пишет в след работы пометку и работает по обычным кускам.

Файл	Что внутри
[171] `test_graph_route.py`	Проверяет путь вопроса через граф знаний (граф — база «кто что делал и где», собранная из рассказов): противоречия и алиби. Характерные: test_contradiction_question_goes_through_graph, test_in_process_tools_equal_the_mcp_tool_functions, test_ambiguous_name_is_reported_not_guessed, test_alibi_with_linked_person_calls_check_alibi. 8 тестов. Пример: имя, подходящее двум людям, не угадывается, а честно сообщается как неоднозначное; вызов инструмента `check_alibi` без person_id даёт ошибку invalid_input.
[172] `test_linking.py`	Тесты привязки имени из вопроса к человеку в графе и разбора времени из вопроса. Характерные: test_extract_names_keeps_mc_names_whole, test_time_window_uses_night_and_morning_words, test_alias_shared_by_two_people_is_ambiguous, test_place_names_are_not_persons. 9 тестов. Пример: «at about one o'clock on the night he died» → окно начинается на 60-й минуте (1:00 ночи); «at nine o'clock at night» → 21:00; «Did Mr. James McCarthy contradict himself?» → человек p:bv:james-mccarthy в рассказе bv.
[173] `test_presence_table.py`	Проверяет таблицу присутствия: инструмент отвечает на вопрос «Где был X?» списком строк «где/когда» из графа с цитатами. Характерные: test_router_presence_table_positive, test_router_golden_sets_only_two_intended_changes, test_conflict_between_rows_uses_the_finding, test_agent_unknown_person_says_not_found. 14 тестов. Пример: пройти по золотым наборам вопросов (64 и 43) — маршрут меняется только у двух вопросов вида «Where was X when …».
[174] `test_rule_first.py`	Тесты ответов «правило прежде всего»: на вопросы о противоречиях и алиби отвечает шаблон по результату логических правил, а модель вообще не вызывается (FakeLLM.calls пуст). Характерные: test_findings_template_has_one_sentence_per_finding, test_garcia_regression_honest_answer_instead, test_alibi_supported_uses_the_verdict, test_tool_error_falls_through_to_the_text. 32 теста. Пример: вопрос про Гарсию в час ночи при пустом результате правил → честная строка «The rule engine found no violation among the facts extracted for wisteria-lodge».
[175] `test_scene_board.py`	Тесты доски сцен: «кто мог быть на месте преступления, а кого исключает алиби». Проверяют роутер, шаблон и агента без модели. Характерные: test_router_scene_positive, test_honesty_block_wording_is_fixed, test_no_guilt_words_no_ranking_no_consistency, test_victim_is_not_assessed. 23 теста. Пример: сцена «Garcia lay dead on Oxshott Common since one o'clock» из мини-рассказа → доска пишет «1. Eccles: excluded by testimony», «2. Baynes: placed at the scene», «3. Lucas Moreno: not excluded»; слов о виновности нет.
[176] `test_security.py`	Тесты защиты ответа: цитата должна реально встречаться в источнике, а служебный текст инструментов не должен ломать ответ. Характерные: test_false_claim_with_unrelated_quote_fails, test_newline_injected_label_is_flattened, test_tool_rule_id_is_sanitised, test_tool_exception_text_does_not_leak. 7 тестов. Пример: ответ 'The band was a snake. [1: "a deadly snake from India"]' проходит self_check; вариант «snake from Persia» проходит при пороге 0.3 и не проходит при 0.9.

Файл	Что внутри
[177] `test_semantic_r3.py`	Тесты «нейро-символьного R3»: правило R3 ищет противоречия, а модель дополнительно судит, противоречат ли друг другу две цитаты разных людей. Характерные: test_pairs_link_different_speakers_on_shared_words_and_skip_unrelated , test_judge_parses_valid_invalid_and_empty_answers, test_block_shows_two_quotes_and_never_claims_proof_or_agreement, test_switch_defaults_to_off. 16 тестов; настоящая модель не нужна. Пример: переключатель semantic_r3 выключен → ответ не меняется; включён → после строки правил добавляется отдельный блок.
[178] `test_translate.py`	Проверяет перевод вопроса с русского на английский перед поиском (рассказы английские). Характерные: test_cyrillic_detection, test_russian_question_goes_the_english_path_and_answers, test_injection_in_translation_refuses, test_cache_reuses_translation_and_is_deterministic. 15 тестов с подставными моделями. Пример: «Какая змея, по словам Холмса, убила доктора Ройлотта?» → переведён, пройдён обычный путь, а в ответе есть строка 'Question translated to English: …'.

14.3 tests/api/

Тесты веб-API (адреса /api/ask, /api/health, граф и т.п.): потоковые ответы, защитные заголовки, пути к данным, прогрев. Используют тестовый клиент Starlette и режим fake без настоящей модели.

[179] **`tests/api/__init__.py`**
Пустой файл (0 строк): делает папку tests/api пакетом Python, чтобы работали импорты вроде `from .conftest import parse_sse`.

[180] **`tests/api/conftest.py`**
Общие заготовки тестов веб-API. Фикстура `cfg` создаёт во временной папке мини-сайт (index.html «hi») и конфиг ApiConfig в режиме fake=True; фикстура `client` строит приложение через `build_app` с подставным списком моделей Ollama (qwen3:8b, bge-m3) и даёт тестовый клиент Starlette. Также тут `parse_sse` — разбор потока событий SSE (потоковые сообщения сервера). 46 строк, своих тестов нет.

[181] **`tests/api/helpers.py`**
Вспомогательный драйвер: вручную говорит с ASGI-приложением и обрывает соединение после N кусков ответа (обычный TestClient так не умеет). Функция `stream_until_disconnect(app, body, after_chunks=…)` отправляет POST с вопросом и сообщает, что клиент ушёл. Нужен тесту обрыва связи в test_ask.py. 49 строк, тестов нет.

[182] **`tests/api/test_ask.py`**
Тесты ответа на вопрос через POST /api/ask, который шлёт ответ потоком SSE (поток событий по частям). Характерные: test_sse_order_and_schema, test_refusal_is_a_normal_event_stream, test_client_disconnect_cancels_run, test_second_run_while_busy_gets_429. 12 тестов. Пример: вопрос «What was the speckled band?» → поток событий, склеенные answer_token равны итоговому answer; вопрос про погоду в Париже → тоже нормальный поток, но с отказом.

[183] **`tests/api/test_demo_rules.py`**
Проверяет демонстрационные данные для правил: ответы на два заготовленных вопроса. Тесты: test_fixture_passages_are_verbatim_with_real_offsets, test_wisteria_question_streams_a_conditional_r1_finding, test_mccarthy_question_gets_no_invented_violation. 3 теста. Пример: «Is there a conflict about

where Garcia was at about one o'clock on the night he died?» → в потоке есть route_decided, graph_query, tool_call, tool_result, а находка помечена «conditional» (условная).

[184] **`tests/api/test_endpoints.py`**
Тесты обычных адресов API: подграф, поиск сущностей, исходный текст, здоровье сервера, конфиг, статические файлы. Характерные: test_subgraph_hops_and_validation, test_source_and_highlights, test_health_degraded_without_data_or_model, test_no_cors_and_host_guard. 16 тестов. Пример: GET /api/graph/subgraph?entity=Helen Stoner&hops=0 → один узел и ноль рёбер; entity=Nobody → пустой список узлов.

[185] **`tests/api/test_hardening.py`**
Тесты защитных заголовков сервера. Тесты: test_security_headers_and_csp, test_real_index_has_no_inline_script_or_style, test_ipv6_loopback_host_accepted, test_origin_check_on_ask. 4 теста. Пример: ответы содержат CSP (политика, что страница может грузить) «default-src 'self'» без 'unsafe-inline', x-frame-options DENY; в настоящем web/index.html нет встроенных скриптов и стилей.

[186] **`tests/api/test_paths.py`**
Тесты путей к данным и запуска: что по умолчанию совпадает со сборщиками, и что программа подсказывает, чего не хватает. Тесты: test_defaults_match_builders, test_data_status_present_and_missing, test_main_without_index_exits_with_actionable_message, test_main_reports_present_files. 4 теста. Пример: индекс есть, графа нет → строка «index: present» и «graph: graph not found» с подсказкой команды `just graph`.

[187] **`tests/api/test_stdio_default.py`**
Проверяет, что по умолчанию инструменты правил запускаются как отдельный процесс по stdio (обмен через стандартный ввод-вывод), а при поломке запуска API откатывается на вызов внутри процесса. Тесты: test_default_tools_are_the_stdio_client_over_the_graph_server, test_fallback_is_labelled_in_process, test_failed_start_falls_back_when_the_child_cannot_spawn. 3 теста. Пример: запуск падает с OSError → имя сервера «timeline-alibi[in-process]» и в stderr «using in-process».

[188] **`tests/api/test_units.py`**
Мелкие тесты частей API: кадр SSE, синхронный клиент инструментов, слои конфигурации, список разрешённых хостов. Характерные: test_sse_frame, test_tool_client_sync_call_and_close, test_tool_client_timeout_and_startup_failure, test_config_rejects_unknown. 8 тестов с подставной сессией. Пример: client.call('check_alibi', {'story':'x'}) → {'status':'ok','tool':'check_alibi'}; клиент после обрыва транспорта перезапускается.

[189] **`tests/api/test_warmup.py`**
Проверяет «прогрев»: при старте API один раз гоняет модель и эмбеддер (преобразователь текста в числа), чтобы первый настоящий вопрос не ждал загрузки. Тесты: test_warmup_primes_system_prompt, test_warmup_primes_the_selection_prompt_in_extractive_mode, test_warmup_failure_is_not_fatal, test_warmup_skipped_in_fake_and_when_disabled. 4 теста. Пример: в извлекающем режиме первый вызов модели несёт SELECT_SYSTEM_PROMPT; сбой эмбеддера (OSError) не роняет запуск.

14.4 tests/curated/

Тесты вручную составленных данных: факты о присутствии и сцены преступлений, которые человек проверил по тексту. Проверяют формат, дословность цитат и отметку о перепроверке.

[190] **`tests/curated/__init__.py`**
Пустой файл (0 строк): делает папку tests/curated пакетом Python.

[191] **tests/curated/test_curated.py**
Тесты «вручную проверенных фактов о присутствии» (curated — отобранные человеком записи вида «кто где был и когда», с дословной цитатой). Проверяют загрузчик и то, что с этими фактами делают правила, а не то, что их выдаёт модель. Характерные: test_quotes_are_verbatim_in_the_clean_text_and_slots_are_honest, test_a_bad_fact_loads_nothing_and_names_every_problem, test_loading_twice_is_idempotent, test_c001_gives_an_r1_finding_with_both_versions. 31 тест. Пример: цитата с «’» вместо «'» и с переводом строки вместо пробела ещё находится в чистом тексте, а цитата с лишним хвостом «and then he left» — нет.

[192] **tests/curated/test_review.py**
Тесты отметки «человек перепроверил»: из списка пар (кто, когда) собирается фраза статуса. Тесты: test_the_shown_phrase_matches_the_data_files, test_review_status, test_bad_review_fields_are_rejected, test_pending_is_the_default_until_the_file_says_otherwise. 5 тестов. Пример: пустой список → статус «pending» (ожидает); пары ('a','2026-10-29') и ('b','2026-10-29') → «human re-check done by …» с именами и датами; фраза должна совпасть с реальными файлами data/curated/presence_demo.jsonl и scenes.jsonl.

[193] **tests/curated/test_scenes.py**
Тесты вручную составленных сцен преступления (место, время, жертва, цитата): схема, загрузка, проверка. Характерные: test_real_scenes_are_grounded_in_the_clean_text_and_the_graph_places, test_quote_must_be_verbatim, test_time_must_be_in_the_quote_exact_and_ends_agree, test_curated_by_label_is_enforced. 18 тестов. Пример: сцена, где цитата не встречается в чистом тексте рассказа, или время которой не названо в цитате, получает список проблем и не загружается.

14.5 tests/eval/

Тесты кода оценки качества: формулы метрик, прогон оценки и проверка «до разгадки». Им нужны только небольшие подставные данные.

[194] **tests/eval/__init__.py**
Пустой файл (0 строк): делает папку tests/eval пакетом Python.

[195] **tests/eval/conftest.py**
Подготовка для тестов оценки: добавляет в путь поиска src, scripts/spike и eval, а функция `load(name)` один раз подгружает модуль из папки eval/ (она не пакет, а набор скриптов). 19 строк, своих тестов нет.

[196] **tests/eval/test_metrics.py**
Тесты формул оценки качества (eval/metrics): доверительный интервал Уилсона, точный критерий Макнемара, доля, recall@k, подсчёт цитат. Характерные: test_wilson_known_values_and_empty, test_share_never_turns_zero_over_zero_into_zero, test_mcnemar_exact, test_citation_needs_verbatim_quote_and_gold_span. 10 тестов. Пример: wilson(5, 10) → (0.2366, 0.7634); share(0, 0) даёт value None, а не ноль; mcnemar_exact(0, 5) ≈ 0.0625.

[197] **tests/eval/test_pre_reveal.py**
Тесты проверки «до разгадки»: можно ли по тексту, обрезанному до момента, когда сыщик называет преступника, догадаться, кто виноват. Проверяют обёртку CutRetriever (отрезает куски и предложения после границы), оценщик и проверяющий файл случаев, всё без Ollama. Характерные: test_cut_drops_chunks_past_the_cut_and_the_straddling_one, test_primary_metric_is_the_culprit_name_and_keywords_are_secondary,

test_rescore_uses_the_saved_answers, test_validator_flags_quote_and_cut_errors. 13 тестов. Пример: обрез на символе 200 в рассказе «a» убирает куски a-2 и a-3, а рассказ «b» остаётся нетронутым.

[198] **`tests/eval/test_run_eval.py`**
Тесты главного скрипта прогона оценки eval/run_eval.py. Характерные: test_short_mode_runs_offline_and_writes_the_report, test_the_vector_arm_has_its_own_gate_threshold, test_a_test_run_is_recorded_once_and_never_overwritten, test_short_mode_is_deterministic. 12 тестов. Пример: короткий режим без сети пишет три файла (json, md, svg), в имени которых есть коммит и хеш настроек; в отчёте стоит пометка, что эталон написан ИИ.

14.6 tests/extract/

Тесты извлечения фактов из текста моделью: разбор и починка её JSON, привязка цитат к тексту, приведение времени, кэш. Вместо настоящей модели — FakeClient.

[199] **`tests/extract/__init__.py`**
Пустой файл (0 строк): делает папку tests/extract пакетом Python.

[200] **`tests/extract/conftest.py`**
Общие заготовки тестов извлечения. Тут CHUNK — короткий кусок текста про Джеймса Маккарти и бассейн в Боскомб, и функция `good_output(**over)`, возвращающая правильный JSON-ответ «модели» (сущности, события, заявления) — тесты портят его по-разному. 52 строки, своих тестов нет.

[201] **`tests/extract/test_ground.py`**
Тесты «заземления» цитат: найти цитату модели в настоящем куске текста, даже если модель ошиблась в паре букв. Тесты: test_exact_and_normalised_rewritten_to_chunk_span, test_fuzzy_reanchors_to_chunk_span, test_fuzzy_small_edit_in_longer_quote, test_hallucinated_or_short_quote_is_none. 4 теста. Пример: «The deceased was founded by the pool at Boscombe» → найдено нечётко и заменено настоящим «The\tdeceased was found by the pool at Boscombe»; выдуманная фраза → None.

[202] **`tests/extract/test_normalise.py`**
Тесты приведения времени и фактов к единому виду. Характерные: test_clock_times, test_since_is_a_start_instant, test_day_part_is_fuzzy, test_calendar_from_dates_and_linked_weekdays, test_resolve_days_is_order_independent. 16 тестов. Пример: «since one o'clock» → начало и конец 60 минут (1:00 ночи), точное, но доверие «low»; части суток вроде «night» считаются нечёткими.

[203] **`tests/extract/test_parse.py`**
Тесты разбора и починки JSON из ответа модели, который бывает обрезан или завёрнут в мусор. Характерные: test_every_truncation_point_gives_valid_json, test_truncated_inside_string_after_complete_item, test_fenced_and_garbage_prefix, test_per_item_validation_keeps_valid_drops_invalid. 10 тестов. Пример: ответ, обрезанный в любом месте, превращается в корректный JSON, где остаются только целые записи, а восстановленное — префикс полного ответа.

[204] **`tests/extract/test_pipeline.py`**
Тесты конвейера извлечения (Extractor): кусок текста → запрос к модели → разбор → проверка цитат → кэш. Характерные: test_extract_ok_grounds_and_rewrites_quotes, test_ungrounded_dropped_unless_kept, test_cache_hit_miss_and_key_sensitivity, test_cli_graph_ingest. 11 тестов с FakeClient. Пример: хороший ответ даёт статус ok и счёт цитат {exact: 4, fuzzy: 0, none: 0}, а заявление «I never touched the gun» получает polarity=deny.

[205] **tests/extract/test_repair.py**
Тесты ремонта ответа модели правилами (участники, отрицание, речь, время и место). Характерные: test_participants_added_from_quote_and_description, test_polarity_from_negation_cue, test_quotation_spans_curly_and_multi_paragraph, test_in_speech_inside_vs_narration. 23 теста. Пример: событие «Mr. McCarthy pass» при известной сущности «Mr. Charles McCarthy» получает участника «Mr. Charles McCarthy» (по уникальной фамилии, без титула).

14.7 tests/fixtures/symbolic/

Данные для проверки логических правил; сейчас там один файл holmes_mini.json.

[206] **tests/fixtures/symbolic/holmes_mini.json**
Крошечная заготовка данных для проверки логических правил (фикстура — готовый набор данных для тестов): рассказ «s», 10 событий, 3 заявления, 4 места, 436 строк, 7535 байт. Каждое заявление имеет цитату с позициями в тексте, человека-говорящего и условие «присутствие» (кто где в какое время). Её читают tests/mcp/conftest.py и tests/symbolic/test_all.py.

14.8 tests/graph/

Тесты графа знаний в SQLite: хранилище, запросы, источник фактов для правил и работа на настоящих цитатах.

[207] **tests/graph/__init__.py**
Пустой файл (0 строк): делает папку tests/graph пакетом Python.

[208] **tests/graph/conftest.py**
Заготовки тестов графа. Читает настоящие куски из data/chunks.jsonl (если файла нет — тесты пропускаются), задаёт рассказ boscombe-valley-mystery и номера кусков C2, C3, C7, и собирает мини-граф Boscombe из сущностей, событий, заявлений и связей. 117 строк, своих тестов нет.

[209] **tests/graph/test_fact_source.py**
Тесты источника фактов для логических правил: граф в SQLite (встроенная файловая база) → набор фактов FactBase. Характерные: test_factbase_shape, test_digest_is_reproducible, test_round_trip_r3_violated_with_both_quotes, test_whole_corpus_load_and_story_filter. 9 тестов. Пример: заявление Маккарти, несовместимое с фактом, даёт правило R3, вердикт VIOLATED и причину CLAIM_INCOMPATIBLE_WITH_FACT, с обеими цитатами.

[210] **tests/graph/test_normalize_time_anchors.py**
Тесты нормализации имён, времени и «якорей» (место цитаты в тексте). Тесты: test_person_keys, test_unnamed_persons_keep_descriptor, test_place_drops_article, test_clock_patterns, test_quote_exact_ambiguous_missing_ws. 7 тестов. Пример: «The Boscombe Pool» как место → «boscombe pool»; «The Deceased» → «the deceased», и это считается безымянным человеком, а «Sherlock Holmes» — нет.

[211] **tests/graph/test_queries.py**
Тесты запросов к графу: поиск сущностей, события человека или места, заявления говорящего, соседи на k шагов, кратчайший путь, трассировка. 6 тестов, среди них test_events_by_person_and_place, test_neighbours_khop_and_determinism, test_shortest_path. Пример: events_by_person('Charles McCarthy') → два события в порядке текста, связи PARTICIPATED_IN и OCCURRED_AT, у каждой есть цитата; для «Nobody» — пусто.

[212] **`tests/graph/test_real_quotes.py`**
Тесты на настоящих цитатах из рассказов: как нормализация времени влияет на вердикты правил. Тесты: test_boscombe_following_claim_is_not_a_violation, test_wisteria_without_a_day_stays_unknown, test_wisteria_with_curated_day_is_a_conditional_finding, test_normalise_is_deterministic_and_idempotent. 6 тестов. Пример: для Wisteria Lodge без указанного дня 3 события получают время, 0 — дату, а правило R1 ничего не находит (unknown 2, consistent 1).

[213] **`tests/graph/test_store.py`**
Тесты хранилища графа GraphStore: миграции, склейка одинаковых людей и мест, повторная загрузка. Характерные: test_dedup_and_alias_resolution, test_alias_only_overlap_does_not_merge, test_idempotent_reingest, test_anchor_invariants. 9 тестов. Пример: «Mr. McCarthy» сливается с Charles McCarthy через псевдоним, а «The Boscombe Pool» — с местом без артикля; в таблице g_node остаётся 8 людей и мест.

14.9 tests/index/

Тесты сборки поисковых индексов (по кускам и по предложениям), включая кэш векторов и дозапуск после сбоя.

[214] **`tests/index/__init__.py`**
Пустой файл (0 строк): делает папку tests/index пакетом Python.

[215] **`tests/index/conftest.py`**
Пять строк: добавляет src в путь поиска модулей, потому что пакет не установлен в голое окружение `uv run --with pytest`. Тестов нет.

[216] **`tests/index/test_build.py`**
Тесты сборки поискового индекса (SQLite с текстами кусков и их векторами — числами, по которым ищут похожее). Характерные: test_complete_cache_needs_no_embedder, test_stale_and_missing_vectors_are_embedded, test_idempotent, test_dimension_mismatch_raises. 7 тестов. Пример: в кэше один вектор годен, один устарел, один отсутствует → из кэша взято 1, заново посчитано 2.

[217] **`tests/index/test_sentences_build.py`**
Тесты сборки индекса предложений (отдельный поиск по предложениям, а не по большим кускам). Тесты: test_build_counts_offsets_and_manifest, test_idempotent_second_run_changes_nothing, test_resumes_after_a_crash_and_embeds_only_the_rest, test_changed_chunk_file_rebuilds_rows. 4 теста. Пример: повторный запуск ничего не пересчитывает (embedded == 0), база остаётся прежней.

14.10 tests/mcp/

Тесты MCP-сервера — программы, дающей модели инструменты (хронология, алиби, доска сцен, таблица присутствия) поверх графа или JSON-файла. Проверяют сами инструменты, песочницу путей, запуск по stdio и защиту.

[218] **`tests/mcp/__init__.py`**
Пустой файл (0 строк): делает папку tests/mcp пакетом Python.

[219] **`tests/mcp/conftest.py`**
Восемь строк: добавляют src в путь поиска и называют файл-заготовку FIXTURE = tests/fixtures/symbolic/holmes_mini.json, который используют тесты MCP-сервера. Тестов нет.

[220] `tests/mcp/test_graph_source.py`
Тесты чтения фактов прямо из графа в SQLite для MCP-сервера (MCP — стандартный способ подключать к модели инструменты). Характерные: test_load_equals_the_sqlite_fact_source, test_read_only_open_cannot_modify_and_does_not_block_a_writer, test_deadline_applies_to_a_slow_graph_read, test_garcia_rule_first_answer_is_identical. 10 тестов. Пример: неизвестный рассказ «nope» → ошибка not_found; человек, известный лишь по заявлениям, даёт статус unknown или insufficient_evidence.

[221] `tests/mcp/test_hardening.py`
Тесты защиты чтения файлов фактов. Характерные: test_symlink_swapped_in_after_resolve_is_not_followed, test_non_regular_file_is_rejected_without_blocking, test_read_is_bounded_by_the_fd_not_by_a_stale_stat, test_deadline_returns_timeout. 7 тестов. Пример: вместо файла подсунули именованный канал (mkfifo) → ошибка store_unavailable, и сервер не зависает.

[222] `tests/mcp/test_presence_table.py`
Тесты инструмента presence_table на мини-данных: он строит таблицу «где был человек» из событий и заявлений. Тесты: test_rows_speakers_denials_and_conflict, test_denial_needs_the_person_names, test_cap_keeps_conflict_rows_and_reports_the_total, test_errors. 4 теста. Пример: из 23 строк при limit=3 остаются 3, среди них строки с конфликтом e1, e2, а в ответе truncated=True и total_rows=23.

[223] `tests/mcp/test_sandbox.py`
Тесты «песочницы» путей: сервер читает файлы только из разрешённых папок. Тесты: test_traversal_and_junk_rejected, test_absolute_outside_rejected, test_symlink_escape_rejected_inside_ok, test_source_maps_sandbox_failure_to_store_unavailable. 7 тестов. Пример: абсолютный путь вне разрешённой папки отклоняется, а в тексте ошибки нет настоящего пути.

[224] `tests/mcp/test_scene_board.py`
Тесты инструмента scene_board на уровне MCP: кто на месте сцены, кого исключает алиби. Характерные: test_each_status_is_what_check_alibi_says, test_a_person_with_two_accounts_is_excluded_and_also_at_scene, test_victims_are_checked_apart, test_no_persons_is_insufficient_evidence. 13 тестов. Пример: статус каждого человека на доске совпадает со статусом, который отдаёт check_alibi для того же человека и окна времени.

[225] `tests/mcp/test_stdio.py`
Тесты настоящего запуска MCP-сервера отдельным процессом с общением через stdio (стандартный ввод и вывод). Тесты: test_stdio_roundtrip, test_stdio_scene_board_matches_in_process, test_stdio_sandbox_violation_is_error_not_crash, test_client_timeout_returns_error_object. 4 теста. Пример: вызов scene_board по stdio для рассказа «s», места «B», окна 1300–1310 даёт тот же результат, что прямой вызов функции.

[226] `tests/mcp/test_tools.py`
Тесты инструментов MCP-сервера на мини-фикстуре: хронология, алиби, пересечения, противоречия. Характерные: test_timeline_matches_rules, test_alibi_equals_direct_call, test_window_validation, test_contradictions_match_run_all. 8 тестов. Пример: build_timeline(limit=1) возвращает одно событие и truncated=True.

14.11 tests/presence/

Тесты прохода presence — извлечения фактов «кто где был» с проверкой по тексту, списками кандидатов, второй проверкой и записью в граф. Модель везде подставная.

[227] **tests/presence/__init__.py**
Пустой файл (0 строк): делает папку tests/presence пакетом Python.

[228] **tests/presence/helpers.py**
Общие помощники: читает настоящие куски из data/chunks.jsonl (если файла нет — пропускает тесты), функция rec(chunk_id) отдаёт один кусок; задаёт рассказы BV (boscombe-valley-mystery) и WL (wisteria-lodge); используется подставной клиент FakeClient с готовым JSON. 48 строк, тестов нет.

[229] **tests/presence/test_presence_candidates.py**
Тесты версии presence-2: в запрос к модели добавлены списки кандидатов (люди и места), чтобы она выбирала из списка, а не выдумывала. Характерные:
test_schema_enum_is_built_from_the_lists_and_capped,
test_pronoun_is_resolved_through_the_list_and_marked_context,
test_cache_key_has_prompt_schema_and_candidates, test_ingest_marks_list_resolved_facts_low.
17 тестов с подставным клиентом. Пример: rank({'Bob':2,'Ann':2,'Zed':5,'ann':1}) → ('Zed','Ann','Bob').

[230] **tests/presence/test_presence_eval.py**
Тесты оценки прохода presence (скрипт scripts/spike/presence_eval.py): строит эталон и считает точность и полноту по готовым предсказаниям. Тесты:
test_gold_is_built_from_the_extraction_gold, test_perfect_predictions_score_one_on_the_strict_tier,
test_wrong_place_hits_the_quote_but_not_the_fields,
test_scores_per_stage_grounded_voted_verified. 6 тестов. Пример: идеальные предсказания дают recall и precision, равные 1.0.

[231] **tests/presence/test_presence_extract.py**
Тесты прохода presence: модель по куску текста выдаёт факты «кто где был», программа проверяет их по тексту. Характерные:
test_prompt_version_and_worked_example_is_self_consistent,
test_fields_missing_from_the_quote_drop_the_fact, test_deictic_place_is_never_a_place,
test_at_most_six_facts_and_duplicates_collapse. 16 тестов. Пример: факт, где место не встречается в его цитате, отбрасывается; слово вроде «here» местом не считается.

[232] **tests/presence/test_presence_graph.py**
Тесты попадания фактов присутствия в граф. Характерные:
test_narrator_fact_is_an_event_with_person_place_and_own_time,
test_speech_fact_adds_a_claim_and_denial_is_a_claim_only,
test_reingest_is_idempotent_and_replaces_without_leftovers,
test_wisteria_c001_with_canned_context_resolved_facts_is_a_conditional_r1_finding 10 тестов.
Пример: слова рассказчика становятся событием; слова персонажа добавляют ещё и заявление; отрицание («Bob was not…») — только заявление.

[233] **tests/presence/test_presence_verify.py**
Тесты проверки фактов второй моделью и голосования нескольких прогонов (подставные клиенты). Характерные: test_schema_is_closed_enum_only,
test_parse_answer_rejects_anything_off_schema, test_decide_inconsistent_yes_and_failures,
test_verifier_error_and_garbage_reject_and_trace. 21 тест. Пример: схема ответа проверяющего допускает только verdict yes/partly/no; ответ «maybe» отвергается.

14.12 tests/retrieval/

Тесты поиска: по ключевым словам (BM25), по смыслу (векторы), их смесь (RRF) и поиск по отдельным предложениям.

[234] **tests/retrieval/__init__.py**
Пустой файл (0 строк): делает папку tests/retrieval пакетом Python.

[235] **tests/retrieval/conftest.py**
Пять строк: добавляют src в путь поиска (пакет не установлен в голое окружение). Тестов нет.

[236] **tests/retrieval/test_retrieval.py**
Тесты поиска по кускам: полнотекстовый поиск BM25 (по словам), векторный поиск (по смыслу) и их смесь RRF. Характерные: test_fts_escaping_never_raises, test_vector_matches_brute_force, test_rrf_hand_computed, test_store_version_mismatch. 11 тестов. Пример: запрос «Holmes's» превращается в безопасную строку "'Holmes'"; пустой запрос « ... » → ".

[237] **tests/retrieval/test_sentences.py**
Тесты разбиения на предложения и поиска по ним. Характерные: test_offsets_round_trip_on_real_chunks, test_short_units_merge_into_the_next_sentence, test_hybrid_search_matches_hand_computed_rrf, test_incompatible_manifest_is_refused. 7 тестов. Пример: 'He nodded. "Yes." "I saw him at the door." It was already very dark.' → два предложения: короткие «He nodded.» и «Yes.» склеены со следующим.

14.13 tests/symbolic/

Тесты логических правил R1 (два места в одно время), R2 (порядок по кругу), R3 (слова свидетеля против фактов). Строят крошечные графы вручную и проверяют вердикты и повторяемость.

[238] **tests/symbolic/__init__.py**
Пустой файл (0 строк): делает папку tests/symbolic пакетом Python.

[239] **tests/symbolic/conftest.py**
Шесть строк: добавляют в путь поиска папку тестов и src, чтобы работал импорт symbolic_helpers. Тестов нет.

[240] **tests/symbolic/symbolic_helpers.py**
Помощники для тестов логических правил: строят маленькие графы вручную. Время пишется как день и «ЧЧ:ММ» (функция hm), каждое событие получает цитату вида «Quote of e1.». Тут же ev, span, fbase, claim, presence, rel_ev. 116 строк, своих тестов нет.

[241] **tests/symbolic/test_all.py**
Тесты запуска всех правил вместе: одинаковый результат при повторе и перестановках, стабильная «сводка» (digest). Характерные: test_determinism_twice_and_shuffled_byte_identical, test_digest_changes_with_facts, test_fixture_file_roundtrip_and_determinism, test_run_all_rejects_unknown_rule. 9 тестов. Пример: добавить одно событие e11 → digest меняется.

[242] **tests/symbolic/test_r1.py**
Тесты правила R1: один человек не может быть в двух разных местах в одно время. Характерные: test_r1_01_overlap_derivation_and_ids, test_r1_12_same_occurrence_guard, test_motion_verb_makes_finding_conditional, test_low_time_confidence_makes_finding_conditional. 29 тестов. Пример: человек W в месте B и потом в D с пересекающимися интервалами → находка с фактами e1 и e2 и рёбрами OCCURRED_AT и PARTICIPATED_IN.

[243] **tests/symbolic/test_r2.py**
Тесты правила R2: порядок событий не должен ходить по кругу. Характерные: test_r2_01_transitivity, test_r2_02_three_cycle, test_r2_07_relation_contradicts_times,

test_r2_10_text_position_is_not_time. 16 тестов. Пример: «e1 после e3, e2 после e1, e3 после e2» → одна находка ORDER_CYCLE с тремя рёбрами.

[244] **`tests/symbolic/test_r3.py`**
Тесты правила R3: сверка слов свидетеля с известными фактами. Характерные: test_r3_01_contradiction, test_r3_02_corroborated_at_finer_granularity, test_r3_03_unstructured, test_r3_06_claim_vs_claim_is_unknown. 11 тестов. Пример: заявление «был в H в 21:30» при событии «W в H в 21:30» → вердикт CLAIM_CORROBORATED; заявление без разбора → UNSTRUCTURED_CLAIM.

14.14 tests/web_js/

JavaScript-тесты веб-страницы (web/js): рисунки доски, разбор SSE, состояние, подсветка цитат, воспроизведение записей, демо-режим, след агента. Запускаются средствами Node (`node --test`), браузер не нужен; два теста с пометкой live ходят на настоящий сервер, если задан адрес.

[245] **`tests/web_js/board.test.mjs`**
JS-тест оформления доски расследования (.mjs — модуль JavaScript, запускается встроенным в Node тест-раннером): рисунки людей, мест и предметов, нитки между карточками, тени, подписи. Тесты: «hash and portrait variants are deterministic», «place print follows the words in the name», «a thread sags below the straight line…», «run clock: ms under a second, tenths, then minutes». 9 тестов. Пример: место «Boscombe Pool» → «water», «Baker Street» → «street»; предмет «the speckled band» → «snake».

[246] **`tests/web_js/demo.test.mjs`**
JS-тесты демо-режима: готовые записи прогонов подставляются, когда живой ответ завис. Тесты: «questions are compared without case, quotes or punctuation», «the stored run of a question: exact, near-exact, never a different question», «a live run is stalled after STALL_MS of silence», «index.json lists every recording in web/replay once». 10 тестов. Пример: один и тот же вопрос с разным регистром и пунктуацией находит ту же запись, а другой вопрос — нет.

[247] **`tests/web_js/highlight.test.mjs`**
JS-тесты подсветки цитат в тексте и разбора ответа с метками [1: "цитата"]. Тесты: «single range splits into before/hit/after», «overlapping and adjacent ranges merge», «ranges are clamped; empty, inverted and non-numeric ranges are dropped», «parseAnswer: unfinished marker while streaming stays text». 8 тестов. Пример: диапазон 12–26 в «Holmes said the speckled band was a swamp adder.» → три куска: до, подсвеченный, после.

[248] **`tests/web_js/replay.test.mjs`**
JS-тесты воспроизведения записанных прогонов: задержки по исходным меткам времени, остановка, проверка файлов записи и безопасность путей. Тесты: «delays follow original timing, with speed and cap», «abort stops playback», «shipped replay files are valid recordings», «safeReplayPath rejects traversal, schemes and absolute paths». 7 тестов. Пример: события с разницей 1.5 с и 10 с воспроизводятся с теми же паузами с учётом скорости и потолка; путь вида «../x» отвергается.

[249] **`tests/web_js/sse.test.mjs`**
JS-тесты разборщика SSE на странице (SSE — поток событий, который сервер шлёт по частям). Тесты: «parses a complete frame», «frame split at every byte position gives the same result», «CRLF, bare CR, and CR+LF split across chunks», «comments and keepalives are ignored…». 7 тестов. Пример: кадр 'event: route_decided\nid: 0\ndata: {"a":1}' даёт одно сообщение, даже если его разрезать в любом месте.

[250] `tests/web_js/state.test.mjs`
JS-тесты «редьюсера» состояния страницы: как события потока превращаются в то, что видит пользователь (черновик ответа, попытки, итог, ошибки, граф). Характерные: «a new attempt drops the tokens of the failed one», «final replaces the draft and ends the run», «duplicate seq is ignored», «stream ending without final or run_error is a lost connection». 19 тестов. Пример: три попытки ответа — остаётся последняя, а счётчики складываются.

[251] `tests/web_js/trace_api.test.mjs`
JS-тесты таблицы следа работы агента и клиента API на странице. Тесты: «traceRows folds tokens into one row per attempt», «retrieval row lists per-arm ranks and the gate», «healthSummary: ok, degraded, down», «askStream maps 429 busy to ApiError…». 8 тестов; два «live»-теста пропускаются без запущенного сервера. Пример: поток, разрезанный посреди кадра и посреди UTF-8 символа, декодируется верно; ответ 429 → дружелюбное сообщение «занято».

15 ПАПКА EVAL/

15.1 eval/

Папка оценки качества: скрипты, которые меряют, насколько хорошо программа ищет цитаты, отвечает, отказывается и находит противоречия, плюс наборы вопросов и эталонов. Внутри: запускалки (run_eval.py, run_answer_check.py, pre_reveal.py), измерители поиска (*_recall.py), формулы (metrics.py) и подпапки golden, extraction_gold, pre_reveal. Эталоны сделаны агентом Claude, не человеком.

[252] `eval/evidence_in_prompt.py`
Скрипт измеряет: у скольких отвечаемых вопросов нужная цитата-доказательство вообще попадает в текст, который уходит модели (без самой модели). Берёт вопросы из eval/golden/questions.jsonl и двух spike-наборов, ищет куски настоящим гибридным поиском по data/index/db.sqlite, режет окна вокруг лучших предложений и сравнивает три стратегии (base, multi, rank). Запуск из шапки: uv run python eval/evidence_in_prompt.py [--stamp S]. Результат пишет в docs/spike/results/evidence-<stamp>.json и .md; его функции потом берут extractive_recall.py и sentence_recall.py.

[253] `eval/extractive_recall.py`
Скрипт без модели считает долю отвечаемых вопросов, у которых золотое предложение попало в список кандидатов «извлекающего» режима (когда ответ собирают из готовых предложений текста). Перебирает варианты: сколько чанков брать, сколько предложений с чанка, общий предел. Базовый вариант — текущий: 4 чанка, до 6 предложений с чанка, предел 24. Запуск: uv run python eval/extractive_recall.py [--stamp S]; результат — docs/spike/results/extractive-recall-<stamp>.json/.md. Предупреждает, что цифры оптимистичны: вопросы те же, на которых настраивали.

[254] `eval/metrics.py`
Чистые формулы метрик для run_eval.py, только стандартная библиотека, без модели и судьи. Считает: верность цитат (цитата — дословная часть чанка и чанк золотой), попадание в золотое предложение, точность и полноту отказов, долю ложных отказов, recall@k по чанкам, точность и полноту поиска противоречий. Для каждой доли даёт интервал Уилсона 95 % и точный тест Макнемара для сравнения двух конфигураций. Проверяется тестами в tests/eval/test_metrics.py.

[255] `eval/pre_reveal.py`
Тест «до развязки»: может ли агент назвать преступника, если из текста вырезан кусок, где его называют. Для 16 случаев из eval/pre_reveal/cases.jsonl сравнивает три условия: full (весь

корпус), cut (текст до cut_char, чанки за разрезом выброшены) и bare (модель без поиска, может помнить рассказ наизусть). Главная метрика — назван ли преступник; ещё считает «обвинён невиновный» и ключевые слова. Запуск: flock /tmp/db-llm.lock uv run python eval/pre_reveal.py; пересчёт сохранённого: --rescore RESULT.json. Нужны Ollama и индекс.

[256] **`eval/r3_semantic.py`**
Опыт «нейро-символьного» правила R3 на 23 dev-случаях противоречий: пары утверждений берутся из графа, а маленькая модель по двум цитатам решает, спорят ли они. Режим --mode pairs — без модели, только охват кандидатов-пар; --mode judge — с моделью (flock /tmp/db-llm.lock uv run python eval/r3_semantic.py --mode judge --max-pairs 12). Тестовые id из split.json отвергаются в коде. Вердикт модели считается гипотезой, не решением правил.

[257] **`eval/run_answer_check.py`**
Проверка моделей-ответчиков: настоящий Agent (путь только через поиск, без графа и MCP-инструментов) на 12 золотых вопросах для каждой модели-кандидата. Запросы встраиваются bge-m3, потом модели гоняются по одной с паузами; результаты дописываются в docs/spike/results/answers-<stamp>-<model>.jsonl (можно продолжить), сводка — answers-<stamp>.md; для лучшей модели есть базовая «голая модель». Метрики грубые: доля слов ожидаемого ответа в тексте. Запуск: uv run python eval/run_answer_check.py (часы) или --summary-only --stamp <stamp>.

[258] **`eval/run_eval.py`**
Главный запуск оценки: четыре конфигурации (bare — модель без поиска, vector — только векторный поиск, vector+rerank — «не запущено», нет реранкера, hybrid+graph — настоящий агент) на замороженном разделении. Считает механические метрики из metrics.py (верное поведение, цитаты, отказы, recall, противоречия правилами) с интервалами Уилсона и тестом Макнемара. Запуск: uv run python eval/run_eval.py --short (CI, 8 вопросов, фальшивая модель), --split dev, --split test. Пишет eval-<дата>-<split>-….json/.md/.svg; тестовый результат дважды для той же конфигурации не пишет.

[259] **`eval/sentence_recall.py`**
Скрипт без модели сравнивает два способа собирать предложения-кандидаты для извлекающего режима: текущий (топ-6 чанков, до 8 предложений с чанка, предел 40) и «sentence-hybrid» (12 лучших предложений из отдельного индекса предложений плюс по 2 лучших из двух первых чанков). Те же 86 отвечаемых вопросов и золото, что в extractive_recall.py. Запуск: uv run python eval/sentence_recall.py [--stamp S] [--index data/index/sentences.sqlite]; результат — docs/spike/results/sentence-recall-<stamp>.json/.md. Есть абляции (больше/меньше предложений, без страховки).

[260] **`eval/spike_chunk_gold.json`**
Таблица «вопрос → какие чанки содержат цитату-доказательство» для 35 spike-вопросов (20 старых + 15 трудных). Для каждого id: группа (old-en, old-ru, hard…), ожидаемые рассказы, список чанков и по каждой цитате позиции в тексте (spans) и чанки, которых она касается. Создаётся скриптом scripts/spike/chunk_gold.py, читают его замеры поиска (bench_retrieval.py, bench_arms.py).

[261] **`eval/spike_questions.jsonl`**
20 первых вопросов для замеров поиска (15 на английском, 5 на русском; русские — переводы английских). Поля: id (en-01…), lang, question, expected_story, evidence (короткая цитата-подтверждение). Пример: en-02 «Какая смертельная индийская змея убила доктора?» → speckled-band, «It is a swamp adder». Используют bench_embed.py, bench_retrieval.py, chunk_gold.py.

[262] `eval/spike_questions_hard.jsonl`
15 трудных вопросов для замеров поиска (все на английском): 8 «detail» (мелкая деталь), 4 «paraphrase» (вопрос перефразирован, слов из текста нет), 3 «cross» (ответ в двух рассказах). Поля: id (hard-01…), lang, category, question, expected_story(ies), evidence (список цитат), evidence_stories. Пример: hard-01 про острое блюдо, скрывшее снотворное → silver-blaze, цитата про карри.

[263] `eval/window_recall.py`
Маленький офлайн-скрипт (без Ollama): сохраняет ли функция best_window() золотую цитату, когда режет окно из золотого чанка по вопросу. Берёт отвечаемые вопросы из golden и двух spike-наборов, цитата считается сохранённой, если окно её содержит или покрывает ≥ 50 %. Запуск: uv run python eval/window_recall.py [--words 110 150] — размеры окна в словах.

15.2 eval/extraction_gold/

12 чанков из рассказов, размеченных под схему извлечения фактов (сущности, события, утверждения, связи) — «золото», с которым сравнивают результат языковой модели. Каждый файл — один чанк, имя вида рассказ-номер. Плюс README с описанием и предупреждением, что разметка машинная.

Файл	Что внутри
[264] `README.md`	Описание папки с эталонами извлечения. Говорит: 12 чанков из data/chunks.jsonl размечены под схему src/detective_board/extract/schema.py (сущности, события, утверждения, связи), в каждой записи дословная цитата. Честно предупреждает, что размечал агент Claude, а не человек, поэтому нужна выборочная проверка, а надёжнее смотреть полноту (recall), а не точность. Внутри таблица: какой чанк о чём (показания, диалог, вывод по предмету и т. д.).
[265] `blue-carbuncle-0004.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «blue-carbuncle-0004»: вывод Холмса по шляпе. В файле сам текст и «золото» — 7 сущностей (люди, предметы), 3 событий, 6 утверждений, 4 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[266] `boscombe-valley-mystery-0006.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «boscombe-valley-mystery-0006»: выводы о поведении человека. В файле сам текст и «золото» — 6 сущностей (люди, предметы), 3 событий, 5 утверждений, 3 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[267] `boscombe-valley-mystery-0007.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «boscombe-valley-mystery-0007»: показания свидетеля Джеймса Маккарти, его перемещения. В файле сам текст и «золото» — 10 сущностей (люди, предметы), 7 событий, 5 утверждений, 5 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[268] `crooked-man-0014.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «crooked-man-0014»: вложенный рассказ-показание, время «без четверти девять». В файле сам текст и «золото» — 7 сущностей (люди, предметы), 4 событий, 4 утверждений, 3 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.

Файл	Что внутри
[269] `naval-treaty-0007.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «naval-treaty-0007»: рассказ от первого лица с указанием времени. В файле сам текст и «золото» — 9 сущностей (люди, предметы), 5 событий, 3 утверждений, 3 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[270] `norwood-builder-0014.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «norwood-builder-0014»: диалог, опрос свидетеля. В файле сам текст и «золото» — 9 сущностей (люди, предметы), 3 событий, 6 утверждений, 4 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[271] `red-headed-league-0003.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «red-headed-league-0003»: вывод Холмса плюс текст объявления. В файле сам текст и «золото» — 11 сущностей (люди, предметы), 4 событий, 4 утверждений, 4 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[272] `reigate-squires-0004.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «reigate-squires-0004»: рассказ полиции о свидетелях, время «без четверти двенадцать». В файле сам текст и «золото» — 7 сущностей (люди, предметы), 4 событий, 5 утверждений, 3 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[273] `reigate-squires-0009.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «reigate-squires-0009»: диалог в духе алиби: кто где был, горели ли лампы. В файле сам текст и «золото» — 12 сущностей (люди, предметы), 3 событий, 6 утверждений, 3 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[274] `silver-blaze-0004.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «silver-blaze-0004»: показания со временем (девять часов, служанка). В файле сам текст и «золото» — 8 сущностей (люди, предметы), 4 событий, 3 утверждений, 4 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[275] `silver-blaze-0018.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «silver-blaze-0018»: диалог про следы. В файле сам текст и «золото» — 9 сущностей (люди, предметы), 4 событий, 5 утверждений, 4 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.
[276] `speckled-band-0020.json`	Эталонная разметка одного куска текста (чанка, то есть абзацев по 250–350 слов) «speckled-band-0020»: описание комнаты и физических улик. В файле сам текст и «золото» — 10 сущностей (люди, предметы), 4 событий, 4 утверждений, 3 связей, у каждой записи есть дословная цитата. Размечал агент Claude, человек не проверял. С ним сравнивают результат модели скриптами score_extraction.py, score_pipeline.py, symbolic_readiness.py и server_gold12.py.

15.3 eval/golden/

«Золотые» наборы для проверки: 64 вопроса по рассказам, 43 случая на противоречия, замороженное разделение на dev/test и README с описанием и честным предупреждением, что наборы написал агент Claude. Все записи содержат дословные цитаты из data/clean/.

[277] **`eval/golden/README.md`**
Паспорт «золотого» набора из 64 вопросов по 20 рассказам. Объясняет поля записи (id, category, lang, question, expected_answer, expected_behavior, gold_quotes, answerable, notes), число вопросов по категориям (факт 8, многошаговые 6, вне корпуса 12, ловушки 5 и т. д.) и как его проверять: python3 scripts/eval/validate_golden.py. Честно сказано: вопросы и ответы написал агент Claude, человек не смотрел, минимум 10 пунктов надо проверить руками до публикации метрик; перечислены самые неуверенные (например g029).

[278] **`eval/golden/contradictions.jsonl`**
Набор из 43 проверочных случаев на противоречия (JSONL — по одному JSON-объекту на строке). 26 случаев с настоящим конфликтом и 17 контрольных «противоречия нет»; виды: R3 утверждение против факта 19, контроль 17, R1 человек в двух местах сразу 4, R2 нарушенный порядок событий 3. Поля: id (c001…), story, kind, question, expected_finding, expected_description, involved_claims (кто, что, дословная цитата), evidence_strength, needs_time_normalisation, notes. По нему run_eval.py проверяет правила, а validate_contradictions.py — сам файл.

[279] **`eval/golden/questions.jsonl`**
«Золотой» набор: 64 вопроса по 20 рассказам (61 на английском, 3 на русском), по одному JSON на строке. Из них 51 отвечаемый, 13 — на отказ; поведение: answer 46, refuse 13, correct_premise 3, ignore_injection_and_answer 2. Поля: id (g001…), category, lang, question, expected_answer, expected_behavior, gold_quotes (рассказ и дословная цитата), answerable, notes. Пример: g001 «Какая змея убила доктора Ройлотта?» → болотная гадюка, цитата из speckled-band.

[280] **`eval/golden/split.json`**
Заморозка разделения «золота» на dev (для настройки) и test (для итоговой честной проверки). В шапке правило: внутри каждой категории id сортируются по sha256(seed+id), первые N уходят в test, остальные в dev; 12 вопросов, на которых уже подгоняли промпты, принудительно в dev. Состав: вопросы 34 dev и 30 test, противоречия 23 dev и 20 test; есть контрольная сумма. Проверяет scripts/eval/split_check.py, читают run_eval.py и r3_semantic.py.

15.4 eval/pre_reveal/

Одна папка с одним файлом cases.jsonl — 16 случаев для теста «до развязки», где из рассказа вырезают место, называющее преступника.

[281] **`eval/pre_reveal/cases.jsonl`**
16 тестовых случаев для pre_reveal.py: по 2 на каждый из 8 рассказов (black-peter, boscombe-valley-mystery, naval-treaty, norwood-builder, reigate-squires, silver-blaze, speckled-band, thor-bridge). Поля: id, story, cut_char (позиция в тексте перед разоблачением), question, truth, truth_keywords(_alt), cleared (кто оправдан), culprit_names, cleared_names, reveal_quotes, rationale. Пример: pr-boscombe-1 «Кто убил Чарльза Маккарти?» → Джон Тернер, он же «Чёрный Джек из Балларата». Вопросов нет в золотом наборе.

16 ПАПКА SCRIPTS/

16.1 scripts/

Служебные программы проекта: подготовка текста (corpus/), проверки наборов оценки (eval/), замеры первого этапа (spike/), запуск извлечения графа на GPU-сервере (server_*.sh, extract_shard*), сборка и проверка офлайн-архива для демо (pack_offline.sh, verify_offline.sh), запись повторов для интерфейса (record_run.py, replay_index.py) и мелкие проверки (preflight.py, check_vendor_hashes.py). Большинство — shell или Python на одной стандартной библиотеке.

Файл	Что внутри
[282] `check_vendor_hashes.py`	Проверяет, что сторонние файлы веб-интерфейса (скрипты, стили, шрифты в web/vendor/) не подменены: сверяет их sha256 с суммами, записанными в web/vendor/README.md. Каждый файл .js/.mjs/.css/.woff2/.ttf… должен быть в README, иначе ошибка. Запуск: python3 scripts/check_vendor_hashes.py [VENDOR_DIR]; код 0 — всё сходится, 1 — расхождение. Только стандартная библиотека.
[283] `extract_shard.sh`	Запускает на одной машине (например GPU-сервере) одну «шарду» — долю рассказов — для извлечения графа. Рассказы берёт у extract_shards.py, для каждого вызывает detective_board.extract.run, кэш отдельный на шарду (data/cache/extract-cache-<модель>-shard<k>.sqlite), можно продолжать. Запуск: scripts/extract_shard.sh <shard-id> <n-shards> [model] (по умолчанию qwen3:4b). Параметры run.py не меняет — они входят в ключ кэша.
[284] `extract_shards.py`	Планирует, как поделить рассказы между N шардами примерно поровну по числу чанков: жадно, самый тяжёлый рассказ — в самую лёгкую шарду. Порядок рассказов читает из scripts/run_extraction_priority.sh. Режимы: extract_shards.py stories <shard-id> <n-shards> (список рассказов шарды) и plan <n-shards> (таблица). Вызывается из extract_shard.sh.
[285] `make_server_archive.sh`	Собирает архив для переноса на сервер кафедры и проверяет, что он работает из чистой копии. Архив — это git archive HEAD (только отслеживаемые файлы, без .git, секретов и кэшей) плюс файл COMMIT. Проверка: распаковать, uv sync, быстрые тесты, bash -n для серверных скриптов. Запуск: scripts/make_server_archive.sh [--no-verify] [OUT.tar.gz].
[286] `merge_extract_cache.py`	Сливает и проверяет кэши извлечения (SQLite с таблицей raw(key, value)). Ключ — sha256 от текста чанка, модели, версии промпта и схемы, поэтому скрипт пересчитывает ожидаемые ключи и отказывается принимать чужие записи без --force. Команды: merge DEST SRC… --model M, verify CACHE_A CACHE_B --model M (сравнить сырые ответы), count CACHE…. Слияние идемпотентно: существующие ключи не перезаписываются.
[287] `pack_offline.sh`	Собирает офлайн-архив для демо-ноутбука с выключенной сетью: репозиторий на нужном коммите, индексы и граф (db.sqlite, sentences.sqlite, graph.sqlite), uv, Python, кэш для uv sync, Ollama и нужные модели (bge-m3, qwen3:4b-instruct, qwen3:4b), INSTALL.md, MANIFEST, SHA256SUMS. Запуск: scripts/pack_offline.sh [--tag REF] [--out DIR] [--with-8b] [--dry-run]. Нужен Linux x86_64; проверка на месте — verify_offline.sh.
[288] `preflight.py`	Рано и понятно падает, если Ollama (локальный сервер моделей) недоступен или нет нужной модели. Читает настоящий конфиг (умолчания < TOML < переменные DETECTIVE_BOARD_*) и через GET /api/tags проверяет модель ответов (умолчание qwen3:4b-instruct) и модель эмбеддингов bge-m3. Запуск: python scripts/preflight.py [--config FILE]; код 0 — готово, 2 — не готово.

Файл	Что внутри
[289] `record_run.py`	Записывает один настоящий прогон API как файл повтора для запасной записи интерфейса. Отправляет вопрос на /api/ask, сохраняет события трассы (SSE — поток событий с сервера) с исходными метками времени; в meta помечено, что это запись реального запуска. Запуск: scripts/record_run.py --question "..." --out web/replay/real-g001.json [--port 8000]; API должен работать.
[290] `replay_index.py`	Пишет web/replay/index.json — список записей, которые предлагает выбор в интерфейсе и клавиша c. Для каждого web/replay/*.json берёт meta и записывает: файл, вопрос, вид (real/fake/synthetic), дату, секунды и шаг демо. Запуск: python3 scripts/replay_index.py [--check] (с --check выходит с кодом 1, если index.json устарел).
[291] `run_extraction_priority.sh`	Запускает извлечение графа рассказ за рассказом, начиная с самых нужных для демо (boscombe-valley-mystery, silver-blaze, black-peter…, всего 20). Для каждого вызывает detective_board.extract.run с кэшем data/cache/extract-cache.sqlite и пишет граф в data/graph/graph.sqlite; можно перезапускать. Запуск: scripts/run_extraction_priority.sh [MODEL] [N_STORIES] (умолчание qwen3:4b и 10). Список рассказов читает extract_shards.py.
[292] `server_bootstrap.sh`	Разовая (но повторяемая) подготовка GPU-сервера к извлечению: проверяет и ставит Ollama, настраивает OLLAMA_NUM_PARALLEL и OLLAMA_KEEP_ALIVE, скачивает модели, гоняет тесты. Идемпотентна — каждый шаг сначала проверяет, сделан ли он. Запуск: scripts/server_bootstrap.sh [--check-only] [--with-8b] [--parallel N] [--keep-alive DUR] [--yes] [--skip-tests]. Пишет data/cache/server/bootstrap.env для server_run_all.sh; инструкция — docs/team/SERVER_RUNBOOK.md.
[293] `server_gold12.py`	Помощник для server_gold12.sh, Ollama не нужен. Команды: chunks (собрать 12 золотых чанков в файл data/cache/gold12.jsonl), slug MODEL PROMPT (название прогона вроде qwen3_4b__extract_v2_1), report --stamp S OUT.jsonl… (оценить прогоны, напечатать таблицу и записать docs/spike/results/gold12-<S>.md).
[294] `server_gold12.sh`	Оценивает боевой извлекатель на 12 золотых чанках для каждой пары «модель × промпт» (по умолчанию qwen3:4b и qwen3:4b-instruct на extract_v2 и extract_v2_1), печатает одну сравнительную таблицу и пишет docs/spike/results/gold12-<stamp>.md. Нужен запущенный Ollama. Можно продолжать: прогон, где уже 12 строк, пропускается (FORCE=1 — повторить). Запуск: scripts/server_gold12.sh [--models "…"] [--prompts "…"].
[295] `server_presence.sh`	Целевой проход «присутствие» (presence: кто где и когда был, extract/presence.py) на GPU-сервере после server_run_all.sh. По одному прогону на модель, у каждого свой кэш, копия графа и файл предсказаний, потом presence_eval. Запуск: scripts/server_presence.sh [--models "qwen3:8b qwen3:4b-instruct"] [--all] [--limit N] [--lenient] [--literal] [--candidates graph|cache|text] [--verify] [--samples N] [--pull] [--dry-run]. Качество, как сказано в шапке, не измерено, пока скрипт не отработал (итог — data/cache/server/presence/eval.json).
[296] `server_progress.sh`	Показывает ход идущих шард: число обработанных чанков на шарду, скорость в минуту, оценку времени окончания, последние строки логов, загрузку GPU (nvidia-smi) и ollama ps. Данные берёт у server_report.py progress. Запуск: scripts/server_progress.sh [--watch] (--watch обновляет каждые 30 секунд).
[297] `server_report.py`	Набор помощников для server_bootstrap.sh, server_run_all.sh и server_progress.sh, Ollama не нужен. Команды: check-models (сверить ID из ollama list с docs/team/models.md), pick-streams (лучшее число параллельных потоков), pick-probe-story, probe-split, kv, baseline, progress (счётчики, скорость, ETA) и report (итоговый report.md). Запуск, например: server_report.py progress --dir data/cache/server.

Файл	Что внутри
[298] `server_run_all.sh`	Вся сессия извлечения на сервере после server_bootstrap.sh, по фазам: determinism, probe, gold12, run, finish. Готовые фазы пропускаются по файлам-меткам в data/cache/server/state/, шарды продолжают с кэшей; параметры извлечения (температура 0, seed 42 и т. д.) не меняются, потому что входят в ключ кэша. Запуск: scripts/server_run_all.sh [--phase LIST] [--streams N] [--parallel P] [--skip-gold12] [--force] [--no-wait] [--allow-partial] [--dry-run]; лучше в tmux, логи в data/cache/server/logs/.
[299] `verify_offline.sh`	Проверяет распакованный офлайн-архив на целевом ноутбуке при выключенной сети: контрольные суммы, офлайн uv sync, быстрые тесты, Ollama из архива с моделями, запуск API на реальных индексе и графе и три демо-вопроса (g001 ответ, c001 нарушение R1, g047 отказ) плюс второй отказ g056. Печатает PASS/FAIL по шагам, код 0 только если всё прошло. Запуск: ./verify_offline.sh [ARCHIVE_DIR] [--require-offline] [--port N] [--ollama-port N] [--keep].

16.2 scripts/corpus/

Две программы подготовки текста: fetch.py скачивает сборники рассказов с Гутенберга, ingest.py чистит их и режет на рассказы и чанки. Результат — data/clean/ и data/chunks.jsonl.

[300] `scripts/corpus/fetch.py`
Скачивает с проекта Гутенберг пять сборников рассказов о Холмсе (adventures, memoirs, return, last_bow, casebook) в data/raw/*.txt. Короткий скрипт на 29 строк, только стандартная библиотека. Дальше с этими файлами работает scripts/corpus/ingest.py. Запуск: python scripts/corpus/fetch.py.

[301] `scripts/corpus/ingest.py`
Чистит сырые сборники и режет их на рассказы и абзацные чанки (куски по 250–350 слов, хвост короче 120 слов приклеивается к предыдущему). Читает data/raw/<сборник>.txt, пишет data/clean/<рассказ>.txt (20 рассказов) и data/chunks.jsonl (596 чанков); текст чанка — ровно срез clean[char_start:char_end]. Заголовки рассказов заданы списком в коде. Только стандартная библиотека.

16.3 scripts/eval/

Проверяльщики наборов оценки: validate_golden, validate_contradictions, validate_pre_reveal и split_check проверяют, что файлы в eval/ целы и цитаты в них дословно есть в рассказах; repair_eval.py сравнивает извлечение до и после ремонта вывода модели. Все без вызова модели.

[302] `scripts/eval/repair_eval.py`
Сравнивает «до/после» шага ремонта вывода модели (extract/repair.py) на уже сохранённых ответах без вызова модели. Берёт копии кэшей SQLite (qwen3:4b, extract-2.1), разбирает и «заземляет» (проверяет цитаты по тексту) с ремонтом и без, строит временный граф, ищет противоречия через InProcessRulesClient, потом сверяет события с золотом из eval/extraction_gold. Запуск: repair_eval.py CACHE.sqlite [CACHE2.sqlite …] [--model qwen3:4b] [--json].

[303] `scripts/eval/split_check.py`
Проверяет eval/golden/split.json: каждый id ровно в одном из dev/test, нет неизвестных id, 12 вопросов проверки ответов лежат в dev, правило воспроизводит список test, сходится sha256. С флагом --write пересоздаёт файл (только до заморозки). Код 1 при любой проблеме. Только стандартная библиотека, без моделей.

[304] **scripts/eval/validate_contradictions.py**
Проверяет файл eval/golden/contradictions.jsonl: корректный JSON в каждой строке, обязательные поля, уникальные id, известные виды, согласованность expected_finding с видом (контроли — false, остальные — true), каждая цитата — дословная подстрока рассказа, у положительных случаев ≥ 2 цитат, минимальное число случаев по видам. Запуск: python3 scripts/eval/validate_contradictions.py [файл] [папка clean]; код 0 — ок.

[305] **scripts/eval/validate_golden.py**
Проверяет eval/golden/questions.jsonl: JSON в строке, обязательные поля, уникальные id, известные категории, минимальные счётчики, согласованность answerable и отказа, дословность каждой цитаты в data/clean/<рассказ>.txt. Запуск: python3 scripts/eval/validate_golden.py [questions.jsonl] [clean_dir]; код 0 — всё хорошо, 1 — есть проблемы.

[306] **scripts/eval/validate_pre_reveal.py**
Проверяет eval/pre_reveal/cases.jsonl: поля и типы, уникальные id, смещение cut_char внутри текста, цитаты разоблачения дословны и стоят после разреза, вопрос не содержит ключевых слов ответа, а до разреза ни одно предложение или чанк не содержит все ключевые слова. Честно перечисляет, чего проверить нельзя (парафразы, память модели). Запуск: python3 scripts/eval/validate_pre_reveal.py [--exposure] [cases.jsonl] [clean_dir] [chunks.jsonl]; без Ollama.

16.4 scripts/spike/

Замеры и пилоты первого этапа (Phase 0): какая модель эмбеддингов и какая языковая модель подходят, как лучше искать (BM25, вектор, гибрид), как хорошо модель извлекает факты. Здесь замерщики (bench_*), общие помощники (_common.py), построитель золота поиска (chunk_gold.py) и оценщики извлечения (score_*, presence_eval, symbolic_readiness). Результаты пишутся в docs/spike/results/.

Файл	Что внутри
[307] `_common.py`	Общие мелочи для замеров первого этапа (Phase 0): чтение JSONL, свободная память, статистика (среднее, разброс), класс Http для запросов к Ollama, измерение памяти процессов Ollama (RssSampler) и write_results — запись результатов в docs/spike/results/ в JSON и Markdown. Сам не запускается, его импортируют остальные скрипты папки. Только стандартная библиотека.
[308] `bench_arms.py`	Сравнивает на уровне чанков «руки» поиска через настоящий модуль src/detective_board: BM25 (слова по ИЛИ/И, со стоп-словами и без), векторный поиск, гибрид RRF (разные k и веса). Временная база SQLite строится из data/chunks.jsonl и кэша векторов bge-m3; эмбеддятся только вопросы. Метрики hit@1/5/10, MRR, recall@10 по chunk gold. Запуск: .venv/bin/python scripts/spike/bench_arms.py; результат в docs/spike/results/.
[309] `bench_embed.py`	Замер моделей эмбеддингов (эмбеддинг — число-вектор, описывающий смысл текста): скорость в чанках в секунду, размерность, память и проверка поиска по косинусу на вопросах из eval/spike_questions.jsonl (hit@5 и hit@1 по рассказам, отдельно en, ru и парные en-оригиналы русских). Работает через POST /api/embed Ollama; префиксы запроса и документа задаются --query-prefix MODEL=TEXT / --doc-prefix. Только стандартная библиотека.
[310] `bench_llm.py`	Замер языковых моделей на локальном Ollama: задержка, скорость генерации, оперативная память. Использует POST /api/generate с seed, temperature, num_ctx; режим «мышления» (--think auto|off|none) выставляет сам и считает утечки <think>. Запуск и параметры — python scripts/spike/bench_llm.py --help; не запускать, пока на машине идёт работа, чувствительная к памяти. Результаты — в docs/spike/results/.

Файл	Что внутри
[311] `bench_retrieval.py`	Замер поиска по чанкам для моделей эмбеддингов: корпус эмбеддится один раз (векторы кэшируются в data/cache/emb/, прерванный запуск продолжается), потом все чанки ранжируются по косинусу для 20 + 15 вопросов. Метрики hit@1/5/10, MRR, recall@10 против золота chunk_gold.py, группы old-en, old-ru, hard, cross. Режим --dry-run ничего не вызывает. Использует bench_embed.embed_all.
[312] `bench_rss.py`	Короткий (52 строки) замер установившейся памяти модели: выгружает все модели из Ollama, грузит одну, гоняет промпт на 300 слов и читает RSS процесса-исполнителя. Берёт помощники из _common.py. Адрес Ollama зашит в коде — http://localhost:11434; отдельных аргументов в шапке нет.
[313] `chunk_gold.py`	Для каждой цитаты-доказательства ищет, в каких чанках data/chunks.jsonl она лежит (поиск по тексту рассказа без учёта пробелов и вида кавычек; чанк золотой, если его отрезок пересекается с цитатой). Читает eval/spike_questions.jsonl и eval/spike_questions_hard.jsonl, пишет eval/spike_chunk_gold.json; ненайденные цитаты выводит и выходит с кодом 1. Функции find_spans и load_questions используют eval/*_recall.py. Без сети и моделей.
[314] `overlap_metric.py`	Вторичная метрика извлечения по пересечению: предсказанное утверждение или связь считается попаданием, если его цитата пересекается в тексте чанка с цитатой золотого элемента того же вида. Игнорирует говорящего и формулировку, то есть меряет «нашла ли модель нужное место», а не «так же ли назвала». Подключается из score_extraction.py и score_pipeline.py.
[315] `presence_eval.py`	Оценка прохода «присутствие» (extract/presence.py) без модели. Золото собирается тут же: события из eval/extraction_gold с человеком, местом и временем плюс четыре случая из contradictions.jsonl (c001, c003, c038, c043) с вручную записанными значениями; метки человек не проверял. Считает полноту по цитате и полную, точность, долю выживших фактов и доли заземлённых цитат. Запуск: scripts/spike/presence_eval.py PRED.jsonl [PRED2.jsonl …] [--gold-out eval/presence_gold.jsonl].
[316] `run_extraction.py`	Пилот извлечения графа на размеченных чанках одной моделью: через Ollama /api/generate с format = JSON-схема (жёсткое ограничение вывода), температура 0, фиксированный seed. Пишет предсказания в JSONL ({chunk_id, output, время}) для score_extraction.py в docs/spike/results/. Одна модель за запуск. Запуск: uv run --with pydantic python scripts/spike/run_extraction.py MODEL [--limit N].
[317] `run_extraction_all.sh`	Девятистрочная обёртка: для каждой модели из аргументов запускает run_extraction.py отдельным процессом и ждёт 45 секунд между ними. Запуск: scripts/spike/run_extraction_all.sh MODEL1 MODEL2 …; в конце печатает ALLDONE, неудачи помечает FAILED.
[318] `score_extraction.py`	Оценивает предсказания извлечения по ручному золоту eval/extraction_gold: строгие точность, полнота и F1, заземление цитат, доля валидного JSON; невалидный или пропущенный вывод считается пустым. Вход — JSONL с {chunk_id, output} или {chunk_id, extraction}. Запуск: score_extraction.py PREDICTIONS.jsonl [--gold eval/extraction_gold] [--json]. Использует схему src/detective_board/extract/schema.py.
[319] `score_pipeline.py`	Оценивает боевой конвейер извлечения (схема v2) по eval/extraction_gold. Вход — JSONL от detective_board.extract.run --out FILE; элементы приводятся к схеме v1 через schema_v2.to_v1, дальше логика score_extraction.py и overlap_metric.py, плюс заполненность новых полей v2 и секунды на чанк. Несколько файлов — одна сравнительная таблица. Запуск: score_pipeline.py OUT.jsonl [OUT2.jsonl …] [--gold DIR] [--json] [--markdown FILE].

Файл	Что внутри
[320] `symbolic_readiness.py`	Показывает, насколько результат извлечения готов для символьных правил: на золотых чанках сравнивает версии промпта. Считает валидность JSON, заземление цитат, F1 и долю полей, от которых зависят правила (участники событий, время и место в собственной цитате, отрицания в утверждениях, about_person) и скорость. Оговорка: золото машинное, 12 чанков, по одному прогону. Запуск: symbolic_readiness.py OUT.jsonl [OUT2.jsonl …] [--gold DIR] [--stamp S] [--md-dir DIR].

17 ПАПКА WEB/

17.1 web/css/

Три файла стилей: шрифты, дизайн-токены (цвета и размеры, светлая и тёмная темы) и основной app.css.

[321] **`web/css/app.css`**
Главная таблица стилей интерфейса (546 строк, около 51 КБ): сетка страницы, шапка, панели, карточки на доске, блокнот с шагами, ответ, мобильные вкладки. Цвета и размеры берёт из tokens.css, а не пишет сама. Подключается из index.html.

[322] **`web/css/fonts.css`**
Объявляет пять шрифтов (IM Fell English, EB Garamond обычный и курсив, Special Elite) и говорит браузеру, откуда их брать: из файлов web/vendor/fonts. Интернет не нужен, всё работает без сети.

[323] **`web/css/tokens.css`**
Набор «токенов» — именованных значений дизайна (цвета, размеры, шрифты, вида --db-bg-board: #e6dcc8). Есть светлая тема и тёмная («ночь на Бейкер-стрит»), тёмная включается по настройке системы. Это копия docs/design/tokens.css; после правок её копируют заново.

17.2 web/

Веб-интерфейс программы: страница index.html, стили, скрипты, записи прогонов и сторонние файлы. Работает без интернета, ничего не загружает со стороны.

[324] **`web/favicon.svg`**
Значок вкладки браузера: бежевый квадрат с лупой (круг и ручка тёмного цвета). SVG — картинка, записанная текстом (334 байта).

[325] **`web/index.html`**
Единственная страница веб-интерфейса «Detective Board» (144 строки). Подключает стили (fonts.css, tokens.css, app.css) и скрипты. Содержит каркас: строку вопроса и кнопку Ask, колонку «Case notes» (ход работы агента), доску улик, панель ответа с источниками и нижнюю строку состояния. Всё оживляет js/app.js.

17.3 web/js/

Скрипты интерфейса. Большая часть — «чистые» модули без страницы и сети (state, trace, highlight, layout, sse, replay, demo, board-art), их легко проверять. app.js связывает их со страницей, api.js говорит с сервером, graph.js рисует доску.

[326] `web/js/api.js`
Сетевой слой. Через fetch шлёт POST /api/ask с вопросом и отдаёт ответ по кускам (функция askStream); есть запросы к /api/health, /api/graph/entities, /api/graph/subgraph и /api/source/<чанк>. Чанк — кусок текста рассказа. Ошибки превращает в понятные сообщения (friendlyError). Использует разбор потока из sse.js.

[327] `web/js/app.js`
Главный скрипт интерфейса (1032 строки): связывает чистые модули, страницу и сервер. Принимает вопрос, запускает живой или записанный прогон, рисует ход работы, ответ с цитатами, источник, историю и список записей, обрабатывает клавиши и тему. Сам почти не считает: состояние ведёт state.js, доску рисует graph.js.

[328] `web/js/board-art.js`
Мелкие чистые функции для вида доски, без DOM. Выбирают картинку карточки (шляпа, борода по хэшу id), считают провисание нити между двумя булавками, тень от лампы и сокращают имена («James McCarthy» → «J. McCarthy», слаг рассказа → название). Одни и те же улики всегда дают одну и ту же доску.

[329] `web/js/demo.js`
Помощники для демо-режимов, без DOM и сети. Находит сохранённую запись по вопросу (приводит текст к простому виду и допускает до 6 опечаток по расстоянию редактирования), подписывает записи и решает, что предложить при обрыве. Если от сервера 90 секунд нет ни байта (STALL_MS), прогон считается потерянным.

[330] `web/js/graph.js`
Рисует доску расследования (919 строк): карточки улик на пробке, нити между булавками, лампа над столом. Форма карточки зависит от типа (человек — портрет, место — гравюра, событие — вырезка, утверждение — карточка). Это обычный DOM+SVG, а не Cytoscape. Функция createGraph; позиции считает layout.js, картинки — board-art.js.

[331] `web/js/highlight.js`
Два чистых помощника по тексту. splitHighlights режет текст на куски по диапазонам подсветки цитаты (склеивает пересечения). parseAnswer находит в ответе пометки вроде [1], [1, 2] и [1: "цитата"] и превращает их в ссылки на источники.

[332] `web/js/layout.js`
Раскладка карточек на доске, без случайных чисел. Новые карточки встают по небольшой «силовой» симуляции (отталкивание, пружины вдоль связей), потом их разводят, чтобы не перекрывались. Уже стоящие карточки не двигаются. Таблица BOX хранит размеры каждого типа, например человек 136×228.

[333] `web/js/replay.js`
Проигрывание сохранённой записи в исходном темпе. normaliseRecording принимает массив событий или {meta, events}; replayDelays считает паузы из разницы меток времени ts; playReplay отдаёт события по одному (можно ускорить и ограничить паузу); safeReplayPath проверяет путь к файлу записи.

[334] `web/js/sse.js`
Разбор SSE (Server-Sent Events — поток, где сервер шлёт события по мере готовности). Класс SseParser принимает куски текста и возвращает готовые сообщения {event, data, id}; учитывает перевод строки, разорванный между кусками. decodeMessage превращает сообщение в событие. Чистый код, на него опирается api.js.

[335] `web/js/state.js`

Чистый «редьюсер» состояния одного прогона (316 строк): получает действия (старт, событие, конец, отмена) и выдаёт новое состояние: фаза, маршрут, найденные фрагменты, текст ответа, проверки, отказ, итог. Здесь же сбор графа улик (mergeGraph) и тексты отказов. Страницу и сеть не трогает; вызывается из app.js.

[336] `web/js/trace.js`

Превращает сырые события агента в строки для блокнота «Case notes». Одна строка на шаг, слова ответа склеены в одну. Таблица TYPE_LABEL даёт короткие метки (route, retrieve, graph query, citation check, refusal), плюс форматирование секунд и сводка состояния сервера healthSummary.

17.4 web/replay/

Записи прогонов: сохранённые потоки событий, которые интерфейс может проиграть без сервера и модели. 13 записей по списку в index.json: 8 настоящих с ноутбука, 4 с заглушкой и 1 написана вручную. Нужны для демо.

Файл	Что внутри
[337] `fake-mccarthy.json`	Запись «ненастоящего» сервера (--fake, ответ заготовлен): вопрос «Are there contradictions in James McCarthy's account?». 63 события, из них 55 слов ответа; есть вывод правил по рассказу Boscombe Valley. Метки времени подправлены под реальный темп.
[338] `fake-refusal.json`	Запись с заглушкой: вопрос «What is the weather in Paris today?» вне корпуса. 5 событий, в конце честный отказ no_evidence («In these materials there is nothing about this»).
[339] `fake-speckled-band.json`	Запись с заглушкой: вопрос «What was the speckled band?». 17 событий, 11 слов ответа; сохранена без правок.
[340] `fake-swamp-adder.json`	Запись с заглушкой: вопрос «What is a swamp adder?», простой фактический вопрос, только текстовый поиск. 26 событий, 20 слов ответа.
[341] `index.json`	Список всех 13 записей прогонов для выпадающего выбора. Для каждой: файл, вопрос, вид (real — настоящая, fake — с заглушкой вместо модели, synthetic — написана вручную), время записи и длительность. Создаётся скриптом scripts/replay_index.py, руками не правят.
[342] `real-abbey-grange-r1-curated.json`	Запись настоящего прогона на ноутбуке: вопрос «Is there a conflict about where Lady Brackenstall was when Sir Eustace burst into the dining-room?». 184 событий (событие — один шаг работы программы: поиск, ответ по словам, проверка). Идёт запрос в граф и проверка правилом (R1) по рассказу Abbey Grange; длительность 1,0 с. Самая крупная после записи сцены (около 86 КБ).
[343] `real-boscombe-control.json`	Запись настоящего прогона на ноутбуке: вопрос «Does James McCarthy's testimony contradict what the gamekeeper William Crowder saw?». 127 событий (событие — один шаг работы программы: поиск, ответ по словам, проверка). Контрольный прогон по Boscombe Valley, длился 25,3 с, MCP-сервер timeline-alibi был запущен. MCP — способ подключать к модели внешние инструменты.
[344] `real-g001-agent.json`	Запись настоящего прогона на ноутбуке: вопрос «What kind of snake killed Dr. Grimesby Roylott?». 22 событий (событие — один шаг работы программы: поиск, ответ по словам, проверка). Простой вопрос без графа, 25,6 с; из 22 событий 16 — слова ответа. Первый шаг демо (g001).
[345] `real-garcia-timeline.json`	Запись настоящего прогона на ноутбуке: вопрос «Give the timeline of the night Aloysius Garcia died.». 267 событий (событие — один шаг работы программы: поиск, ответ по словам, проверка). Хронология ночи смерти Гарсии; 257 событий — слова ответа; запись шла 0,2 с (быстрый ответ).

Файл	Что внутри
[346] `real-refusal-baskerville.json`	Запись настоящего прогона: вопрос о существе, мучившем род Баскервилей. 5 событий; программа честно отказывается (no_evidence), потому что в корпусе нет подходящего текста. Искала по словам baske, creat, devon, famil, haunt.
[347] `real-refusal-fee.json`	Запись настоящего прогона: вопрос о гонораре полковника Росса за поиск Silver Blaze. 7 событий; текст нашёлся, но проверка цитат не прошла, и программа отказалась (no_evidence). Искала по словам fee, large, pay, recov.
[348] `real-scene-board.json`	Запись настоящего прогона на ноутбуке: вопрос «Who can be excluded by alibi in the Wisteria Lodge case?». 561 событий (событие — один шаг работы программы: поиск, ответ по словам, проверка). Самая большая запись (около 158 КБ): доска алиби для места преступления (Oxshott Common, час ночи) — кого текст помещает в другое место. Без вызова модели, это показания, а не доказательство.
[349] `real-wisteria-r1-curated.json`	Запись настоящего прогона на ноутбуке: вопрос «Is there a conflict about where Aloysius Garcia was at about one o'clock on the night he died?». 188 событий (событие — один шаг работы программы: поиск, ответ по словам, проверка). Вывод правилом R1 по Wisteria Lodge, длился 1,1 с, использован граф и инструмент.
[350] `synthetic-derived.json`	Записанная вручную проба, не настоящий прогон: вопрос «SYNTHETIC: does Helen Stoner's alibi hold?». 13 событий, в них две проверки цитат. Нужна, чтобы испытать в интерфейсе «выведенные» факты, панель «Why?» и повтор.

17.5 web/vendor/

Чужие файлы, лежащие в проекте: библиотека Cytoscape.js (сейчас не используется), её лицензия и опись README.md. В подпапке fonts — шрифты.

Файл	Что внутри
[351] `README.md`	Опись сторонних библиотек в папке: Cytoscape.js 3.34.4 (MIT) и пять шрифтов с контрольными суммами sha256, источниками и лицензиями. Отмечает, что с 2026-10-25 Cytoscape не подключается, доска рисуется через DOM+SVG (graph.js), файл пока оставлен.
[352] `cytoscape.LICENSE`	Текст лицензии MIT библиотеки Cytoscape.js, Copyright (c) 2016-2026 The Cytoscape Consortium. Разрешает свободно использовать и копировать при сохранении этого текста.
[353] `cytoscape.esm.min.mjs`	Библиотека Cytoscape.js версии 3.34.4 (рисует графы), в сжатом виде одной строкой, около 434 КБ, без изменений из пакета npm. Сейчас не подключается ни одним скриптом.

17.6 web/vendor/fonts/

Пять шрифтов в формате woff2 и три файла лицензий (OFL и Apache 2.0). Подключаются через css/fonts.css.

[354] **`web/vendor/fonts/EBGaramond-OFL.txt`**
Лицензия SIL Open Font License 1.1 на шрифт EB Garamond (4,4 КБ). Разрешает свободно использовать и встраивать шрифт.

[355] **`web/vendor/fonts/IMFellEnglish-OFL.txt`**
Лицензия SIL Open Font License 1.1 на шрифт IM Fell English (4,5 КБ).

[356] **`web/vendor/fonts/SpecialElite-LICENSE.txt`**
Лицензия Apache 2.0 на шрифт Special Elite (под печатную машинку), около 11 КБ.

[357] **web/vendor/fonts/eb-garamond-italic-latin.woff2**
Шрифт EB Garamond, курсив, латинский набор букв, формат woff2 (сжатый веб-шрифт), 25 КБ. Подключается через css/fonts.css.

[358] **web/vendor/fonts/eb-garamond-latin.woff2**
Шрифт EB Garamond, прямой, изменяемая насыщенность 400–800, латиница, woff2, 44 КБ. Подключается через css/fonts.css.

[359] **web/vendor/fonts/im-fell-english-italic-latin.woff2**
Шрифт IM Fell English, курсив, латиница, woff2, 58 КБ. Подключается через css/fonts.css.

[360] **web/vendor/fonts/im-fell-english-latin.woff2**
Шрифт IM Fell English, прямой, латиница, woff2, 60 КБ. Подключается через css/fonts.css.

[361] **web/vendor/fonts/special-elite-latin.woff2**
Шрифт Special Elite (имитация печатной машинки), латиница, woff2, 53 КБ. Подключается через css/fonts.css.

18 ПАПКА DOCS/

18.1 docs/

Вся документация проекта: требования (PRD), статус и дорожная карта, схема данных и API, описание правил, безопасность, план тестов. Есть подпапки с решениями (adr), макетами, слайдами, замерами и заметками команды.

[362] **docs/DISCLOSURE.md**
Короткая «честная справка» для жюри (обновлена 2026-10-27): что на показе работает офлайн на ноутбуке ThinkPad, что посчитали заранее на других, более мощных машинах и что проверено, а что нет. Например, граф знаний по всем 596 кускам текста строили на других компьютерах, потому что на ноутбуке это заняло бы дни. Часть пунктов ещё помечена «TO BE FILLED» (какая машина какой кусок считала). Опирается на data/index/db.sqlite, data/graph/graph.sqlite, web/replay и docs/team/models.md.

[363] **docs/HANDOFF.md**
Записка «прочти это первым» от 2026-10-23, потом дополнялась до 2026-10-29. Описывает, где остановилась работа: например, проверенные вручную («курируемые») факты для дела Wisteria Lodge в data/curated/presence_demo.jsonl и индекс по предложениям data/index/sentences.sqlite. Указывает, какие ветки не отправлены и что запускать. Отсылает к ROADMAP, symbolic-layer.md, data-schema.md, api.md.

[364] **docs/PRD.md**
Требования к продукту (PRD, «что и зачем делаем»), черновик 2 от 2026-10-23. Объясняет идею: нейросеть достаёт факты из текста («extracted»), правила выводят проверяемые выводы («derived»), на каждом утверждении стоит метка. Перечисляет пользователей (жюри, зрители, разработчики), цели и ограничения по скорости. Опирается на docs/spike/REPORT.md; цифры-цели там, где нет замеров, оставлены TBD («ещё не определено»).

[365] **docs/ROADMAP.md**
Дорожная карта до показа 2026-10-30, написана вечером 2026-10-25. Показывает критический путь (извлечение на мощных машинах, склейка кэша, сборка графа, подключение стека, замеры, заморозка и репетиции) и расписание по дням. Заморозка функций: 2026-10-28 18:00. Отсылает к HANDOFF.md и DISCLOSURE.md.

[366] **`docs/STATUS.md`**
Журнал состояния проекта, последняя запись от 2026-10-28 (теги rc1 и rc2). Содержит итоги экспериментов: выбор «кто виновен» небезопасен, семантическое правило R3 не помогло, тестовый прогон один раз: агент правильно в 13 из 30 вопросов против 3 из 30 у голой модели. Граф по всем 596 кускам из 20 рассказов построен. Ссылается на файлы в docs/spike/results.

[367] **`docs/api.md`**
Описание HTTP API версии 1. Основной вызов POST /api/ask отдаёт ответ потоком SSE (события приходят по одному), есть GET-вызовы /api/graph/subgraph, /api/graph/entities, /api/source/{chunk_id}, /api/health, /api/config. Ошибки всегда в виде JSON {error: {code,message}}. Запуск: python -m detective_board.api, режим --fake без Ollama. Код в src/detective_board/api/, тесты в tests/api/.

[368] **`docs/backlog.md`**
Бэклог (список задач) в формате «ID | исполнитель | задача | критерий | приоритет | день»: 47 пунктов, например T-36 (модуль правил R1-R3 на чистом Python) и T-01 (разбить корпус на 596 кусков). Приоритеты P0-P2 и порядок, что отрезать первым (анимации, второй MCP-сервер). Дни с 2026-10-23 по 29.

[369] **`docs/data-schema.md`**
Схема данных. Запись куска текста в data/chunks.jsonl (поля chunk_id, story, char_start, text и др.), таблицы поиска SQLite (FTS5 + sqlite-vec), узлы и рёбра графа, события трассы. Исходно DRAFT фазы 0; потом дополнялся (раздел 2b — индекс по предложениям). Источники: scripts/corpus/ingest.py, extract/schema.py.

[370] **`docs/mcp-contracts.md`**
Контракты MCP-серверов: общие правила, свой сервер timeline-alibi (build_timeline, check_alibi, find_overlaps), необязательный evidence-notes, модель ошибок, таймауты и белые списки для готовых серверов (разделы 1-8). Статус DRAFT фазы 0, позже есть раздел о реализации в src/detective_board/mcp/. Основа — ADR-0003 и db-research.md.

[371] **`docs/security.md`**
Обзор безопасности от 2026-10-23 (только чтение, код не менялся): 7 тем — инъекция через промпт, безопасность MCP-инструментов, API, XSS во фронтенде, SQL-инъекции, секреты и цепочка поставок, лицензии данных. Таблица со статусами и самой высокой серьёзностью (Medium). Охватывает src/, web/, scripts/, tests/, pyproject.toml, .github/.

[372] **`docs/symbolic-layer.md`**
Главный документ о правилах (875 строк, 12 разделов). Принцип: нейросеть достаёт факты, правила R1-R3 делают точные выводы, одинаковые на одном и том же графе. R1 — один человек в двух местах одновременно, R2 — порядок «до/после», R3 — слова свидетеля против установленного факта. Разделы 7-11: форма реализации, интерфейс Python, мини-тесты, заметки о реальных прогонах. Исходно DRAFT, потом дополнен.

[373] **`docs/test-plan.md`**
План тестирования (2026-10-22, фаза 0): пирамида — unit около 70 %, integration около 20 %, live, e2e через Playwright, eval. Правило: баг в высоком слое получает регрессионный тест в самом низком. Многое помечено TODO. Инструменты: pytest, pytest-cov, ruff, pyright.

18.2 docs/acceptance/

Одна папка с критериями приёмки: что именно должна уметь система, чтобы считаться готовой.

[374] `docs/acceptance/acceptance.md`
Критерии приёмки в формате Given/When/Then («дано / когда / тогда»): 25 пунктов AC-01… AC-25. Например, AC-01: на фактический вопрос ответ содержит дословную цитату с указанием рассказа и куска; AC-03: на вопрос вне корпуса агент честно отказывается. Пороги, помеченные TBD, определены в docs/PRD.md. По этим пунктам пишутся тесты и проверяется демо.

18.3 docs/adr/

Записи об архитектурных решениях (ADR): 12 пронумерованных файлов плюс шаблон и оглавление. В каждом контекст, варианты, решение и последствия; часть решений ещё предварительные.

Файл	Что внутри
[375] `0000-template.md`	Пустой шаблон для ADR (записи об архитектурных решениях). Поля: статус, дата, кто решал, что провизорно, контекст, варианты, решение, последствия. По нему написаны ADR 0001-0012.
[376] `0001-repo-and-tooling.md`	ADR-0001 (2026-10-22): выбор инструментов. Один репозиторий (монорепо), Python 3.12, менеджер пакетов uv, запуск команд через just, линтер ruff. Проверка типов (pyright или mypy) ещё не выбрана, поэтому часть записи «Proposed». Опирается на pyproject.toml, uv.lock, justfile.
[377] `0002-storage.md`	ADR-0002 (2026-10-22, обновлён 2026-10-23): где хранить данные. Решили: один файл SQLite с полнотекстовым поиском FTS5 и векторным расширением sqlite-vec (вектор = набор чисел, описывающий смысл текста); граф тоже в SQLite, база Ladybug отложена. Сравнивались LanceDB, Chroma, Qdrant. Ссылается на замеры docs/spike/results/arms-20261023-080849.md.
[378] `0003-mcp.md`	ADR-0003 (2026-10-22, статус Proposed): про MCP (стандартный способ дать модели вызывать инструменты). Решили сделать свой сервер timeline-alibi (инструменты build_timeline, check_alibi, find_overlaps) плюс один готовый сервер файлов с ограничением доступа. SDK: mcp версии 2.3 и выше, но меньше 3. Контракты вынесены в docs/mcp-contracts.md.
[379] `0004-symbolic-layer.md`	ADR-0004 (2026-10-22, Proposed): как устроен «символический слой» — обычные правила на Python, которые читают граф и без нейросети делают проверяемые выводы с объяснением (R1, R2, R3). Сравнивали pyDatalog и другие варианты, выбрали простой Python. Подробности в docs/symbolic-layer.md, данные в docs/data-schema.md.
[380] `0005-web-stack.md`	ADR-0005 (2026-10-22, Accepted): веб-часть. Сервер Starlette + uvicorn, поток событий SSE (сервер по одному шлёт браузеру сообщения) оформлен вручную, интерфейс на чистом JS-модулями без сборки, граф рисует Cytoscape.js 3.34.4, лежащий в репозитории. Всё работает без сети. Это основа docs/api.md и каталога web/.
[381] `0006-embedding-model.md`	ADR-0006 (2026-10-23, Accepted): выбор модели эмбеддингов (эмбеддинг — набор чисел для смысла текста) — bge-m3 через Ollama. В таблице сравнение четырёх моделей на 35 вопросах: hit@5 у bge-m3 0,89, и только она хорошо понимает русские вопросы. Поиск гибридный: FTS5 + векторы, слияние RRF. Цифры из docs/spike/REPORT.md.
[382] `0007-extraction-model.md`	ADR-0007 (2026-10-23, Proposed): какая модель достаёт сущности и факты из куска текста. Выбрана qwen3:4b: на 12 кускам верный JSON 12/12, 0,934 цитат дословно, около 180 с на кусок. Оговорка: эталон писал ИИ, человек его не проверял. Сравнивались ещё 5 моделей.
[383] `0008-answer-model.md`	ADR-0008 (2026-10-23, Accepted): модель для ответов — qwen3:4b-instruct. В таблице скорость шести моделей (например, qwen3:8b 4,34 токена/с, первый токен 29,5 с). Выбор по проверке на 12 вопросах, уверенность средняя. Проверка запускается через eval/run_answer_check.py.

Файл	Что внутри
[384] `0009-extractive-answers.md`	ADR-0009 (2026-10-27, Accepted): «экстрактивные ответы» — модель не пишет текст, а выбирает готовые предложения из рассказа. Причина: на ноутбуке медленно, а 4B-модель часто отказывалась отвечать, хотя ответ был в тексте. Опирается на docs/perf/ttft-levers-2026-10-26.md и прогон 12 вопросов (n=12, один раз).
[385] `0010-rule-first-answers.md`	ADR-0010 (2026-10-27, Accepted): для вопросов про противоречия и алиби ответ определяют правила, а не языковая модель. Есть «защита честности»: если правило ничего не нашло, система не имеет права писать «противоречий нет, всё согласовано». Основа — первый реальный прогон на графе wisteria-lodge из docs/symbolic-layer.md.
[386] `0011-presence-pass-and-curated-facts.md`	ADR-0011 (2026-10-27): как получить факты «кто был где и когда». Автоматический проход «присутствия» на малой модели дал мало (в таблице две попытки), поэтому для показа используются проверенные человеком («курируемые») факты, помеченные меткой curated. Человек перепроверил их 2026-10-29. Связан с HANDOFF.md, разделом 11.4 symbolic-layer.md.
[387] `0012-extraction-prompt-v2-1-plus-repair.md`	ADR-0012 (принято 2026-10-26, записано 2026-10-27): оставить промпт извлечения (инструкцию для модели) версии 2.1 и добавить детерминированную «починку» вывода. Версию 2.2 пробовали на 12 эталонных кусках — стала хуже по многим метрикам. Починка проверяется скриптом scripts/eval/repair_eval.py.
[388] `README.md`	Оглавление всех ADR: таблица с номером, названием, статусом и пометкой, что в решении ещё провизорно. Даёт ссылки на файлы 0001-0012 и шаблон.

18.4 docs/architecture/

Схема архитектуры (C4) одним файлом c4.md.

[389] `docs/architecture/c4.md`

Схема архитектуры по методу C4 (уровни контекста и контейнеров, диаграммы Mermaid) и описание потоков данных. Статус DRAFT фазы 0 — написан до PRD, поэтому с ним не сверен. Показывает: пользователь, Detective Board, Ollama, внешние MCP-серверы. Опирается на docs/spike/db-research.md и extract/schema.py.

18.5 docs/demo/

Материалы для сцены: чек-лист, журнал репетиций, ответы на вопросы жюри, примеры ошибок голой модели и инструкция по резервной записи экрана.

[390] `docs/demo/README.md`

Оглавление материалов для показа и инструкция по резервной записи экрана (issue #15): один ролик на шаг и один полный; mp4 1920x1080, имена вида demo-<tag>-step<N>-<short>.mp4, записываются через OBS или ffmpeg. Файлы большие и в git не кладутся. Записанные прогоны для интерфейса лежат в web/replay/.

[391] `docs/demo/STAGE_CHECKLIST.md`

Чек-лист для сцены, который печатают: что сделать накануне вечером (зарядка, архив на флешке, проверка sha256), за час на месте и точные слова на случай каждой поломки. Главный тест: свежий just run из архива без сети; три вопроса (g001, Garcia R1, отказ).

[392] `docs/demo/bare-vs-agent.md`

Таблица ошибок «голой» модели (без источников) против агента на первых вопросах dev-выборки, прогон 2026-10-28. Например, на g001 голая модель говорит, что Ройлотта змея не

убивала. Используется на слайдах. Источник: docs/spike/results/eval-2026-10-28-dev-…-rescored.json.

[393] **docs/demo/jury-faq.md**
Заготовленные ответы на трудные вопросы жюри: раздел A (новый, числа от 2026-10-29) и раздел B (старый черновик, 37 вопросов). Каждый ответ помечен BUILT (построено и проверено), DESIGNED (спроектировано, проверено только на условных данных) или TBD. Статус: DRAFT 4, 2026-10-29. Опирается на docs/spike/results.

[394] **docs/demo/rehearsals.md**
Пустой журнал трёх репетиций с секундомером (все ячейки TBD) и таблица учебных сбоев: остановить Ollama, замедлить модель, выключить Wi-Fi и др. Цель: три прогона короче 7:00, восстановление за 20 с по словам из STAGE_CHECKLIST.md.

18.6 docs/design/

Дизайн интерфейса «доска расследования»: описание, цветовые токены, скрипт проверки контраста, сценарий показа и три снимка экрана.

[395] **docs/design/board-design.md**
Дизайн «доски расследования» — единого экрана с вопросом, графом и ходом работы агента. Холст 1920x1080, минимум 1280x720; разделы: цвета, шрифты, анимации (все можно вырезать), доступность, пустые и ошибочные состояния. Статус DRAFT 1 (2026-10-22), позже дописаны разделы 9-11 о реализации в web/. Опирается на tokens.css и contrast_check.py.

[396] **docs/design/contrast_check.py**
Скрипт на Python (только стандартная библиотека), который проверяет контрастность цветов из tokens.css по правилам WCAG: 4,5 для текста и 3,0 для значков. Читает светлую и тёмную темы, считает яркость, печатает таблицу и завершается с кодом 1, если какая-то пара не проходит. Запуск: python3 docs/design/contrast_check.py.

[397] **docs/design/demo-script.md**
Сценарий семиминутного показа (DRAFT 4, 2026-10-27): ограничения, шаги, карточки оператора (раздел 2a) под колоду на 6:30, что говорить и что делать при сбое. Показ 2026-10-30, на английском. Шаг 2 — случай Garcia (правило R1). Опирается на docs/slides/deck.md, HANDOFF.md, spike/results.

[398] **docs/design/tokens.css**
CSS-файл дизайн-токенов (именованных значений) — цвета и размеры с именами вида --db-<группа>-<имя>. Светлая тема по умолчанию («дневной кабинет»), тёмная («ночь на Бейкер-стрит») включается по настройке ОС или data-theme="dark". 284 строки. Контраст проверяет contrast_check.py; на него опираются слайды и интерфейс.

18.7 docs/design/screens/

Три снимка экрана доски (800x482 JPEG): пустая, посреди запуска, итог с противоречием.

[399] **docs/design/screens/1-empty.jpg**
Снимок экрана доски: пустое состояние до вопроса. JPEG 800x482, 42 КБ. Иллюстрация к docs/design/board-design.md.

[400] **docs/design/screens/2-mid-run-prefill.jpg**
Снимок экрана доски посреди запуска, пока модель читает запрос (prefill). JPEG 800x482, 68 КБ. Иллюстрация к docs/design/board-design.md.

[401] `docs/design/screens/3-final-contradiction.jpg`
Снимок экрана доски в конце: найденное противоречие. JPEG 800x482, 75 КБ. Иллюстрация к docs/design/board-design.md.

18.8 docs/perf/

Замеры скорости и памяти на ноутбуке ThinkPad: сырые данные JSON и отчёты Markdown, проверка офлайн-установки, эксперимент по ускорению первого токена.

[402] `docs/perf/clean-install-20261029.md`
Отчёт о сборке и проверке офлайн-архива rc4 (2026-10-29): tar 6,2 ГБ, распакован в чистую папку, проверен без сети; 976 тестов прошли, 2 пропущены. Вопросы к API: g001 за 148,3 с, c001 R1 за 4,6 с. Проверена одна машина автора, тест на двух разных ноутбуках не сделан.

[403] `docs/perf/laptop-20261028.json`
Сырые замеры ноутбука от 2026-10-28: 6 записей (g001, g002, g003, g007, g009, g010) со временем ответа, пиком памяти и температурой плюс общие максимумы (5017 МБ, 99 градусов). Данные для таблицы в laptop-20261028.md.

[404] `docs/perf/laptop-20261028.md`
Таблица замеров на ThinkPad X1 Carbon (2026-10-28, 15:05): шесть вопросов dev-выборки через реальный API; ответ за 30-43 с, первый (холодный старт) 43,2 с. Оговорка: на машине работали VPN и Docker, значит это верхняя оценка. Сырые данные — laptop-20261028.json.

[405] `docs/perf/laptop-20261029-3x6.json`
Сырые строки замера 2026-10-29: три серии по 6 вопросов (18 записей), у каждой id, холодный ли старт, статус, ttft_s, total_s, память и температура. Источник цифр в laptop-20261029.md.

[406] `docs/perf/laptop-20261029.md`
Итоги трёх серий замеров на ноутбуке (2026-10-29): медиана по 16 отвеченным вопросам 28,4 с, диапазон 18,6-44,1 с, 2 из 18 отказов, пик памяти 6289 МБ, температура до 100 градусов. Вопросы подобраны разные, чтобы кэш префикса не ускорял повторы.

[407] `docs/perf/ttft-levers-2026-10-26.md`
Эксперимент «что ускоряет первый токен» (2026-10-26): обработка запроса в 1080 токенов занимает около 43-53 с; потоки, размер батча и num_ctx не помогли, помог только кэш неизменного системного промпта (минус 8-10 с). Основание для ADR-0009.

18.9 docs/slides/

Слайды доклада на 6:30: русская колода (md, html, pdf), английский архив, две темы Marp в CSS, таблица источников чисел и README.

[408] `docs/slides/README.md`
Инструкция к слайдам (English): основной доклад идёт с русской колоды, а deck.md — архив и источник чисел. Как собрать PDF офлайн через Marp CLI (npx @marp-team/marp-cli) и что лежит в папке.

[409] `docs/slides/README.ru.md`
Русская инструкция к слайдам: deck.ru.md — основная колода (10 слайдов на 6:30), deck.md — английский архив, numbers.md — источники чисел. Как собирать PDF с тёмной темой; цитаты из рассказов остаются по-английски.

[410] **docs/slides/deck.md**
Английская колода для Marp (Markdown со слайдами, разделёнными ---): 10 основных слайдов на 6:30 и запасные B1-B6. В комментарии каждого слайда — хронометраж, текст речи, действие, числа и что делать при сбое. Тема detective. Архив и источник чисел для русской версии.

[411] **docs/slides/deck.ru.html**
HTML-версия русской колоды, собранная из deck.ru.md (по виду — Marp), 128 КБ, 17 блоков section. Открывается в браузере как презентация. Назначение подтверждено шапкой документа и заголовком.

[412] **docs/slides/deck.ru.md**
Основная русская колода для Marp (46 КБ, тема detective-night): 10 основных слайдов на 6:30 плюс запасные; в комментариях текст речи и подсказки при сбое. Порядок слайдов и числа совпадают с deck.md.

[413] **docs/slides/deck.ru.pdf**
PDF русской колоды, 2,6 МБ. Содержимое по тексту не прочитано; по имени и README.ru.md это экспорт deck.ru.md.

[414] **docs/slides/detective-night.css**
Тёмная тема слайдов Marp «ночь на Бейкер-стрит»: подключает detective.css и меняет только цвета (значения из тёмного блока tokens.css), добавляет тёплое «свечение лампы» слева вверху.

[415] **docs/slides/detective.css**
Светлая тема слайдов Marp в стиле интерфейса: деревянная рамка, бумага, красная нить, медная кнопка, машинописные подписи. Подключает шрифты IM Fell English, EB Garamond, Special Elite из web/vendor/fonts. Используется deck.md.

[416] **docs/slides/numbers.md**
Таблица «каждое число на слайде»: номер слайда, число, что оно значит, файл-источник и количество прогонов n. Правило: числа на слайде допустимы только из этой таблицы. Строки TBD ещё без источника. Пример: g001 на 9,3 с из api12-20261027-extractive.md.

18.10 docs/spike/

Файлы фазы 0 (разведка): отчёт с замерами и кабинетное исследование хранилищ. Сырые результаты лежат в docs/spike/results (в эту группу не входят).

[417] **docs/spike/REPORT.md**
Отчёт фазы 0 («разведка», 2026-10-23): выбор моделей и хранилища. Таблица экспериментов (скорость LLM, эмбеддинги, поиск BM25/вектор/гибрид, извлечение, проверка модели ответов) и ссылки на файлы в docs/spike/results. Все числа привязаны к файлам. Раздел 9 про проверку модели ответов на момент записи был в статусе PENDING.

[418] **docs/spike/db-research.md**
Кабинетное исследование 2026-10-22: сравнение хранилищ векторов (lancedb, chromadb и др.), графовых баз, BM25, ранжировщиков и MCP. Ничего не устанавливали и не запускали, всё из официальных страниц; таблицы с размером пакета и рисками. Основа для ADR-0002 и 0003.

18.11 docs/spike/results/_aborted/

Оборванные прогоны проверки моделей-ответчиков: в двух файлах только 4 и 6 записей из 12. Оставлены как след, но не используются; полные прогоны лежат в соседней папке.

[419] docs/spike/results/_aborted/answers-20261023-082109-qwen3_4b.jsonl

Оборванный прогон проверки моделей-ответчиков: в файле только 4 записи (g001, g007, g028, g032) из 12 вопросов, модель qwen3:4b. JSONL — это текстовый файл, где одна строка — один JSON-объект; в записи вопрос, найденные куски текста, статус ответа, время, число токенов и сам ответ. Размер 21 КБ; полный прогон лежит в answers-20261023-v3-*.jsonl.

[420] docs/spike/results/_aborted/answers-20261023-v2-qwen3_4b-instruct.jsonl

Тоже оборванный прогон: 6 записей из 12 (g001–g019) для qwen3:4b-instruct, версия v2 проверки. Поля те же, что в v3: вопрос, маршрут поиска, найденные куски, статус ответа, время, покрытие. Размер 11 КБ; заменён полным прогоном v3.

18.12 docs/spike/results/

Результаты замеров исследовательской («спайк») фазы проекта: для каждого замера обычно пара файлов — .md (читаемая сводка) и .json (сырые данные), иногда .jsonl, .svg или журналы. Замеры: скорость и память языковых моделей и эмбеддингов, качество поиска, извлечение фактов, ответы агента, тест «до разоблачения», оценки dev/test.

Файл	Что внутри
[421] ab-prompt-v2_1.md	Сравнение двух версий подсказки для извлечения фактов (v2.0 и v2.1) на модели qwen3:4b и 4 проверочных кусках. Новая v2.1 просит короче цитаты и меньше сущностей: на кусок уходит 189.9 с вместо 252.0 (быстрее примерно на 25%), но оставляет 59 пунктов вместо 82. Качество оставленного не оценивалось. v2.1 стала подсказкой по умолчанию.
[422] answers-20261023-v3-BARE-qwen3_4b-instruct.jsonl	Ответы «голой» модели qwen3:4b-instruct без поиска по рассказам: 12 записей на те же 12 золотых вопросов. В каждой записи вопрос, ожидаемое поведение, ответ, время до первого токена, число токенов, скорость и автопометка, отказалась ли модель. Нужны как контроль к прогонам с поиском; 21 КБ.
[423] answers-20261023-v3-gemma3_4b.jsonl	Ответы агента (поиск по рассказам + модель) с моделью gemma3:4b на 12 золотых вопросов: 12 записей. В записи маршрут поиска, найденные куски, попал ли золотой кусок, статус (ответила/отказалась/частично), покрытие, цитаты, время. По сводке в answers-20261023-v3.md: 4 из 10 ответимых вопросов отвечены и проверены; 27 КБ.
[424] answers-20261023-v3-qwen3_4b-instruct.jsonl	Ответы агента с моделью qwen3:4b-instruct на 12 золотых вопросов (g001, g007, g028 и т.д.): 12 записей с маршрутом, найденными кусками, статусом, цитатами и временем. В сводке: 5 из 10 ответимых вопросов отвечены с проверкой, среднее время 59.5 с; 25 КБ.
[425] answers-20261023-v3-qwen3_8b.jsonl	Ответы агента с большей моделью qwen3:8b на те же 12 золотых вопросов: 12 записей тех же полей. В сводке 5 из 10 ответимых вопросов отвечены и проверены, но медленно: в среднем 141.5 с на вопрос против 59.5 у qwen3:4b-instruct; 25 КБ.
[426] answers-20261023-v3.md	Сводка проверки трёх моделей-ответчиков (qwen3:4b-instruct, gemma3:4b, qwen3:8b) на 12 золотых вопросах. Используется реальный Agent только с поиском (без графа и MCP-инструментов). Верное поведение: 8/12 у qwen3:4b-instruct, 6/12 у gemma3:4b, 9/12 у qwen3:8b; ни одна не справилась с русским вопросом. Есть таблицы по категориям и по каждому вопросу.
[427] api12-20261027-extractive.md	Прогон 12 контрольных вопросов через настоящий сервер (POST /api/ask) в «извлекающем» режиме: модель только выбирает одно-два предложения из текста, ответ — эти предложения дословно. Ответы приходят за 8–20 с вместо 45–140 с, вопросы вне текста снова отвергаются. Не отвечены g007, g016 и русский g062; n=12, один прогон.

Файл	Что внутри
[428] `api12-20261027-generative.md`	Базовый прогон тех же 12 вопросов через сервер в «генеративном» режиме, где модель сама пишет ответ (с повтором после отказа). Много ответов получили статус partial («вот ближайшие отрывки»), некоторые вопросы шли 106–140 с, а g056/g057 вместо отказа тоже дали отрывки. Это точка сравнения ДО перехода на извлекающий режим.
[429] `arms-20261023-080849.json`	Полный дамп сравнения «рук» поиска (BM25 по словам, вектор bge-m3, смесь RRF) на 596 кусках и 35 вопросах. В нём ключи timestamp, model, best, configs (таблица метрик hit@k, MRR, recall@10 для каждого варианта), buckets, failures_best_k5. Сводка в парном arms-20261023-080849.md; 540 КБ.
[430] `arms-20261023-080849.md`	Сравнение способов поиска по рассказам на 35 вопросах: только слова (BM25), только вектор, их смесь с разными весами. Лучшая смесь — hyb[or] k=30 w=0.5:1: hit@5 = 0.97, MRR 0.75, против 0.89 у одного вектора и 0.69 у BM25. Режим AND для слов даёт 0, поэтому берут OR.
[431] `curated-review-2026-10-29.md`	Протокол проверки вручную подготовленных «курируемых» фактов (8 фактов о том, кто где был, и 1 сцена), задача #22. Сначала ИИ сверил каждую цитату с текстом и исправил 5 заметок, потом человек (LitrumCoder) перечитал всё и расхождений не нашёл. Все 9 цитат дословно есть в тексте; отдельно отмечено, что время «около часа» — слова самого Гарсии.
[432] `dev-failure-analysis-20261028.md`	Разбор ошибок агента на 27 ответимых dev-вопросах: 6 верных, 4 верных, но строгая метрика занижает, 7 неверное предложение при найденном куске, 5 ложных отказов, 5 промахов поиска. Главный вывод: по честной метрике агент (47%) не лучше одного векторного поиска (59%), разница в пределах шума. Опирается на eval-...-rescored.json.
[433] `embed-20261022-180354.json`	Сырые данные того же замера эмбеддинг-модели bge-m3: параметры запуска (args), версия Ollama 0.35.0, время каждого прохода, замеры памяти и результаты поиска по вопросам. Парная сводка — embed-20261022-180354.md; 15 КБ.
[434] `embed-20261022-180354.md`	Тест эмбеддинг-модели bge-m3 (эмбеддинг — превращение текста в набор чисел, по которому ищут похожее). Размерность 1024, скорость 0.46 куска/с, пик памяти 2076 МБ памяти; на 596 кусках: hit@5 = 1.00 и на английском, и на русском. Выборка мала (15 английских и 5 русских вопросов), это проверка «на дым», не рейтинг.
[435] `embed-20261022-181410.json`	Сырые данные того же замера эмбеддинг-модели nomic-embed-text с приставками search_query/search_document: параметры запуска (args), версия Ollama 0.35.0, время каждого прохода, замеры памяти и результаты поиска по вопросам. Парная сводка — embed-20261022-181410.md; 15 КБ.
[436] `embed-20261022-181410.md`	Тест эмбеддинг-модели nomic-embed-text с приставками search_query/search_document (эмбеддинг — превращение текста в набор чисел, по которому ищут похожее). Размерность 768, скорость 1.29 куска/с, пик памяти 457 МБ; на 596 кусках: русский hit@5 всего 0.20. Выборка мала (15 английских и 5 русских вопросов), это проверка «на дым», не рейтинг.
[437] `embed-20261022-182431.json`	Сырые данные того же замера эмбеддинг-модели nomic-embed-text без приставок: параметры запуска (args), версия Ollama 0.35.0, время каждого прохода, замеры памяти и результаты поиска по вопросам. Парная сводка — embed-20261022-182431.md; 15 КБ.
[438] `embed-20261022-182431.md`	Тест эмбеддинг-модели nomic-embed-text без приставок (эмбеддинг — превращение текста в набор чисел, по которому ищут похожее). Размерность 768, скорость 1.31 куска/с, пик памяти 457 МБ; на 596 кусках: английский hit@5 0.93, русский 0.00. Выборка мала (15 английских и 5 русских вопросов), это проверка «на дым», не рейтинг.

Файл	Что внутри
[439] `embed-20261022-182604.json`	Сырые данные того же замера эмбеддинг-модели all-minilm: параметры запуска (args), версия Ollama 0.35.0, время каждого прохода, замеры памяти и результаты поиска по вопросам. Парная сводка — embed-20261022-182604.md; 15 КБ.
[440] `embed-20261022-182604.md`	Тест эмбеддинг-модели all-minilm (эмбеддинг — превращение текста в набор чисел, по которому ищут похожее). Размерность 384, скорость 9.36 куска/с, пик памяти 172 МБ; на 596 кусках: английский 0.93, русский 0.00. Выборка мала (15 английских и 5 русских вопросов), это проверка «на дым», не рейтинг.
[441] `embed-20261022-185253.json`	Сырые данные того же замера эмбеддинг-модели snowflake-arctic-embed: параметры запуска (args), версия Ollama 0.35.0, время каждого прохода, замеры памяти и результаты поиска по вопросам. Парная сводка — embed-20261022-185253.md; 15 КБ.
[442] `embed-20261022-185253.md`	Тест эмбеддинг-модели snowflake-arctic-embed (эмбеддинг — превращение текста в набор чисел, по которому ищут похожее). Размерность 1024, скорость 0.49 куска/с, пик памяти 780 МБ; на 596 кусках: английский 1.00, русский 0.20. Выборка мала (15 английских и 5 русских вопросов), это проверка «на дым», не рейтинг.
[443] `entity-answer-2026-10-29.md`	Отчёт об экспериментальной ветке exp/entity-answer: на вопросы «кто сделал» модель выбирает человека из списка кандидатов графа. Вывод: включать нельзя и для демо небезопасно. На отложенных историях называет невинного (Нелигана) в 2 из 8 ответов, виновного указывает 1 из 8 против 2 из 8 при выключенном режиме. Описаны три варианта подсказки V1–V3.
[444] `eval-2026-10-28-dev-4bceaf8-dcf42c8a3eb8-162624.json`	Сырые данные того же замера: meta (коммит, хэш настроек), summary по конфигурациям, rows с ответом на каждый вопрос в каждой конфигурации, contradiction_rows по случаям противоречий. Сводка — в парном .md.
[445] `eval-2026-10-28-dev-4bceaf8-dcf42c8a3eb8-162624.md`	Отчёт замера качества агента: dev-набор, коммит 4bceaf8; запущена только конфигурация hybrid+graph: верное поведение 14 из 34 (0.41), ложных отказов 5/27. Контрольные варианты bare и vector отмечены «not run». Модель qwen3:4b-instruct. Таблица метрик с интервалами Уилсона (доверительный интервал 95%) и блок правил R1–R3 по противоречиям, где найдено 0. Золотые вопросы написаны ИИ, человек их ещё не проверял.
[446] `eval-2026-10-28-dev-4bceaf8-dcf42c8a3eb8-162624.svg`	Столбиковая картинка «Behaviour correct, Wilson 95 %» с долей верного поведения по конфигурациям и усами интервала; рисуется из данных того же замера. Размер около 1.5–1.9 КБ, это векторный рисунок (SVG) для вставки в отчёт.
[447] `eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202-rescored.json`	Сырые данные того же замера: meta (коммит, хэш настроек), summary по конфигурациям, rows с ответом на каждый вопрос в каждой конфигурации, contradiction_rows по случаям противоречий. Сводка — в парном .md.
[448] `eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202-rescored.md`	Отчёт замера качества агента: те же ответы dev, пересчитанные после исправления ошибки: bare 4/34 (0.12), vector 15/34 (0.44), hybrid+graph 12/34 (0.35). Модель qwen3:4b-instruct. Таблица метрик с интервалами Уилсона (доверительный интервал 95%) и блок правил R1–R3 по противоречиям, где найдено 0. Золотые вопросы написаны ИИ, человек их ещё не проверял.
[449] `eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202-rescored.svg`	Столбиковая картинка «Behaviour correct, Wilson 95 %» с долей верного поведения по конфигурациям и усами интервала; рисуется из данных того же замера. Размер около 1.5–1.9 КБ, это векторный рисунок (SVG) для вставки в отчёт.
[450] `eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202.json`	Сырые данные того же замера: meta (коммит, хэш настроек), summary по конфигурациям, rows с ответом на каждый вопрос в каждой конфигурации, contradiction_rows по случаям противоречий. Сводка — в парном .md.

Файл	Что внутри
[451] `eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202.md`	Отчёт замера качества агента: dev-набор, коммит ab945e8, первый полный прогон четырёх конфигураций. УСТАРЕЛ: метрика верного поведения занижена из-за ошибки (6/34 для агента). Модель qwen3:4b-instruct. Таблица метрик с интервалами Уилсона (доверительный интервал 95%) и блок правил R1–R3 по противоречиям, где найдено 0. Золотые вопросы написаны ИИ, человек их ещё не проверял.
[452] `eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202.svg`	Столбиковая картинка «Behaviour correct, Wilson 95 %» с долей верного поведения по конфигурациям и усами интервала; рисуется из данных того же замера. Размер около 1.5–1.9 КБ, это векторный рисунок (SVG) для вставки в отчёт.
[453] `eval-2026-10-28-dev-f8f1d0c-e1d16fd71889-170618.json`	Сырые данные того же замера: meta (коммит, хэш настроек), summary по конфигурациям, rows с ответом на каждый вопрос в каждой конфигурации, contradiction_rows по случаям противоречий. Сводка — в парном .md.
[454] `eval-2026-10-28-dev-f8f1d0c-e1d16fd71889-170618.md`	Отчёт замера качества агента: dev-набор, коммит f8f1d0c; только hybrid+graph, снова 14 из 34 (0.41); в разборе противоречий добавлен столбец found_with_curated_facts. Модель qwen3:4b-instruct. Таблица метрик с интервалами Уилсона (доверительный интервал 95%) и блок правил R1–R3 по противоречиям, где найдено 0. Золотые вопросы написаны ИИ, человек их ещё не проверял.
[455] `eval-2026-10-28-dev-f8f1d0c-e1d16fd71889-170618.svg`	Столбиковая картинка «Behaviour correct, Wilson 95 %» с долей верного поведения по конфигурациям и усами интервала; рисуется из данных того же замера. Размер около 1.5–1.9 КБ, это векторный рисунок (SVG) для вставки в отчёт.
[456] `eval-2026-10-28-test-7b7b197-7857839ed9be-175321.json`	Сырые данные того же замера: meta (коммит, хэш настроек), summary по конфигурациям, rows с ответом на каждый вопрос в каждой конфигурации, contradiction_rows по случаям противоречий. Сводка — в парном .md.
[457] `eval-2026-10-28-test-7b7b197-7857839ed9be-175321.md`	Отчёт замера качества агента: тестовое разбиение (30 вопросов, 20 случаев противоречий), коммит 7b7b197, запущено один раз по предварительной регистрации: bare 3/30 (0.10), vector 11/30 (0.37), hybrid+graph 13/30 (0.43); разница vector и hybrid+graph незначима (McNemar p=0.5). Модель qwen3:4b-instruct. Таблица метрик с интервалами Уилсона (доверительный интервал 95%) и блок правил R1–R3 по противоречиям, где найдено 0. Золотые вопросы написаны ИИ, человек их ещё не проверял.
[458] `eval-2026-10-28-test-7b7b197-7857839ed9be-175321.svg`	Столбиковая картинка «Behaviour correct, Wilson 95 %» с долей верного поведения по конфигурациям и усами интервала; рисуется из данных того же замера. Размер около 1.5–1.9 КБ, это векторный рисунок (SVG) для вставки в отчёт.
[459] `evidence-20261023-093201.json`	Сырые данные замера evidence: 85 строк-вариантов (стратегия, top_k, ширина окна W, покрыто, доверительный интервал), таблицы pareto для бюджетов 1000/1200/1500 токенов, chunk_level_by_top_k и список 12 непокрытых вопросов. Парная сводка — evidence-20261023-093201.md; 39 КБ.
[460] `evidence-20261023-093201.md`	Замер без LLM: сколько доказательств попадает в подсказку модели. На 86 ответимых вопросах: золотой кусок в топ-4 найден у 70 из 86 (81%), в топ-8 у 74 (86%). При бюджете до 1000 токенов лучшая стратегия rank-2x (top_k 4, окно 80 слов) покрывает цитату в 65% (56/86). Числа оптимистичны, так как подгонялись на этих же вопросах.
[461] `extract-gemma3_4b-20261022-194124.jsonl`	Сырые ответы модели gemma3:4b при извлечении фактов из 12 золотых кусков текста: 12 строк (chunk_id, output — сырой JSON-текст, wall_s, число токенов, done_reason, mem_available_mib). Среднее время на кусок 185.0 с, без ошибок; num_predict 1536. Нужны, чтобы сравнить модели по качеству и скорости извлечения.

Файл	Что внутри
[462] `extract-phi4-mini-20261022-222356.jsonl`	Сырые ответы модели phi4-mini при извлечении фактов из 12 золотых кусков текста: 12 строк (chunk_id, output — сырой JSON-текст, wall_s, число токенов, done_reason, mem_available_mib). Среднее время на кусок 174.4 с, без ошибок; num_predict 1536. Нужны, чтобы сравнить модели по качеству и скорости извлечения.
[463] `extract-qwen2.5_7b-20261022-212626.jsonl`	Сырые ответы модели qwen2.5:7b при извлечении фактов из 12 золотых кусков текста: 12 строк (chunk_id, output — сырой JSON-текст, wall_s, число токенов, done_reason, mem_available_mib). Среднее время на кусок 283.4 с, без ошибок; num_predict 1536. Нужны, чтобы сравнить модели по качеству и скорости извлечения.
[464] `extract-qwen3_4b-20261022-225938.jsonl`	Сырые ответы модели qwen3:4b (think выкл.) при извлечении фактов из 12 золотых кусков текста: 12 строк (chunk_id, output — сырой JSON-текст, wall_s, число токенов, done_reason, mem_available_mib). Среднее время на кусок 179.7 с, без ошибок; num_predict 1536. Нужны, чтобы сравнить модели по качеству и скорости извлечения.
[465] `extract-qwen3_4b-instruct-20261022-185328.jsonl`	Сырые ответы модели qwen3:4b-instruct при извлечении фактов из 12 золотых кусков текста: 12 строк (chunk_id, output — сырой JSON-текст, wall_s, число токенов, done_reason, mem_available_mib). Среднее время на кусок 235.3 с, один ответ оборван по лимиту токенов; num_predict 1536. Нужны, чтобы сравнить модели по качеству и скорости извлечения.
[466] `extract-qwen3_8b-20261022-201913.jsonl`	Сырые ответы модели qwen3:8b при извлечении фактов из 12 золотых кусков текста: 12 строк (chunk_id, output — сырой JSON-текст, wall_s, число токенов, done_reason, mem_available_mib). Среднее время на кусок 332.0 с, без ошибок; num_predict 1536. Нужны, чтобы сравнить модели по качеству и скорости извлечения.
[467] `extractive-recall-20261027.json`	Дамп замера «извлекающего» режима: 85 вариантов (k, per, cap, order, windows) с числом покрытых вопросов, интервалом и токенами; также n=86, best_under_budget (8 лучших вариантов). Парный .md; 72 КБ.
[468] `extractive-recall-20261027.md`	Замер без LLM: какие предложения-кандидаты получает модель для выбора. Текущая настройка покрывает золотую цитату в 44/86 (51%), лучшие варианты k4/p4 — в 47/86 (55%) при пике около 700 токенов. Русских вопросов покрыто 0/7. Числа подгонялись на тех же вопросах, поэтому оптимистичны.
[469] `gold12-20261026-210753.md`	Таблица оценки рабочего извлечения (схема v2) на 12 золотых кусках для одного прогона qwen3_4b с подсказкой extract_v2_2. JSON 100%, цитаты обоснованы 100%, 152 с на кусок; F1 по сущностям 0.506, по утверждениям 0.200, по связям 0.060. Золото размечено ИИ.
[470] `gold12-20261027-server-log.txt`	Журнал запуска оценки gold12 на GPU-сервере (71 строка): команды uv run, прогресс по 12 кускам (например, «blue-carbuncle-0004 ok 8s») для qwen3:4b и qwen3:8b. В конце таблица: qwen3_4b 6.8 с на кусок, F1 по сущностям 0.588; qwen3_8b 11.3 с, 0.531.
[471] `gold12-v22.md`	Сравнение подсказок v2.1 и v2.2 для qwen3:4b на 12 золотых кусках. v2.2 быстрее (170.5 с против 233.7), F1 по утверждениям выше (0.283 против 0.200), но по сущностям F1 упал с 0.557 до 0.000. Подробный разбор — в prompt-v2-2-20261026.md.
[472] `graph-sanity-20261028.md`	Контрольные числа графа знаний по всему корпусу, построенного на GPU-сервере: 596 из 596 кусков извлечены, ~3.2 с на кусок; 2157 утверждений, 1698 событий, 743 человека, 631 место. Правила нашли 0 противоречий в 19 из 20 рассказов: события трудно сравнить (нет человека, места или времени). Единственное найденное — курируемый случай из Wisteria Lodge.
[473] `llm-20261022-171202.json`	Сырые данные того же замера модели qwen3:4b-instruct: параметры, состояние машины до и после (загрузка, температура, питание), результаты каждого прогона и замеры памяти. Читается вместе с парным .md; около 10 КБ.

Файл	Что внутри
[474] `llm-20261022-171202.md`	Замер скорости языковой модели qwen3:4b-instruct на ноутбуке без GPU: на задаче «пересказать кусок» 7.04 ток/с; TTFT 19.5 с (TTFT — время до первого слова). Затем короткий вопрос-ответ; подсказка: температура 0, seed 42, 3 измеренных прогона. Таблица также содержит пик памяти и время холодной загрузки.
[475] `llm-20261022-171654.json`	Сырые данные того же замера модели qwen3:4b: параметры, состояние машины до и после (загрузка, температура, питание), результаты каждого прогона и замеры памяти. Читается вместе с парным .md; около 10 КБ.
[476] `llm-20261022-171654.md`	Замер скорости языковой модели qwen3:4b на ноутбуке без GPU: на задаче «пересказать кусок» 7.07 ток/с; TTFT 17.2 с (TTFT — время до первого слова). Затем короткий вопрос-ответ; подсказка: температура 0, seed 42, 3 измеренных прогона. Таблица также содержит пик памяти и время холодной загрузки.
[477] `llm-20261022-172011.json`	Сырые данные того же замера модели gemma3:4b: параметры, состояние машины до и после (загрузка, температура, питание), результаты каждого прогона и замеры памяти. Читается вместе с парным .md; около 10 КБ.
[478] `llm-20261022-172011.md`	Замер скорости языковой модели gemma3:4b на ноутбуке без GPU: на задаче «пересказать кусок» 7.73 ток/с; TTFT 15.3 с (TTFT — время до первого слова). Затем короткий вопрос-ответ; подсказка: температура 0, seed 42, 3 измеренных прогона. Таблица также содержит пик памяти и время холодной загрузки.
[479] `llm-20261022-172330.json`	Сырые данные того же замера модели phi4-mini: параметры, состояние машины до и после (загрузка, температура, питание), результаты каждого прогона и замеры памяти. Читается вместе с парным .md; около 10 КБ.
[480] `llm-20261022-172330.md`	Замер скорости языковой модели phi4-mini на ноутбуке без GPU: на задаче «пересказать кусок» 7.73 ток/с; TTFT 17.1 с (TTFT — время до первого слова). Затем короткий вопрос-ответ; подсказка: температура 0, seed 42, 3 измеренных прогона. Таблица также содержит пик памяти и время холодной загрузки.
[481] `llm-20261022-172808.json`	Сырые данные того же замера модели qwen2.5:7b: параметры, состояние машины до и после (загрузка, температура, питание), результаты каждого прогона и замеры памяти. Читается вместе с парным .md; около 10 КБ.
[482] `llm-20261022-172808.md`	Замер скорости языковой модели qwen2.5:7b на ноутбуке без GPU: на задаче «пересказать кусок» 4.92 ток/с; TTFT 27.0 с (TTFT — время до первого слова). Затем короткий вопрос-ответ; подсказка: температура 0, seed 42, 3 измеренных прогона. Таблица также содержит пик памяти и время холодной загрузки.
[483] `llm-20261022-173245.json`	Сырые данные того же замера модели qwen3:8b: параметры, состояние машины до и после (загрузка, температура, питание), результаты каждого прогона и замеры памяти. Читается вместе с парным .md; около 10 КБ.
[484] `llm-20261022-173245.md`	Замер скорости языковой модели qwen3:8b на ноутбуке без GPU: на задаче «пересказать кусок» 4.34 ток/с; TTFT 29.5 с (TTFT — время до первого слова). Затем короткий вопрос-ответ; подсказка: температура 0, seed 42, 3 измеренных прогона. Таблица также содержит пик памяти и время холодной загрузки.
[485] `pre-reveal-2026-10-28-rescored.json`	Данные теста «до разоблачения» (48 строк = 16 вопросов × условия full, cut, bare), пересчитанные 28.10 по главной метрике «назван виновный». Сохранены сами ответы и описание кейсов (cases, exposure). Заменяет оценку первого файла.
[486] `pre-reveal-2026-10-28-rescored.md`	Тот же тест «до разоблачения», пересчитанный: теперь главная метрика — названо ли имя виновного. Проверяет, угадывает ли система виновного, когда текста с разгадкой нет (условие B), в сравнении с полным текстом (A) и голой моделью (C). Основной отчёт этого теста; 16 вопросов в 8 рассказах, кейсы написаны ИИ.

Файл	Что внутри
[487] `pre-reveal-2026-10-28.json`	Первые сырые данные теста «до разоблачения»: 48 строк (16 вопросов × 3 условия), meta и cases. Оценка здесь только по ключевым словам разгадки; позднее пересчитана в pre-reveal-2026-10-28-rescored.json.
[488] `pre-reveal-2026-10-28.md`	Первый отчёт теста «до разоблачения»: полный текст — виновный назван 1/16, обрезанный до разоблачения 0/16, голая модель 1/16 (при 13/16 отказов). Помечен как устаревший по оценке; тот же ответ пересчитан в rescored.md. Единственная метрика тут — ключевые слова из текста разгадки.
[489] `presence-pass-server-20261029.md`	Первый полный прогон «presence pass» (кто где был) на GPU-сервере для задачи #20: qwen3:8b, 10.3 с на кусок, и qwen3:4b-instruct, 7.1 с. После проверки остаётся 16 (8b) и 7 (4b-instruct) фактов из 537/587; ни один не имеет «дня» истории. Золото мало (4 строгих факта), поэтому число 0.857 F1 не доказывает качество.
[490] `presence21-c001-20261027.md`	Проба поиска фактов «кто где был» (подсказка presence-2.1) на трёх кусках wisteria-lodge (случай c001: Гарсия около часа ночи). qwen3:4b-instruct оставил 1 факт, и тот неверен; qwen3:8b — 11 из 15, ключевой верен, но многие приписаны не тому. Поэтому для демо взяты факты, написанные вручную.
[491] `prompt-v2-2-20261026.md`	Сравнение подсказок извлечения v2.1 и v2.2 по «готовности к символьным правилам» на 12 золотых кусках (qwen3:4b). v2.2: быстрее (170.5 против 233.7 с), участники событий заполнены у 33% (было 0%), но F1 по сущностям упал 0.557 → 0.000. Один прогон, золото размечено ИИ.
[492] `r3-semantic-dev-judge-r3j2.json`	Данные эксперимента «ИИ-судья ищет противоречия между утверждением и фактом (R3)»: 23 строки по случаям dev-набора (пары, кандидаты, вердикты contradicts), summary и meta (модель qwen3:4b-instruct, 12 пар максимум). Парный .md; 38 КБ.
[493] `r3-semantic-dev-judge-r3j2.md`	Результат семантического судьи R3J-2 на dev (23 случая): без модели правила находят 0 из 14; судья находит 5 из 14 (мягко) и 1 из 14 (строго), но ошибочно «находит» противоречие в 8 из 9 контрольных случаев (0.89). Вывод: ложных срабатываний слишком много; 978 с на всё. Настройка на dev, потому оптимистично.
[494] `retrieval-20261023-005117.json`	Сырые данные того же замера для bge-m3: args, summary по группам и per_question — ранги золотых кусков для каждого из 35 вопросов. Парный .md; около 30 КБ.
[495] `retrieval-20261023-005117.md`	Таблица поиска на уровне кусков для модели bge-m3: hit@5 = 0.89 на всех 35 вопросах, MRR 0.70. Группы вопросов — old-en, old-ru, hard, cross, all (всего 35). Попадание = золотой кусок в топ-k.
[496] `retrieval-20261023-005819.json`	Сырые данные того же замера для nomic-embed-text: args, summary по группам и per_question — ранги золотых кусков для каждого из 35 вопросов. Парный .md; около 30 КБ.
[497] `retrieval-20261023-005819.md`	Таблица поиска на уровне кусков для модели nomic-embed-text: hit@5 = 0.69, MRR 0.53. Группы вопросов — old-en, old-ru, hard, cross, all (всего 35). Попадание = золотой кусок в топ-k.
[498] `retrieval-20261023-005917.json`	Сырые данные того же замера для all-minilm: args, summary по группам и per_question — ранги золотых кусков для каждого из 35 вопросов. Парный .md; около 30 КБ.
[499] `retrieval-20261023-005917.md`	Таблица поиска на уровне кусков для модели all-minilm: hit@5 = 0.46, MRR 0.35. Группы вопросов — old-en, old-ru, hard, cross, all (всего 35). Попадание = золотой кусок в топ-k.
[500] `retrieval-20261023-011731.json`	Сырые данные того же замера для snowflake-arctic-embed: args, summary по группам и per_question — ранги золотых кусков для каждого из 35 вопросов. Парный .md; около 30 КБ.
[501] `retrieval-20261023-011731.md`	Таблица поиска на уровне кусков для модели snowflake-arctic-embed: hit@5 = 0.63, MRR 0.41, на русском 0.00. Группы вопросов — old-en, old-ru, hard, cross, all (всего 35). Попадание = золотой кусок в топ-k.

Файл	Что внутри
[502] `server-normalise-20261027.log`	Журнал нормализации графа на сервере (20 строк, по строке на рассказ): для каждого рассказа сколько событий с временем, с датой, сколько утверждений структурировано и причины, почему пары нельзя сравнить (PERSON_MISSING, NO_PLACE, DENIAL_NOT_MODELLED). Объясняет, почему правила не нашли противоречий.
[503] `server-report-20261027.md`	Автоотчёт запуска извлечения на GPU-сервере (RTX 4070, Ollama 0.40.11): секунд на кусок (6.5–7.0 при одном потоке; 3.16 при 4 потоках, выбрано), всего 596 кусков ok, оставлено 8663 пункта, отброшено 125. Проверка детерминизма с ноутбуком: 0 из 3 совпали побайтно.
[504] `test-preregistration-20261028.md`	Предварительная регистрация прогона на тестовом наборе, написана ДО запуска: что заморожено, главная метрика (верное поведение hybrid+graph, n=30), ожидания (25–45% для агента, bare ниже 20%) и правила (запуск один раз, результат публикуется, любым). Нужна, чтобы нельзя было подогнать числа задним числом.

18.13 docs/spike/results/entity-answer-runs/

Прогоны экспериментального режима «выбор человека для вопроса „кто сделал“» (ветка exp/entity-answer) от 29.10. Для каждого прогона пара .md и .json: dev (разные подсказки V1–V3 и выключенный контроль), золотой вопрос g007 и единственный замороженный прогон на отложенных историях.

Файл	Что внутри
[505] `pre-reveal-2026-10-29-dev-off.json`	Сырые данные того же прогона (dev-набор (5 вопросов, 3 рассказа), режим выключен): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[506] `pre-reveal-2026-10-29-dev-off.md`	Отчёт одного прогона теста «до разоблачения»: dev-набор (5 вопросов, 3 рассказа), режим выключен. Модель qwen3:4b-instruct, seed 42, temperature 0; контроль: виновный назван 0 из 5, отказов 1. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[507] `pre-reveal-2026-10-29-dev-on-v1.json`	Сырые данные того же прогона (dev-набор, режим включён, подсказка V1): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[508] `pre-reveal-2026-10-29-dev-on-v1.md`	Отчёт одного прогона теста «до разоблачения»: dev-набор, режим включён, подсказка V1. Модель qwen3:4b-instruct, seed 42, temperature 0; виновный назван 0 из 5, отказов 3 из 5. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[509] `pre-reveal-2026-10-29-dev-on-v2.json`	Сырые данные того же прогона (dev-набор, режим включён, подсказка V2): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[510] `pre-reveal-2026-10-29-dev-on-v2.md`	Отчёт одного прогона теста «до разоблачения»: dev-набор, режим включён, подсказка V2. Модель qwen3:4b-instruct, seed 42, temperature 0; виновный назван 0 из 5, отказались все 5. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[511] `pre-reveal-2026-10-29-dev-on-v3.json`	Сырые данные того же прогона (dev-набор, режим включён, подсказка V3): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.

Файл	Что внутри
[512] `pre-reveal-2026-10-29-dev-on-v3.md`	Отчёт одного прогона теста «до разоблачения»: dev-набор, режим включён, подсказка V3. Модель qwen3:4b-instruct, seed 42, temperature 0; виновный назван 0 из 5, отказов 4 из 5. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[513] `pre-reveal-2026-10-29-dev-untriggered-off.json`	Сырые данные того же прогона (3 dev-вопроса, на которые режим не срабатывает, режим выключен): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[514] `pre-reveal-2026-10-29-dev-untriggered-off.md`	Отчёт одного прогона теста «до разоблачения»: 3 dev-вопроса, на которые режим не срабатывает, режим выключен. Модель qwen3:4b-instruct, seed 42, temperature 0; виновный назван 0 из 3, отказов 1. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[515] `pre-reveal-2026-10-29-dev-untriggered-on-v3.json`	Сырые данные того же прогона (те же 3 вопроса, режим включён (V3)): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[516] `pre-reveal-2026-10-29-dev-untriggered-on-v3.md`	Отчёт одного прогона теста «до разоблачения»: те же 3 вопроса, режим включён (V3). Модель qwen3:4b-instruct, seed 42, temperature 0; результат совпадает с выключенным: режим не вмешался. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[517] `pre-reveal-2026-10-29-golden-off.json`	Сырые данные того же прогона (золотой вопрос g007, режим выключен): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[518] `pre-reveal-2026-10-29-golden-off.md`	Отчёт одного прогона теста «до разоблачения»: золотой вопрос g007, режим выключен. Модель qwen3:4b-instruct, seed 42, temperature 0; виновный не назван (0 из 1). Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[519] `pre-reveal-2026-10-29-golden-on-v1.json`	Сырые данные того же прогона (золотой вопрос g007, V1): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[520] `pre-reveal-2026-10-29-golden-on-v1.md`	Отчёт одного прогона теста «до разоблачения»: золотой вопрос g007, V1. Модель qwen3:4b-instruct, seed 42, temperature 0; отказ, виновный не назван. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[521] `pre-reveal-2026-10-29-golden-on-v2.json`	Сырые данные того же прогона (золотой вопрос g007, V2): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[522] `pre-reveal-2026-10-29-golden-on-v2.md`	Отчёт одного прогона теста «до разоблачения»: золотой вопрос g007, V2. Модель qwen3:4b-instruct, seed 42, temperature 0; отказ, виновный не назван. Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.
[523] `pre-reveal-2026-10-29-golden-on-v3.json`	Сырые данные того же прогона (золотой вопрос g007, V3): метаданные запуска (коммит 98067d8, настройки агента), сводка summary, строки rows с ответами и цитатами, описания кейсов и «утечки» имён до разреза. Читается вместе с парным .md.
[524] `pre-reveal-2026-10-29-golden-on-v3.md`	Отчёт одного прогона теста «до разоблачения»: золотой вопрос g007, V3. Модель qwen3:4b-instruct, seed 42, temperature 0; единственный прогон, где виновный назван (1 из 1). Таблицы с доверительными интервалами и разбор по рассказам. Часть эксперимента entity-answer.

Файл	Что внутри
[525] `pre-reveal-2026-10-29-heldout-on-v3.json`	Сырые данные отложенного прогона: 16 строк (8 вопросов × 2 условия: полный корпус и корпус, обрезанный до разоблачения), с ответом, цитатами и статусом. Плюс meta, summary с full и cut, cases и exposure (сколько кусков до разреза уже содержат имя); 32 КБ.
[526] `pre-reveal-2026-10-29-heldout-on-v3.md`	Отчёт единственного «замороженного» прогона на отложенных историях (8 вопросов, 4 рассказа; режим V3, чистое дерево 4150c4f). Полный текст: виновный назван 3/8, отказов 4/8, невинных названо 0/8; текст, обрезанный до разоблачения: виновный 0/8, невинный 2/8. Основа вердикта «режим не включать».

18.14 docs/team/

Командные заметки: рабочие пакеты, руководства по серверу и офлайн-установке, планы после показа, список моделей.

[527] `docs/team/ASSIGNMENTS.md`
Рабочие пакеты на четырёх человек: A — извлечение на мощных машинах, B — интерфейс и e2e, C — оценка качества и ручная проверка, D — запуск, упаковка и показ, плюс фокус оркестратора. Срок всего — 2026-10-27 18:00, заморозка 2026-10-28. Указаны ветки, оценки в часах, зоны ответственности.

[528] `docs/team/CLEAN_INSTALL.md`
Инструкция по установке из офлайн-архива на Linux x86_64 без сети; цель — меньше 15 минут. Нужны флешка с detective-board-offline-<ref>.tar и контрольной суммой, 16 ГБ памяти. Порты 8765 и 11435 должны быть свободны. Попадает в архив под именем INSTALL.md.

[529] `docs/team/NEXT_STEPS_RU.md`
План после dev-оценки (2026-10-28), по-русски: основные выводы (конфигурация vector тоже наш агент; много кандидатов путает 4B-модель) и этапы A (на ноутбуке), B (1-3 дня), C (сервер). Опирается на docs/spike/results/dev-failure-analysis-20261028.md и graph-sanity-20261028.md.

[530] `docs/team/README.md`
Оглавление командной папки: CONTRIBUTING, ASSIGNMENTS, models, SERVER_RUNBOOK. Показаны команды шардированного извлечения (extract_shards.py plan, extract_shard.sh, merge_extract_cache.py). Доступ к серверу вынесен в отдельный репозиторий gpu-server-access.

[531] `docs/team/SERVER_GUIDE_RU.md`
Пошаговое руководство по работе на GPU-сервере по-русски (488 строк): сеанс во вторник 27.10 с 14:30, запасной — среда 28.10. Что такое пакетный инференс, команды с пометками [НОУТ] и [СЕРВЕР], блоки «Ожидаем» и «Если не так». Русская версия SERVER_RUNBOOK.md.

[532] `docs/team/SERVER_RUNBOOK.md`
Runbook на английском для сеанса извлечения на GPU-сервере (4-6 часов): собрать архив scripts/make_server_archive.sh, перенести, запустить в tmux, проверка детерминированности, проверка качества на 12 эталонах, полный прогон, склейка, сборка графа. Правило: на общий сервер не класть GitHub-учётные данные.

[533] `docs/team/SUSPECTS_DESIGN_RU.md`
Проект (не реализован) задачи «подозреваемые по уликам до развязки» от 2026-10-28. Объясняет, почему сейчас система не решает: ответ одним предложением, граф хранит факты, а не улики, малая модель слаба в глобальных выводах. Не для показа 2026-10-30.

[534] `docs/team/TRAINING_PLAN_RU.md`
План дообучения модели-экстрактора и пути к агенту-детективу (после показа). Статус: только план. Отправная точка: у 0,7 % утверждений есть пропозиция присутствия, а правила находят 0 из 14 случаев на автоматическом графе. Таблица целей с критериями «берём / не берём».

[535] `docs/team/models.md`
Точные теги и отпечатки моделей Ollama (bge-m3, qwen3:4b, qwen3:4b-instruct, gemma3:4b, qwen3:8b и др.) с ноутбука-эталона от 2026-10-23. Объясняет, что ключ кэша извлечения зависит от модели и промпта, поэтому у всех должны совпадать теги.

18.15 docs/team/plans/

Планы отдельных веток; сейчас один файл — маршрут agent-graph-route.

[536] `docs/team/plans/agent-graph-route.md`
План ветки feat/agent-graph-route (issue #3): вопрос о противоречии или алиби должен идти агент -> граф -> FactBase -> правила, с объяснением в трассе. Описаны изменения, включая новый extract/normalise.py: нормализация времени и вычисление дней. Без LLM, проверяется на вручную построенных графах.

19 УКАЗАТЕЛЬ ФАЙЛОВ

Файлы по алфавиту путей. «№» – номер в квадратных скобках, «Стр.» – страница этого PDF.

Файл	№	Стр.
.claude/agents/agent-engineer.md	12	13
.claude/agents/backend-engineer.md	13	13
.claude/agents/code-reviewer.md	14	13
.claude/agents/data-engineer.md	15	13
.claude/agents/demo-director.md	16	13
.claude/agents/devops-engineer.md	17	13
.claude/agents/eval-engineer.md	18	14
.claude/agents/frontend-engineer.md	19	14
.claude/agents/graph-engineer.md	20	14
.claude/agents/mcp-engineer.md	21	14
.claude/agents/nlp-extraction-engineer.md	22	14
.claude/agents/performance-engineer.md	23	14
.claude/agents/product-owner.md	24	14
.claude/agents/qa-lead.md	25	15
.claude/agents/retrieval-engineer.md	26	15
.claude/agents/security-reviewer.md	27	15
.claude/agents/tech-lead.md	28	15
.claude/agents/tech-writer.md	29	15
.claude/agents/test-automation-engineer.md	30	15
.claude/agents/ux-designer.md	31	15
.github/workflows/ci.yml	10	12
.github/workflows/release.yml	11	12
.gitignore	1	11
.pre-commit-config.yaml	2	11
.python-version	3	11
CONTRIBUTING.md	4	11
README.md	5	12
THIRD_PARTY_NOTICES.md	6	12
data/LICENSES.md	32	16
data/chunks.jsonl	33	16
data/clean/abbey-grange.txt	34	16
data/clean/beryl-coronet.txt	35	16
data/clean/black-peter.txt	36	16
data/clean/blue-carbuncle.txt	37	16
data/clean/boscombe-valley-mystery.txt	38	16
data/clean/copper-beeches.txt	39	16
data/clean/crooked-man.txt	40	16
data/clean/man-with-the-twisted-lip.txt	41	17
data/clean/musgrave-ritual.txt	42	17
data/clean/naval-treaty.txt	43	17
data/clean/norwood-builder.txt	44	17
data/clean/priory-school.txt	45	17
data/clean/red-headed-league.txt	46	17
data/clean/reigate-squires.txt	47	17
data/clean/resident-patient.txt	48	17
data/clean/scandal-in-bohemia.txt	49	17
data/clean/silver-blaze.txt	50	17
data/clean/speckled-band.txt	51	17
data/clean/thor-bridge.txt	52	17
data/clean/wisteria-lodge.txt	53	17
data/curated/presence_demo.jsonl	54	18
data/curated/scenes.jsonl	55	18
data/extraction/extract-cache-qwen3_4b-extract-2.1-partial.sqlite	56	18
data/extraction/laptop-cache-qwen3_4b-extract-2.1.sqlite	57	18
docs/DISCLOSURE.md	362	57
docs/HANDOFF.md	363	57
docs/PRD.md	364	57
docs/ROADMAP.md	365	57

Файл	№	Стр.
docs/STATUS.md	366	58
docs/acceptance/acceptance.md	374	59
docs/adr/0000-template.md	375	59
docs/adr/0001-repo-and-tooling.md	376	59
docs/adr/0002-storage.md	377	59
docs/adr/0003-mcp.md	378	59
docs/adr/0004-symbolic-layer.md	379	59
docs/adr/0005-web-stack.md	380	59
docs/adr/0006-embedding-model.md	381	59
docs/adr/0007-extraction-model.md	382	59
docs/adr/0008-answer-model.md	383	59
docs/adr/0009-extractive-answers.md	384	60
docs/adr/0010-rule-first-answers.md	385	60
docs/adr/0011-presence-pass-and-curated-facts.md	386	60
docs/adr/0012-extraction-prompt-v2-1-plus-repair.md	387	60
docs/adr/README.md	388	60
docs/api.md	367	58
docs/architecture/c4.md	389	60
docs/backlog.md	368	58
docs/data-schema.md	369	58
docs/demo/README.md	390	60
docs/demo/STAGE_CHECKLIST.md	391	60
docs/demo/bare-vs-agent.md	392	60
docs/demo/jury-faq.md	393	61
docs/demo/rehearsals.md	394	61
docs/design/board-design.md	395	61
docs/design/contrast_check.py	396	61
docs/design/demo-script.md	397	61
docs/design/screens/1-empty.jpg	399	61
docs/design/screens/2-mid-run-prefill.jpg	400	61
docs/design/screens/3-final-contradiction.jpg	401	62
docs/design/tokens.css	398	61
docs/mcp-contracts.md	370	58
docs/perf/clean-install-20261029.md	402	62
docs/perf/laptop-20261028.json	403	62
docs/perf/laptop-20261028.md	404	62
docs/perf/laptop-20261029-3x6.json	405	62
docs/perf/laptop-20261029.md	406	62
docs/perf/ttft-levers-2026-10-26.md	407	62
docs/security.md	371	58
docs/slides/README.md	408	62
docs/slides/README.ru.md	409	62
docs/slides/deck.md	410	63
docs/slides/deck.ru.html	411	63
docs/slides/deck.ru.md	412	63
docs/slides/deck.ru.pdf	413	63
docs/slides/detective-night.css	414	63
docs/slides/detective.css	415	63
docs/slides/numbers.md	416	63
docs/spike/REPORT.md	417	63
docs/spike/db-research.md	418	63
docs/spike/results/_aborted/answers-20261023-082109-qwen3_4b.jsonl	419	64
docs/spike/results/_aborted/answers-20261023-v2-qwen3_4b-instruct.jsonl	420	64
docs/spike/results/ab-prompt-v2_1.md	421	64
docs/spike/results/answers-20261023-v3-BARE-qwen3_4b-instruct.jsonl	422	64
docs/spike/results/answers-20261023-v3-gemma3_4b.jsonl	423	64
docs/spike/results/answers-20261023-v3-qwen3_4b-instruct.jsonl	424	64
docs/spike/results/answers-20261023-v3-qwen3_8b.jsonl	425	64
docs/spike/results/answers-20261023-v3.md	426	64
docs/spike/results/api12-20261027-extractive.md	427	64
docs/spike/results/api12-20261027-generative.md	428	65
docs/spike/results/arms-20261023-080849.json	429	65
docs/spike/results/arms-20261023-080849.md	430	65
docs/spike/results/curated-review-2026-10-29.md	431	65
docs/spike/results/dev-failure-analysis-20261028.md	432	65
docs/spike/results/embed-20261022-180354.json	433	65

Файл	№	Стр.
docs/spike/results/embed-20261022-180354.md	434	65
docs/spike/results/embed-20261022-181410.json	435	65
docs/spike/results/embed-20261022-181410.md	436	65
docs/spike/results/embed-20261022-182431.json	437	65
docs/spike/results/embed-20261022-182431.md	438	65
docs/spike/results/embed-20261022-182604.json	439	66
docs/spike/results/embed-20261022-182604.md	440	66
docs/spike/results/embed-20261022-185253.json	441	66
docs/spike/results/embed-20261022-185253.md	442	66
docs/spike/results/entity-answer-2026-10-29.md	443	66
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-off.json	505	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-off.md	506	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-on-v1.json	507	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-on-v1.md	508	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-on-v2.json	509	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-on-v2.md	510	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-on-v3.json	511	71
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-on-v3.md	512	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-untriggered-off.json	513	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-untriggered-off.md	514	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-untriggered-on-v3.json	515	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-dev-untriggered-on-v3.md	516	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-off.json	517	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-off.md	518	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-on-v1.json	519	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-on-v1.md	520	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-on-v2.json	521	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-on-v2.md	522	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-on-v3.json	523	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-golden-on-v3.md	524	72
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-heldout-on-v3.json	525	73
docs/spike/results/entity-answer-runs/pre-reveal-2026-10-29-heldout-on-v3.md	526	73
docs/spike/results/eval-2026-10-28-dev-4bceaf8-dcf42c8a3eb8-162624.json	444	66
docs/spike/results/eval-2026-10-28-dev-4bceaf8-dcf42c8a3eb8-162624.md	445	66
docs/spike/results/eval-2026-10-28-dev-4bceaf8-dcf42c8a3eb8-162624.svg	446	66
docs/spike/results/eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202-rescored.json	447	66
docs/spike/results/eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202-rescored.md	448	66
docs/spike/results/eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202-rescored.svg	449	66
docs/spike/results/eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202.json	450	66
docs/spike/results/eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202.md	451	67
docs/spike/results/eval-2026-10-28-dev-ab945e8-78b3bbb3f2a6-145202.svg	452	67
docs/spike/results/eval-2026-10-28-dev-f8f1d0c-e1d16fd71889-170618.json	453	67
docs/spike/results/eval-2026-10-28-dev-f8f1d0c-e1d16fd71889-170618.md	454	67
docs/spike/results/eval-2026-10-28-dev-f8f1d0c-e1d16fd71889-170618.svg	455	67
docs/spike/results/eval-2026-10-28-test-7b7b197-7857839ed9be-175321.json	456	67
docs/spike/results/eval-2026-10-28-test-7b7b197-7857839ed9be-175321.md	457	67
docs/spike/results/eval-2026-10-28-test-7b7b197-7857839ed9be-175321.svg	458	67
docs/spike/results/evidence-20261023-093201.json	459	67
docs/spike/results/evidence-20261023-093201.md	460	67
docs/spike/results/extract-gemma3_4b-20261022-194124.jsonl	461	67
docs/spike/results/extract-phi4-mini-20261022-222356.jsonl	462	68
docs/spike/results/extract-qwen2.5_7b-20261022-212626.jsonl	463	68
docs/spike/results/extract-qwen3_4b-20261022-225938.jsonl	464	68
docs/spike/results/extract-qwen3_4b-instruct-20261022-185328.jsonl	465	68
docs/spike/results/extract-qwen3_8b-20261022-201913.jsonl	466	68
docs/spike/results/extractive-recall-20261027.json	467	68
docs/spike/results/extractive-recall-20261027.md	468	68
docs/spike/results/gold12-20261026-210753.md	469	68
docs/spike/results/gold12-20261027-server-log.txt	470	68
docs/spike/results/gold12-v22.md	471	68
docs/spike/results/graph-sanity-20261028.md	472	68
docs/spike/results/llm-20261022-171202.json	473	68
docs/spike/results/llm-20261022-171202.md	474	69
docs/spike/results/llm-20261022-171654.json	475	69
docs/spike/results/llm-20261022-171654.md	476	69

Файл	№	Стр.
docs/spike/results/llm-20261022-172011.json	477	69
docs/spike/results/llm-20261022-172011.md	478	69
docs/spike/results/llm-20261022-172330.json	479	69
docs/spike/results/llm-20261022-172330.md	480	69
docs/spike/results/llm-20261022-172808.json	481	69
docs/spike/results/llm-20261022-172808.md	482	69
docs/spike/results/llm-20261022-173245.json	483	69
docs/spike/results/llm-20261022-173245.md	484	69
docs/spike/results/pre-reveal-2026-10-28-rescored.json	485	69
docs/spike/results/pre-reveal-2026-10-28-rescored.md	486	69
docs/spike/results/pre-reveal-2026-10-28.json	487	70
docs/spike/results/pre-reveal-2026-10-28.md	488	70
docs/spike/results/presence-pass-server-20261029.md	489	70
docs/spike/results/presence21-c001-20261027.md	490	70
docs/spike/results/prompt-v2-2-20261026.md	491	70
docs/spike/results/r3-semantic-dev-judge-r3j2.json	492	70
docs/spike/results/r3-semantic-dev-judge-r3j2.md	493	70
docs/spike/results/retrieval-20261023-005117.json	494	70
docs/spike/results/retrieval-20261023-005117.md	495	70
docs/spike/results/retrieval-20261023-005819.json	496	70
docs/spike/results/retrieval-20261023-005819.md	497	70
docs/spike/results/retrieval-20261023-005917.json	498	70
docs/spike/results/retrieval-20261023-005917.md	499	70
docs/spike/results/retrieval-20261023-011731.json	500	70
docs/spike/results/retrieval-20261023-011731.md	501	70
docs/spike/results/server-normalise-20261027.log	502	71
docs/spike/results/server-report-20261027.md	503	71
docs/spike/results/test-preregistration-20261028.md	504	71
docs/symbolic-layer.md	372	58
docs/team/ASSIGNMENTS.md	527	73
docs/team/CLEAN_INSTALL.md	528	73
docs/team/NEXT_STEPS_RU.md	529	73
docs/team/README.md	530	73
docs/team/SERVER_GUIDE_RU.md	531	73
docs/team/SERVER_RUNBOOK.md	532	73
docs/team/SUSPECTS_DESIGN_RU.md	533	73
docs/team/TRAINING_PLAN_RU.md	534	74
docs/team/models.md	535	74
docs/team/plans/agent-graph-route.md	536	74
docs/test-plan.md	373	58
eval/evidence_in_prompt.py	252	43
eval/extraction_gold/README.md	264	45
eval/extraction_gold/blue-carbuncle-0004.json	265	45
eval/extraction_gold/boscombe-valley-mystery-0006.json	266	45
eval/extraction_gold/boscombe-valley-mystery-0007.json	267	45
eval/extraction_gold/crooked-man-0014.json	268	45
eval/extraction_gold/naval-treaty-0007.json	269	46
eval/extraction_gold/norwood-builder-0014.json	270	46
eval/extraction_gold/red-headed-league-0003.json	271	46
eval/extraction_gold/reigate-squires-0004.json	272	46
eval/extraction_gold/reigate-squires-0009.json	273	46
eval/extraction_gold/silver-blaze-0004.json	274	46
eval/extraction_gold/silver-blaze-0018.json	275	46
eval/extraction_gold/speckled-band-0020.json	276	46
eval/extractive_recall.py	253	43
eval/golden/README.md	277	47
eval/golden/contradictions.jsonl	278	47
eval/golden/questions.jsonl	279	47
eval/golden/split.json	280	47
eval/metrics.py	254	43
eval/pre_reveal.py	255	43
eval/pre_reveal/cases.jsonl	281	47
eval/r3_semantic.py	256	44
eval/run_answer_check.py	257	44
eval/run_eval.py	258	44
eval/sentence_recall.py	259	44

Файл	№	Стр.
eval/spike_chunk_gold.json	260	44
eval/spike_questions.jsonl	261	44
eval/spike_questions_hard.jsonl	262	45
eval/window_recall.py	263	45
justfile	7	12
prompts/extract/extract_v2.txt	58	18
prompts/extract/extract_v2_1.txt	59	18
prompts/extract/extract_v2_2.txt	60	19
prompts/extract/presence_v1.txt	61	19
prompts/extract/presence_v2.txt	62	19
prompts/extract/presence_verify_v1.txt	63	19
pyproject.toml	8	12
scripts/check_vendor_hashes.py	282	48
scripts/corpus/fetch.py	300	50
scripts/corpus/ingest.py	301	50
scripts/eval/repair_eval.py	302	50
scripts/eval/split_check.py	303	50
scripts/eval/validate_contradictions.py	304	51
scripts/eval/validate_golden.py	305	51
scripts/eval/validate_pre_reveal.py	306	51
scripts/extract_shard.sh	283	48
scripts/extract_shards.py	284	48
scripts/make_server_archive.sh	285	48
scripts/merge_extract_cache.py	286	48
scripts/pack_offline.sh	287	48
scripts/preflight.py	288	48
scripts/record_run.py	289	49
scripts/replay_index.py	290	49
scripts/run_extraction_priority.sh	291	49
scripts/server_bootstrap.sh	292	49
scripts/server_gold12.py	293	49
scripts/server_gold12.sh	294	49
scripts/server_presence.sh	295	49
scripts/server_progress.sh	296	49
scripts/server_report.py	297	49
scripts/server_run_all.sh	298	50
scripts/spike/_common.py	307	51
scripts/spike/bench_arms.py	308	51
scripts/spike/bench_embed.py	309	51
scripts/spike/bench_llm.py	310	51
scripts/spike/bench_retrieval.py	311	52
scripts/spike/bench_rss.py	312	52
scripts/spike/chunk_gold.py	313	52
scripts/spike/overlap_metric.py	314	52
scripts/spike/presence_eval.py	315	52
scripts/spike/run_extraction.py	316	52
scripts/spike/run_extraction_all.sh	317	52
scripts/spike/score_extraction.py	318	52
scripts/spike/score_pipeline.py	319	52
scripts/spike/symbolic_readiness.py	320	53
scripts/verify_offline.sh	299	50
src/detective_board/__init__.py	64	19
src/detective_board/agent/__init__.py	65	19
src/detective_board/agent/agent.py	66	19
src/detective_board/agent/answer.py	67	20
src/detective_board/agent/backends.py	68	20
src/detective_board/agent/config.py	69	20
src/detective_board/agent/context.py	70	20
src/detective_board/agent/events.py	71	20
src/detective_board/agent/extractive.py	72	20
src/detective_board/agent/honesty.py	73	20
src/detective_board/agent/linking.py	74	20
src/detective_board/agent/llm.py	75	20
src/detective_board/agent/memory.py	76	20
src/detective_board/agent/presence_answer.py	77	21
src/detective_board/agent/retrieval_step.py	78	21

Файл	№	Стр.
src/detective_board/agent/router.py	79	21
src/detective_board/agent/rule_first.py	80	21
src/detective_board/agent/rules_client.py	81	21
src/detective_board/agent/scene_answer.py	82	21
src/detective_board/agent/semantic_r3.py	83	21
src/detective_board/agent/translate.py	84	21
src/detective_board/agent/window.py	85	21
src/detective_board/api/__init__.py	86	22
src/detective_board/api/__main__.py	87	22
src/detective_board/api/app.py	88	22
src/detective_board/api/config.py	89	22
src/detective_board/api/fixture.py	90	22
src/detective_board/api/graph_view.py	91	22
src/detective_board/api/runtime.py	92	22
src/detective_board/api/services.py	93	22
src/detective_board/api/sse.py	94	23
src/detective_board/api/tool_client.py	95	23
src/detective_board/api/warmup.py	96	23
src/detective_board/extract/__init__.py	97	23
src/detective_board/extract/cache.py	98	23
src/detective_board/extract/curated.py	99	23
src/detective_board/extract/curated_schema.py	100	23
src/detective_board/extract/ground.py	101	23
src/detective_board/extract/ingest.py	102	23
src/detective_board/extract/llm.py	103	24
src/detective_board/extract/normalise.py	104	24
src/detective_board/extract/parse.py	105	24
src/detective_board/extract/pipeline.py	106	24
src/detective_board/extract/presence.py	107	24
src/detective_board/extract/presence_candidates.py	108	24
src/detective_board/extract/presence_schema.py	109	24
src/detective_board/extract/presence_verify.py	110	24
src/detective_board/extract/repair.py	111	24
src/detective_board/extract/run.py	112	24
src/detective_board/extract/scenes.py	113	24
src/detective_board/extract/schema.py	114	25
src/detective_board/extract/schema_v2.py	115	25
src/detective_board/graph/__init__.py	116	25
src/detective_board/graph/anchors.py	117	25
src/detective_board/graph/facts_source.py	118	25
src/detective_board/graph/normalize.py	119	25
src/detective_board/graph/results.py	120	25
src/detective_board/graph/schema.py	121	25
src/detective_board/graph/store.py	122	25
src/detective_board/graph/timenorm.py	123	25
src/detective_board/index/__init__.py	124	26
src/detective_board/index/build.py	125	26
src/detective_board/index/sentences.py	126	26
src/detective_board/mcp/__init__.py	127	26
src/detective_board/mcp/client.py	128	26
src/detective_board/mcp/errors.py	129	26
src/detective_board/mcp/sandbox.py	130	26
src/detective_board/mcp/server.py	131	26
src/detective_board/mcp/source.py	132	27
src/detective_board/mcp/tools.py	133	27
src/detective_board/retrieval/__init__.py	134	27
src/detective_board/retrieval/config.py	135	27
src/detective_board/retrieval/embedder.py	136	27
src/detective_board/retrieval/loader.py	137	27
src/detective_board/retrieval/rerank.py	138	27
src/detective_board/retrieval/search.py	139	27
src/detective_board/retrieval/sentences.py	140	27
src/detective_board/retrieval/store.py	141	28
src/detective_board/symbolic/__init__.py	142	28
src/detective_board/symbolic/_explain.py	143	28
src/detective_board/symbolic/api.py	144	28

Файл	№	Стр.
src/detective_board/symbolic/facts.py	145	28
src/detective_board/symbolic/model.py	146	28
src/detective_board/symbolic/places.py	147	28
src/detective_board/symbolic/rules/__init__.py	149	28
src/detective_board/symbolic/rules/r1_presence.py	150	29
src/detective_board/symbolic/rules/r2_order.py	151	29
src/detective_board/symbolic/rules/r3_claims.py	152	29
src/detective_board/symbolic/time.py	148	28
tests/__init__.py	153	29
tests/agent/__init__.py	165	31
tests/agent/conftest.py	166	31
tests/agent/test_agent.py	167	31
tests/agent/test_compaction.py	168	31
tests/agent/test_extractive.py	169	31
tests/agent/test_extractive_sentences.py	170	31
tests/agent/test_graph_route.py	171	32
tests/agent/test_linking.py	172	32
tests/agent/test_presence_table.py	173	32
tests/agent/test_rule_first.py	174	32
tests/agent/test_scene_board.py	175	32
tests/agent/test_security.py	176	32
tests/agent/test_semantic_r3.py	177	33
tests/agent/test_translate.py	178	33
tests/api/__init__.py	179	33
tests/api/conftest.py	180	33
tests/api/helpers.py	181	33
tests/api/test_ask.py	182	33
tests/api/test_demo_rules.py	183	33
tests/api/test_endpoints.py	184	34
tests/api/test_hardening.py	185	34
tests/api/test_paths.py	186	34
tests/api/test_stdio_default.py	187	34
tests/api/test_units.py	188	34
tests/api/test_warmup.py	189	34
tests/curated/__init__.py	190	34
tests/curated/test_curated.py	191	35
tests/curated/test_review.py	192	35
tests/curated/test_scenes.py	193	35
tests/eval/__init__.py	194	35
tests/eval/conftest.py	195	35
tests/eval/test_metrics.py	196	35
tests/eval/test_pre_reveal.py	197	35
tests/eval/test_run_eval.py	198	36
tests/extract/__init__.py	199	36
tests/extract/conftest.py	200	36
tests/extract/test_ground.py	201	36
tests/extract/test_normalise.py	202	36
tests/extract/test_parse.py	203	36
tests/extract/test_pipeline.py	204	36
tests/extract/test_repair.py	205	37
tests/fixtures/symbolic/holmes_mini.json	206	37
tests/graph/__init__.py	207	37
tests/graph/conftest.py	208	37
tests/graph/test_fact_source.py	209	37
tests/graph/test_normalize_time_anchors.py	210	37
tests/graph/test_queries.py	211	37
tests/graph/test_real_quotes.py	212	38
tests/graph/test_store.py	213	38
tests/index/__init__.py	214	38
tests/index/conftest.py	215	38
tests/index/test_build.py	216	38
tests/index/test_sentences_build.py	217	38
tests/mcp/__init__.py	218	38
tests/mcp/conftest.py	219	38
tests/mcp/test_graph_source.py	220	39
tests/mcp/test_hardening.py	221	39

Файл	№	Стр.
tests/mcp/test_presence_table.py	222	39
tests/mcp/test_sandbox.py	223	39
tests/mcp/test_scene_board.py	224	39
tests/mcp/test_stdio.py	225	39
tests/mcp/test_tools.py	226	39
tests/presence/__init__.py	227	40
tests/presence/helpers.py	228	40
tests/presence/test_presence_candidates.py	229	40
tests/presence/test_presence_eval.py	230	40
tests/presence/test_presence_extract.py	231	40
tests/presence/test_presence_graph.py	232	40
tests/presence/test_presence_verify.py	233	40
tests/retrieval/__init__.py	234	41
tests/retrieval/conftest.py	235	41
tests/retrieval/test_retrieval.py	236	41
tests/retrieval/test_sentences.py	237	41
tests/symbolic/__init__.py	238	41
tests/symbolic/conftest.py	239	41
tests/symbolic/symbolic_helpers.py	240	41
tests/symbolic/test_all.py	241	41
tests/symbolic/test_r1.py	242	41
tests/symbolic/test_r2.py	243	41
tests/symbolic/test_r3.py	244	42
tests/test_bench_retrieval.py	154	29
tests/test_extract_shards.py	155	29
tests/test_ingest.py	156	29
tests/test_offline_scripts.py	157	30
tests/test_preflight.py	158	30
tests/test_score_extraction.py	159	30
tests/test_score_pipeline.py	160	30
tests/test_server_gold12.py	161	30
tests/test_server_report.py	162	30
tests/test_symbolic_readiness.py	163	30
tests/test_vendor_hashes.py	164	30
tests/web_js/board.test.mjs	245	42
tests/web_js/demo.test.mjs	246	42
tests/web_js/highlight.test.mjs	247	42
tests/web_js/replay.test.mjs	248	42
tests/web_js/sse.test.mjs	249	42
tests/web_js/state.test.mjs	250	43
tests/web_js/trace_api.test.mjs	251	43
uv.lock	9	12
web/css/app.css	321	53
web/css/fonts.css	322	53
web/css/tokens.css	323	53
web/favicon.svg	324	53
web/index.html	325	53
web/js/api.js	326	54
web/js/app.js	327	54
web/js/board-art.js	328	54
web/js/demo.js	329	54
web/js/graph.js	330	54
web/js/highlight.js	331	54
web/js/layout.js	332	54
web/js/replay.js	333	54
web/js/sse.js	334	54
web/js/state.js	335	55
web/js/trace.js	336	55
web/replay/fake-mccarthy.json	337	55
web/replay/fake-refusal.json	338	55
web/replay/fake-speckled-band.json	339	55
web/replay/fake-swamp-adder.json	340	55
web/replay/index.json	341	55
web/replay/real-abbey-grange-r1-curated.json	342	55
web/replay/real-boscombe-control.json	343	55
web/replay/real-g001-agent.json	344	55

Файл	№	Стр.
web/replay/real-garcia-timeline.json	345	55
web/replay/real-refusal-baskerville.json	346	56
web/replay/real-refusal-fee.json	347	56
web/replay/real-scene-board.json	348	56
web/replay/real-wisteria-r1-curated.json	349	56
web/replay/synthetic-derived.json	350	56
web/vendor/README.md	351	56
web/vendor/cytoscape.LICENSE	352	56
web/vendor/cytoscape.esm.min.mjs	353	56
web/vendor/fonts/EBGaramond-OFL.txt	354	56
web/vendor/fonts/IMFellEnglish-OFL.txt	355	56
web/vendor/fonts/SpecialElite-LICENSE.txt	356	56
web/vendor/fonts/eb-garamond-italic-latin.woff2	357	57
web/vendor/fonts/eb-garamond-latin.woff2	358	57
web/vendor/fonts/im-fell-english-italic-latin.woff2	359	57
web/vendor/fonts/im-fell-english-latin.woff2	360	57
web/vendor/fonts/special-elite-latin.woff2	361	57

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	**Номера листов (страниц)**			**Всего листов**	**№ докум.**	**Подп.**	**Дата**
	изменённых	замененных	новых				